Evaluating Transport Layer Congestion Control Algorithms: A Comprehensive Survey

Christian L. Vielhaus, Caspar von Lengerke, *Graduate Student Member, IEEE*, Vincent Latzko, Justus Rischke, Martin Reisslein, *Fellow, IEEE*, and Frank H. P. Fitzek, *Fellow, IEEE*

***Abstract*—Congestion Control (CC) limits the transmission rate of data packets in the transport layer protocol, e.g., in TCP and QUIC, of a sending endpoint so as to mitigate the congestion in packet-switched communication networks. Numerous studies have developed and evaluated Congestion Control Algorithms (CCAs) for a wide range of network types. Prior surveys have covered the functioning of the existing CCAs and their performance characteristics. In contrast, this survey focuses on the quantitative *evaluation* of CCAs; specifically, we comprehensively survey the CCA *evaluation methodologies*, i.e., the *why* and *how* of CCA evaluations. We define a CCA *evaluation* as a combination of a prescribed evaluation scenario with prescribed evaluation metrics. In turn, we define an evaluation scenario as a combination of a prescribed network (e.g., path) model and a particular traffic model. We present comprehensive sets of network and traffic model characteristics (i.e., network scenario characteristics) that can influence the performance of CCAs, as well as a comprehensive set of evaluation metrics for measuring the performance of CCAs. We taxonomize CCA evaluations into the main categories: Steady-state, transient-state, network variation, dynamic (e.g., wireless) network, and competition evaluations. We place and survey the existing CCA evaluation studies in this taxonomy to uncover the main trends and shortcomings of the CCA evaluation field to date. To facilitate the comprehensive rigorous evaluation of CCAs, we have developed the publicly available CCA evaluation framework ccperf, which incorporates the insights from our survey. We also outline future research directions to further advance the comprehensive rigorous evaluation of CCAs.**

***Index Terms*—Bufferbloat, congestion, evaluation, fairness, Internet, operating point, throughput, traffic flows, wireless networks.**

Received 31 August 2024; revised 11 December 2024; accepted 20 January 2025. Date of publication 23 January 2025; date of current version 2 January 2026. This work was supported in part by the German Research Foundation (DFG, Deutsche Forschungsgemeinschaft) as part of Germany's Excellence Strategy—EXC 2050/1—Cluster of Excellence "Centre for Tactile Internet with Human-in-the-Loop" (CeTI) of Technische Universität Dresden under Project 390696704; in part by the German Ministry for Education and Research (BMBF) in DAAD [School of Embedded and Composite AI (SECAI)] as part of the Program Konrad Zuse Schools of Excellence in Artificial Intelligence under Project 57616814; and in part by the Federal Ministry of Education and Research of Germany in the Programme "Souverän. Digital. Vernetzt." Joint Project 6G-life under Project 16KISK001K. *(Corresponding author: Martin Reisslein.)*

Christian L. Vielhaus, Caspar von Lengerke, Vincent Latzko, Justus Rischke, and Frank H. P. Fitzek are with the Centre for Tactile Internet with Human-in-the-Loop, Technische Universität Dresden, 01062 Dresden, Germany, and also with the Deutsche Telekom Chair of Communication Networks, 5G Lab Germany, Technical University of Dresden, 01069 Dresden, Germany (e-mail: christian.vielhaus@tu-dresden.de; caspar.lengerke@tu-dresden.de; vincent.latzko@tu-dresden.de; justus.rischke@tu-dresden.de; frank.fitzek@tu-dresden.de).

Martin Reisslein is with the School of Electrical, Computer, and Energy Engineering, Arizona State University, Tempe, AZ 85287 USA (e-mail: reisslein@asu.edu).

Digital Object Identifier 10.1109/COMST.2025.3533303

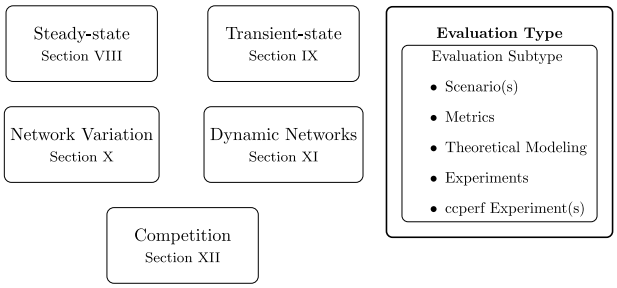
I. Introduction

CONGESTION Control (CC) is an important component of prominent transport layer protocols, such as the Transmission Control Protocol (TCP) [1], [2] and QUIC [3]. CC throttles the data packet transmissions of the sending endpoint (host) of a packet-switched network, such as the Internet, so as to prevent congestion along the transport path, such as in the buffers of Internet Protocol (IP) routers in the network [4]. Following the end-to-end Internet design principle [5], [6], [7], the CC is commonly conducted by algorithms operating at the transport protocol layer in the sending and receiving hosts without assistance from the packet switches (network nodes).

The design and evaluation of CCAs are challenging because the CCAs have to cope with a wide range of scenarios. Commonly, an evaluation scenario is defined to encompass the network model characteristics (which include the network topology and the network path parameters, e.g., bottleneck bitrate and buffer size) and the traffic model (packet traffic characteristics). Performance trade-offs may only become visible by examining an extensive set of performance metrics across a wide range of specific evaluation scenarios. Over the last decades, numerous CCAs have been proposed and evaluated in the respective studies for various scenarios and with various metrics. While there have been some prior efforts to establish common evaluation metrics, simulation frameworks, and testbeds for CC research [8], [9], [10], [11], [12], [13], [14], to the best of our knowledge, an up-to-date CC evaluation survey that comprehensively covers both the varied evaluation scenarios (i.e., varied network and traffic models) as well as the evaluation metrics is missing.

To address this gap in the CC literature, this article provides an up-to-date comprehensive survey of evaluations that have been considered in the CCA studies to date. Throughout, we focus on single-path CCAs (the evaluation of multipath CCAs is left for future work). Our survey of the CC evaluations covers exhaustive sets of network model (network topology and network path) characteristics, packet traffic model characteristics, and quantitative CC evaluation metrics. Importantly, a meaningful CCA evaluation needs to examine appropriate network and traffic models with a matching appropriate set of evaluation metrics. The comprehensive survey of these meaningful evaluations is the main focus of this article and is conducted with a novel taxonomy, cf. Fig. 1 and Table V. Our comprehensive survey informs the designers of CCAs about the comprehensive evaluation methodology that should

1553-877X © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.



(a) Evaluation categories (= core survey sections)

(b) Structure of each core survey section.

Fig. 1. Illustration of our taxonomy of the existing Congestion Control Algorithm (CCA) evaluations; the main taxonomy categories map to core survey sections.

ideally be followed for CCA evaluations. To facilitate rigorous comprehensive CCA evaluations, we introduce the CC performance (ccperf) tool, which provides all surveyed evaluations in a unified publicly available benchmarking tool for CCA research [15].

A. Related Tutorials and Surveys

The existing tutorials and surveys on CCAs have mainly focused on the operational CCA mechanisms and the related performance aspects that were obtained for various evaluation scenarios and metrics; in contrast, our survey focuses on comprehensively cataloging the CCA evaluation scenarios and metrics. In the 1990s, surveys covered the Asynchronous Transfer Mode (ATM) network congestion control [16], [17]. First comprehensive surveys on Internet congestion control started to appear in the early 2000s [18], [19], [20], [21]. Subsequently, an extensive survey literature focused on CCAs for particular wireless network types, such as wireless sensor networks [22], [23], [24], [25], [26], [27], [28], [29], mobile ad hoc networks [30], [31], [32], vehicular ad hod networks [33], [34], [35], [36], body area network [37], satellite networks [38], [39], and 5G wireless networks [40]. Another set of surveys covered specific wired network types, e.g., networks on chip [41], high-speed wired networks [42], named data networks [43], [44], and data center networks [45], [46]. Specific network applications and contexts have been considered in another set of CC surveys that covered multipath transport [47], multicast video applications [48], opportunistic networks [49], Internet of Things (IoT) networks [50], [51], smart grid networks [52], Tor network [53], as well as CC in Linux TCP [54] and standard TCP [55]. One recent survey focused on delay-based and hybrid TCP CC [56], while a few surveys have focused on CC mechanisms for optimizing a particular metric, namely network utility [57] and fairness in TCP CC [58], [59]. Recently, several surveys have considered CC mechanisms based on machine learning [60], [61], [62], [63], [64], [65], [66], [67], [68].

Our survey is complementary to these existing surveys on CCAs in that we focus on the evaluation methodology for CCAs. More specifically, the existing surveys have covered the questions of how the existing CCAs operate and what their performance characteristics are. In contrast, we focus on the question of *how to evaluate* CCAs. We provide, to the best of our knowledge, the first comprehensive survey of all aspects of the CC evaluation methodology. In particular, our survey encompasses the varied network and traffic models (and their parameters), as well as the evaluated CC performance metrics.

Importantly, our survey encompasses the varied evaluations that have been employed to characterize specific operational aspects of CCAs. In particular, our survey covers the varied network models (including the network topologies and path parameters), traffic models, and the various CC evaluation metrics. We define these three components of network model, traffic model, and evaluation metrics to compose an evaluation for characterizing the performance of CCAs.

We note that a few prior publications have provided overview and survey style coverage of parts of the CC evaluation methodology. The Pantheon project evaluation study [11] has focused on Internet measurements and the calibration of an emulation framework for CCA evaluation. Compared to Pantheon, our survey comprehensively covers the full scope of evaluation scenarios and metrics that have been evaluated in the existing CC literature. From our survey, we distill a taxonomy of clear definitions of evaluation categories for a comprehensive CCA evaluation. Specifically, Pantheon focused on evaluating CCAs over Internet paths and considered only a few selected emulation experiments. In contrast, we broadly survey the evaluation scenarios and metrics; whereby, the actual evaluation can be conducted in any controlled environment, e.g., in simulation or emulation. Correspondingly, our ccperf tool features a comprehensive set of evaluation scenarios and metrics, i.e., enables more comprehensive evaluations than Pantheon.

The conference papers [8], [69] and the related documents [9], [10] from the Internet Engineering Task Force (IETF) as well as the technical report [70] have begun to assemble performance metrics for CCAs. The related recent study in [14] has theoretically analyzed metrics that characterize how well a CCA fulfills a set of eight requirements (referred to as axioms). Our survey includes all the evaluation scenarios and metrics in these preceding publications, and goes beyond path parameters and metrics by surveying the evaluations that have been formed to date in the CC literature.

The conference papers [12], [13] described a simulation evaluation framework for CCAs based on the network simulator 3 (ns-3) network simulator and focused on evaluating three elementary metrics from [9]. In contrast, our ccperf benchmarking tool provides a comprehensive collection of all existing CC evaluations (combinations of network and traffic model characteristics as well as performance metrics).

B. Contributions and Structure of This Survey

1) Overview: This article provides a comprehensive survey of the existing CCAs evaluations that have been composed by combining network and traffic models with evaluation metrics. To the best of our knowledge, this article is the *first to comprehensively survey and taxonomize all existing compositions of network model, traffic model, and evaluation*

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

metrics that have been considered in the CCA literature to date. Our survey only assumes that generalist readers are familiar with the elementary principles of TCP/IP communication networks, and does not assume specialized knowledge of TCP mechanisms or congestion control. In order to enable generalist readers to comprehend the core survey sections on the evaluation experiments, we review the required concepts in tutorial background sections and appendices.

2) Section Structure: Section II gives tutorial background on congestion control (with additional details in Appendix A). Section III introduces our general structure of composing a CCA evaluation of three components, namely network model, traffic model, and the evaluation metrics. The subsequent sections elaborate these components. Sections IV and V give brief tutorial-style background on the network model and the traffic model (with additional traffic model details in Appendix B). Section VI comprehensively surveys the evaluation metrics that have been considered in the CCA evaluations studies to date, while Appendix C gives tutorial background on the measurement techniques and provides additional mathematical background on the metrics.

Section VII introduces our original ccperf evaluation framework of CCAs, which is publicly available [15]. The ccperf framework flexibly accommodates various simulators, emulators, and testbeds in the ccperf backend; the current ccperf implementation is based on an ns-3 backend. Informed by the comprehensive survey of the CCA evaluations in this article, ccperf uses a comprehensive set of evaluations.

The core survey sections present the survey of the existing compositions of CCA evaluations following the section structure illustrated in Fig. 1. We have organized this survey into the following main categories of CCA evaluations: Steady-state evaluations that evaluate the operating point of an individual long-running flow and a collection of long-running flows utilizing the same CCA (i.e., with so-called intra-protocol competition) are surveyed in Section VIII. Complementary transient-state evaluations are surveyed in Section IX.

Sections X and XI cover complementary aspects of variations in the network model. Section X covers evaluations that examine varied network parameter settings. More specifically, the network variation evaluations take place in a static network, i.e., the network parameters are static, i.e., do not depend on time. The network variation evaluations often evaluate how a certain (static) parameter setting, e.g., the queue size at the bottleneck, affects CCAs. Complementarily, Section XI covers evaluations that take place in a dynamic network (e.g., a wireless network) whose network parameters vary with time.

Section XII covers evaluations that let different CCAs compete against each other, i.e., evaluations of inter-protocol competition. More specifically, Section XII covers competition evaluations that consider different traffic patterns, e.g., on-off flow patterns, and evaluations of the notion of TCP-friendliness.

Section XIII outlines future research directions for advancing CCA evaluations. These future directions include examining and widening the range of evaluation scenarios and metrics, adapting CCA evaluation to the gamut of emerging traffic engineering approaches and specialized transport protocols, as well as the CCA evaluation in large-scale networks and the evaluation of CCAs based on learning algorithms.

3) Summary: We emphasize that this article is *not* about the findings of the evaluations of the various CCAs. Instead, this article comprehensively addresses the *why* and *how* of CCA evaluations. That is, this survey focuses on the CCA *evaluation methodology* (and selected evaluation results are presented to illustrate and justify the evaluation methodology). We comprehensively and rigorously survey the evaluations that have been conducted in CCA studies to date.

From our survey, we distill guidance for CCA evaluations and develop the clearly-structured ccperf evaluation framework. In the ccperf evaluation framework, we define a naming convention for CCA evaluations so as to facilitate the convenient use and comparison of the various CCA evaluations. We assign a unique mnemonic name to each specific evaluation (combination of network model, traffic model, and evaluation metrics), explain why the evaluation should be conducted (purpose), provide tutorial instructions on how the evaluations should be conducted (methodology, including network and traffic models), and what metrics should be monitored and reported (results). Overall, ccperf thus provide a comprehensive set of CCA evaluations with a clearly defined scope.

II. Background on Congestion Control (CC)

Congestion Control is provided by many transport layer protocols as an end-to-end service for applications. Historically, CCAs have been primarily developed in conjunction with TCP. Nowadays, other transport layer protocols, such as QUIC, implement similar CC functionality. While we mainly focus TCP in this article, most concepts apply also to other transport layer protocols, e.g., QUIC.

Network congestion was first described as a problem in the Internet in 1984 [71]. Without deploying any CCAs, the Internet operated in a state coined *congestion collapse*, in which routers were continuously overwhelmed by traffic. The queues of these overwhelmed routers were frequently full. Unable to process newly arriving packets, the nodes had no other option than to drop those arriving packets. These packet drops due to congestion led to retransmissions from sending nodes, which used Automatic Repeat Request (ARQ) mechanisms at the time. Consequently, a significant portion of packets transmitted through the Internet were retransmissions. Packets that are forwarded hop-by-hop through the Internet only to be dropped before reaching their destination waste data transmission rate (i.e., transmission bitrate, which is also commonly referred to as "bandwidth" in the CC literature) and energy resources at routers. Furthermore, applications suffer from the reduced end-to-end communication quality caused by congestion. CCAs were consequently designed to avoid congestion collapse.

The landscape of deployed CCAs in the Internet is heterogeneous and constantly changing. In 1986, TCP started with a CCA called Tahoe that was later iteratively refined to Reno and NewReno [4], [72]. For a long time, these CCAs were predominantly used by the majority of hosts connected to the

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

Internet [73], [74]. In 2011, the distribution of CCAs started to shift towards newer CCAs [75]. Most notably Cubic [76] was on the rise which later became the default TCP CCA for Linux, Windows, and MacOS. Nowadays, the majority of hosts use Cubic followed by Bottleneck Bandwidth and Round-trip time (BBR) [77], [78] and a few other less common CCAs [79]. The heterogeneity of CCAs present in real networks is a challenge for CCA designers. A newly proposed CCA should be able to coexist with common CCAs and allow an iterative adaptation; otherwise, it may not be deployable in large networks.

A. Modeling Congestion Control as a Dynamical System

Congestion Control can be modeled as a dynamical system [80], [81]. For a system with K, $K \geq 1$, flows, the state vector $\boldsymbol{x}(t) = [x_1(t), x_2(t), \ldots, x_K(t)]^{\mathrm{T}}$, where $[\,\cdot\,]^{\mathrm{T}}$ denotes the transpose of $[\,\cdot\,]$, gathers the resource allocations of the sender nodes that generate the traffic. The resource allocation of a sender can be represented by its transmission rate, throughput, sending window size, or a related variable that quantifies the amount of traffic of a sender. The CCAs running at the senders control the state variables x_k in a distributed manner. The network $\mathcal{G}(t)$, defined in Section IV, and the traffic $\mathcal{T}(t)$, defined in Section V, constitute the scenario $\mathcal{S}(t) = (\mathcal{G}(t), \mathcal{T}(t))$ of the dynamical system. The scenario $\mathcal{S}(t)$ is a time-dependent parameter of the dynamical system that sets the system conditions. The dynamical system

$$\dot{\boldsymbol{x}} = \Phi(t, \boldsymbol{x}; \mathcal{S}(t)), \quad 0 \leq t \leq T, \tag{1}$$

governs the state $\boldsymbol{x}(t)$ over time after the system starts at $t = t_0 = 0$ in the initial state $\boldsymbol{x}(t_0)$ with the initial system conditions $\mathcal{S}(t_0)$. The system ends after the duration T; however, analytically we allow infinite durations with $T \to \infty$. The evolution function Φ captures how the state $\boldsymbol{x}(t)$ continuously changes. Theoretical studies may model the evolution function and solve the differential equation to obtain $\boldsymbol{x}(t)$.

Suppose that we have a dynamical system with a static scenario $\mathcal{S}(t) = \mathcal{S}_0 = (\mathcal{G}_0, \mathcal{T}_0)$. All flows use the same CCA to control $\boldsymbol{x}(t)$. The state $\boldsymbol{x}(t)$ starts with a transient phase. After some time t_c, the transient phase may end when the system converges to an equilibrium $\boldsymbol{x}^e(t)$. This may not occur if the system is stopped prematurely or $\boldsymbol{x}(t)$ does not converge. When $\boldsymbol{x}(t)$ converges, we also say that the CCA converges to its steady-state behavior. The shape of the equilibrium depends on the scenario $\mathcal{S}(t)$ and the initial state $\boldsymbol{x}(t_0)$. Many CCAs converge to an equilibrium $\boldsymbol{x}^e(t)$ that is a periodic function. For example, a Reno flow converges to a sawtooth function as its equilibrium. The state $\boldsymbol{x}(t)$ then follows the equilibrium as long as the scenario $\mathcal{S}(t)$ remains constant.

Now suppose that at time $t = t_1 > t_c$ the scenario abruptly changes such that $\mathcal{G}(t) = \mathcal{G}_1$ and $\mathcal{T}(t) = \mathcal{T}_1$ for $t \geq t_1$. The change may be a random event, such as a flow starting or ending, or a change in link conditions. The dynamical system then enters a new transient phase with the initial state $\boldsymbol{x}(t_1)$ and initial scenario $\mathcal{S}(t_1) = (\mathcal{G}_1, \mathcal{T}_1)$. In other words, a change in the system conditions triggers a restart of the dynamical system with different initial conditions and a likely altered equilibrium solution of the system. Consequently, the state may converge to the new equilibrium after some time.

The changes of the system conditions can be so frequent that the dynamical system never converges to an equilibrium, because the duration between two consecutive system condition changes is too short. In such a scenario the dynamical system continuously operates in successive transient phases. In contrast, in a long-running evaluation with a static scenario, the initial transient phase may be negligible, because the dynamical system spends almost all of its time in the equilibrium state.

B. Congestion Control as a Distributed Resource Allocation

A dynamical system by itself does not have a notion of a beneficial evolution of its state $\boldsymbol{x}(t)$. To compare CCAs in terms of performance, performance metrics are required. A widely used formulation of CC as a distributed resource allocation problem is the optimization framework proposed by Kelly et al. [80]. Suppose we have a scenario with K flows using the same CCA and a network with M links. Let U be some utility function, $\boldsymbol{c}$ be an M-vector of link capacities, and A be a route matrix with $A_{m,k} = 1$ if flow k uses link m, and $A_{m,k} = 0$ otherwise. Then, any algorithm that solves

$$\max_{\boldsymbol{x}} \sum_{k=1}^{K} U(x_k) \quad \text{subject to } A\boldsymbol{x} \leq \boldsymbol{c}, \quad x_k \geq 0, \tag{2}$$

generates an optimal rate allocation $\boldsymbol{x}^*$ with respect to U. We omit the time variables for readability. In Section VIII, we describe that some CCAs do converge to optimal solutions for widely used utility functions in certain theoretical and practical evaluations. However, in general, no CCA can be optimal at all times, because the information about changes of the system state needs time to propagate to all senders before they can react. Nevertheless, the formulation as a resource allocation problem can be understood (interpreted) as a guideline for CCA designers to find schemes that increase a utility measure for a wide range of experiments.

C. Sender-Side and In-Network CCAs

A sender-side CCA adapts the transmission rate of a sender with the primary objective of avoiding network congestion while utilizing the available bandwidth. Typically, a sender-side CCA is employed directly at the transport layer of a sender that generates network traffic [4]. Therefore, the CC problem is mainly handled by the communication endpoints and not by nodes inside the network. This design decision is in line with two guiding principles of the Internet: the *end-to-end argument* [5] and *fate-sharing* [82]. To keep networks simple, complex algorithms should be deployed at the endpoints. In that way, each sender runs its own complex CCA instead of having network nodes run algorithms that have to scale up to thousands of hosts in a fail-safe way. Both, the TCP and QUIC transport layer protocols use sender-side CCAs and generate the majority of the traffic volume in the Internet today [83].

CCAs usually operate at the transport protocol layer in the sending host (sender, source), i.e., as so-called sender-side

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

CCAs, while the receiving host (receiver, sink, destination) simply returns acknowledgments as part of the transport layer reliable data transfer protocol and does not further assist in the CC. Placing the onus of CC mainly on the sender is a practical design choice since the goal of CC is to avoid congestion by controlling the transmission rate at the sender. Also, if the receiver were to play an active role in the CC, the receiver would need to control (or at least influence) the actions of the sender, which would require additional control communication.

TCP allows the sender to freely choose a CCA on a per-socket basis, which is further explained in Section II-F. The receiver is not informed about the chosen CCA by the sender. CCAs typically do not modify the behavior of receivers in any way. Instead, the transport layer protocol defines how a receiver processes and responds to received packets. The receiver always behaves the same, which is important to decouple the sender behavior from the receiver behavior.

Less commonly, CCAs require changes at both the sender and the receiver of a flow. These CCAs include some logic at the receiver that helps the sender with the control of the transmission rate. Usually, information within ACKnowledgements (ACKs) is returned from the receiver to the sender to inform the sender about the congestion state of the network. Based on that information, the sender controls its sending rate. Furthermore, some CCAs require changes to routers or switches. For example, eXplicit Control Protocol (XCP) lets routers communicate the congestion state to senders, which is used by the sender-side XCP algorithm to set the transmission rate [84]. Studies that propose new CCAs that require modifications to receivers, switches, or routers should always explicitly mention the respective requirements.

Active Queue Management (AQM) algorithms operate at the intermediate (in-network) nodes, where congestion takes place [85], [86]. AQM algorithms are packet schedulers deployed at queues. The algorithms control the way queues store-and-forward packets and provide early congestion signals to sender-side CCAs [85]. The interaction between the sender-side CCAs and AQM algorithms in the context of CCA evaluation is further explained in Section X-E.

D. Congestion Signals

Senders cannot observe congestion directly as it typically occurs somewhere inside the network. Instead, senders use feedback signals contained in ACKs that are returned by receivers. CCAs may process different congestion signals to make their rate decisions. The most common signals are packet loss, Round-Trip Time (RTT) measurements, and Explicit Congestion Notification (ECN) marks. In any case, the feedback in ACKs that originates at the receiver needs to be transmitted to the sender through the network. Hence, sender-side CCAs work with delayed feedback that can be outdated. Furthermore, the feedback can be noisy and ambiguous, which can lead to false conclusions and erroneous rate control decisions by senders. Packet loss at the transport layer is a widely used congestion signal. TCP has two mechanisms to detect packet loss: i) via duplicated ACKnowledgements (dupACKs) and ii) via Retransmission TimeOuts (RTOs).

1) Duplicate Acknowledgments: By adding sequence numbers to segments, TCP can use cumulative ACKs to track which segments have successfully been delivered at the receiver. An ACK acknowledges the in-order delivery of all bytes of payload data up to and including the byte number that precedes the sequence number that is contained in the ACK, i.e., the ACK specifies the byte number that the receiver expects next. When a segment is lost, the receiver cannot advance the highest acknowledged sequence number when a new segment arrives, because the packet loss created a gap in the sequence number space. Therefore, the receiver re-acknowledges the highest previously acknowledged sequence number by transmitting a dupACK. By default, TCP interprets three dupACKs as a packet loss event at the transport layer.

Loss-based CCAs solely use packet loss events as a binary congestion signal; other potential congestion signals, such as delay, are not considered. Typically, loss-based CCAs slow down when packet loss is detected, otherwise they keep on sending at the same rate or accelerate. Between packet loss events, purely loss-based CCAs operate blindly, e.g., the CCAs keep growing the transmission rate unconditionally. Many loss-based CCAs interpret every packet loss event as a signal for congestion despite the possibility that packet loss may occur for other reasons. This ambiguity and lack of distinction can degrade the performance of CCAs in networks where packet loss can be caused by effects other than congestion. We refer to packet loss that is not caused by congestion as random packet loss and do not further distinguish the reasons for random packet losses.

2) Time Outs: The other mechanism for loss detection is to track if a segment is acknowledged within an expected time span with a retransmission timer that is maintained on a per-connection basis. Whenever TCP sends a segment, a retransmission timer is initialized if no timer is running already. The retransmission timer records the sequence number of the oldest unacknowledged segment. The duration of the timer is based on RTT measurements. When the ACK of the recorded segment number is received, the timer is canceled, and a new timer is started if there are still unacknowledged segments in transit. On the other hand, when a timer expires and the corresponding segment has not yet been acknowledged, then an RTO is triggered. Packet losses detected via dupACKs and RTOs are generally treated differently by TCP. An RTO is a more severe and rare loss event that triggers a strong response by TCP from the perspective of a CCA. The transmission rate is typically reset to a small initial value. Hence, from the perspective of CCAs, packet losses detected via dupACKs and RTOs have to be distinguished. Once packet loss is detected, Loss Recovery (LR) algorithms are invoked. Further details about LR algorithms involving ACKs and RTOs to avoid congestion and how they are evaluated is surveyed in Section IX-B.

3) Explicit Congestion Notification: Instead of dropping packets, AQM algorithms in routers can mark packets with ECN [87] to provide in-network congestion feedback. The routers modify ECN bits in the IP and transport layer headers

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

to signal congestion with a Congestion Experienced (CE) codepoint. Note that ECN is a built-in feature supported by TCP and IP headers and requires no further modifications to the protocols. Other transport layer protocols, such as QUIC, also support ECN. However, the usage of ECN is optional and is negotiated at the beginning of each connection. For CCAs, the ECN mechanism specified in [87] is a binary signal that indicates congestion similar to a packet loss event (three dupACKs) if the CE codepoint is set, but without actually dropping packets that need to be retransmitted. AQM algorithms determine which packets are marked. Both marked and lost packets signal congestion to a CCA, which should react to both signals in an equivalent manner. ECN is by default enabled in modern Operating Systems (OSs), such as Linux.

Other variants that make different use of the ECN bits exist. For example, the ECN protocol for Low Latency, Low Loss, and Scalable Throughput (L4S) [88] repurposes the ECN bits to continuously communicate the congestion state to senders. L4S frequently marks packets and reacts differently to marked packets versus lost packets. L4S enables senders to frequently adjust their transmission rate in a fine-grained manner. The implementation of CCAs that support L4S is an ongoing IETF activity, e.g., TCP Prague [89]. Despite the potential of ECN for avoiding congestion, the adoption of ECN in the Internet has been slow for a variety of reasons, e.g., some network elements do not support ECN or incorrectly reset ECN bits [90]. In contrast, ECN has widely been adopted in the controlled environments of data centers, which we will cover in Section X-H.

4) Round-Trip Time: Many transport layer protocols let senders add a timestamp to each packet header that is reflected (mirrored back) in the ACK by the receiver. TCP calculates the RTT for acknowledged packets using Karn's algorithm [91]. Based on these measurements, TCP maintains an Exponentially Weighted Moving Average (EWMA), called smoothed RTT (sRTT), which tracks the RTT of the connection at the sender. These delay signals can be used by delay-based CCAs to adjust the transmission rate. Typically, they interpret increased delay as a congestion signal and reduce the transmission rate. As queues fill up, queueing delay is usually observable before packets are dropped or marked by a queue. Thus, delay-based CCAs can react to congestion earlier than loss-based CCAs. However, delay is a noisy signal that can fluctuate for a variety of reasons. For example, traffic bursts can lead to a temporary queueing delay which dissipates on its own after the burst ends. A CCA may react to such an increased queueing delay by prematurely forfeiting bandwidth. Therefore, many delay-based CCAs use mechanisms to disambiguate temporary and persistent queueing delays to detect so-called standing queues that are not dissolving. Moreover, the RTT measures the two-way delay instead of only the one-way delay from the sender to the receiver. When ACKs are delayed in the reverse direction, senders may incorrectly conclude that an increase of the RTT indicates congestion in the forward direction.

5) Other Congestion Signals: Besides loss and delay, senders can derive additional congestion signals from the information contained in ACKs and from the sender's socket state. Hybrid CCAs use a mixture of congestion signals. Often, hybrid CCAs model the network based on their observations and estimate parameters to adjust their transmission rate. For example, a sender can estimate the throughput of the connection by tracking the progress of acknowledged packets [92]. With that estimate, the sender can detect that the throughput stagnates despite increasing the transmission rate and conclude that it is sending too fast. However, CCAs that model the network with such estimates are at risk of creating incorrect models if they misinterpret feedback signals.

E. Window-Based and Rate-Based Control

A CCA typically limits the transmission rate (control variable in the control theory vernacular [93], [94]) so as to control the level of congestion, i.e., the queue length (process variable). A CCA can limit the transmission rate directly by adjusting the pacing rate [95]; or, indirectly by adjusting the size w of a sliding window, which limits the number of packets in flight [4]. In TCP, the congestion window `cwnd` may be adjusted by a CCA, which limits the sliding window size with $w = \min(\texttt{cwnd}, \texttt{rwnd})$. The receiver window `rwnd` is a flow control variable to impose a receiver-side limit. In typical CCA evaluations `rwnd` is always larger than `cwnd`.

Both rate control mechanisms, i.e., pacing rate and the congestion window (cwnd), can be used simultaneously; whereby, CCAs that use both, usually use only one variable as their primary control variable and the other control variable as a safety net, e.g., BBR [77] uses the pacing rate as the primary control variable, and the congestion window size is used as a backup control. A sender is cwnd-limited if the cwnd limits the transmission rate. Analogously, a sender is rate-limited if the pacing rate limits the transmission rate. The packet departure times from the sender depend on the value(s) of the control variable(s).

Receivers generate ACKs after successfully receiving segments. Since common transport layer protocols use delayed ACKs, a single ACK may cumulatively acknowledge more than one segment. A receiver generates an ACK after receiving a prescribed number of new segments that match the delayed ACK threshold or after a time limit is reached [1, Sec. 4.2]. In TCP, the delayed ACK threshold should not be larger than two. When a CCA solely uses the cwnd as the control variable, one or more segments can be sent whenever a new ACK arrives at the sender. As a consequence, the reception (arrival) times of ACKs dictate the transmission (departure) times of segments (ACK-clocked) of the sender. Delayed ACKs impair the ACK-clocking slightly. By being ACK-clocked, window-based rate control regulates itself and is highly responsive. If the network cannot deliver any packets and no ACKs are returned, then the congestion window stalls and the sender does not transmit new packets. In contrast, a sender that is limited by only a pacing rate would continue sending packets even during periods of no returned ACKs and would risk overwhelming the network.

Pacing packet departures has the advantage that it reduces the burstiness of traffic [95]. For example, suppose that an ACK arrives at a sender that cumulatively acknowledges n

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

packets. Without rate pacing, the sender would immediately advance its sliding window by n segments and transmit n packets as a burst. The burst may overwhelm a queue, cause packet loss, and a rate reduction by the sender, which may be avoided by pacing out the burst. Different implementations of packet pacing for transport layer protocols exist, whose standardization is actively worked on by the IETF [96].

F. Composition of the Standard TCP CCA

A single CCA is not one algorithm, but a composition of different algorithms that are executed in an interleaved way. The original TCP congestion control was named TCP Tahoe [4] and consists of four separate algorithms [1]: slow start, Congestion Avoidance (CA), fast retransmit, and fast recovery. TCP Tahoe was further improved by its two successors, Reno and NewReno [72], to arrive at what is today specified as the standard TCP congestion control by [1]. The Reno CCA adapts the cwnd as its control variable. In particular, the slow start and CA algorithms adapt the cwnd as long as the TCP connection is not recovering lost segments, i.e., during *normal* operation periods. In contrast, the fast retransmit and fast recovery algorithms are LR algorithms that are executed once one or more segments are lost, i.e., during LR periods.

1) Slow Start: A TCP connection typically starts with a small `cwnd` (the initial cwnd) and a large slow start threshold `ssthresh` (the initial `ssthresh`). As long as $\texttt{cwnd} < \texttt{ssthresh}$, Reno executes the slow start algorithm. Reno doubles `cwnd` within one RTT to achieve exponential growth in slow start. According to the TCP specification [1], CCAs should not grow their transmission rate faster than that.

2) Congestion Avoidance (CA): If $\texttt{cwnd} \geq \texttt{ssthresh}$, Reno uses an Additive-Increase/Multiplicative-Decrease (AIMD) scheme to update `cwnd` during CA by incrementing its value by approximately one segment each RTT and halving the *cwnd* once a packet loss event occurs. In general, the update rule of an AIMD scheme is given by

$$\texttt{cwnd}_{t+1} = \begin{cases} \texttt{cwnd}_t + \alpha \text{ if congestion is not detected} \\ \texttt{cwnd}_t \times \beta \text{ if congestion is detected} \end{cases} \tag{3}$$

with t as the index of the t-th RTT, the additive increase parameter α [segments], and multiplicative decrease parameter β. Reno uses an AIMD($\alpha = 1$, $\beta = 1/2$) scheme.

3) Loss Recovery (LR): When a segment is lost, the fast retransmit and fast recovery algorithms take over to recover the lost segments. The fast recovery algorithm adapts `cwnd` during LR. Furthermore, Reno sets the `ssthresh` value to $\max(\texttt{cwnd}/2,\ 2)$ right after entering LR. Once all segments are recovered, Reno halves `cwnd` by setting $\texttt{cwnd} = \texttt{ssthresh}$ and CA continues.

4) Composing New CCAs: The modularity of CCAs makes it possible to construct new CCAs by composition. For example, NewReno uses the CA and slow start algorithms of Reno, but defines its own fast recovery algorithm [72]. The Linux implementation of Cubic uses HyStart [97] as its slow start algorithm and the Proportional Rate Reduction (PRR) fast recovery algorithm [98]. Newer CCAs may deviate from the initial design of letting a fast recovery algorithm adapt the transmission rate during LR. Instead, these newer CCAs, such as BBR [77], use a single rate adaptation algorithm that remains active during LR. Furthermore, BBR does not use `ssthresh` to decide whether a slow start or CA algorithm is executed.

Although newer CCAs may implement completely different algorithms in comparison to Reno, the terms slow start, CA, and LR remain common terms in the CCA jargon. Rather than referring to the specific algorithms of Reno, these terms refer to phases of a CCA with different objectives.

The objective of slow start is to ramp up the transmission rate quickly to saturate the available bandwidth. Slow start is often used at the beginning of a flow, after an RTO, or after an idle period [1]. In all these cases, the transmission rate has been reset to a low value. In CA, the main objective is to prevent congestion while continuously utilizing the available bandwidth. Typically, CA conservatively (slow growth) probes for newly freed bandwidth to keep the network path saturated. LR algorithms are concerned with handling or avoiding packet loss. Apart from the fast retransmit and fast recovery algorithms of Reno, many more LR algorithms exist, as briefly reviewed in Section IX-B.

III. Evaluating Congestion Control

A. Overall Structure: Evaluation Components

CCAs can be evaluated theoretically and experimentally in a variety of ways. This section defines the structural components that constitute a CCA evaluation. We distinguish two types of CCA evaluations: theoretical evaluations and experiments. Experimental evaluations include simulations, emulations, hardware testbeds, and real-world experiments in the Internet or a data center.

In Section II-A, we introduced CC as a dynamical system with the scenario $\mathcal{S}(t)$ as system conditions. The scenario consists of the network $\mathcal{G}(t)$ and traffic $\mathcal{T}(t)$. Both the network $\mathcal{G}(t)$ and the traffic $\mathcal{T}(t)$ may have parameters to configure an evaluation. We refer to the union of the network parameters and the traffic parameters as the scenario parameters. As illustrated in Fig. 2, a CCA evaluation is related to this modeling as the evaluation has three components: network $\mathcal{G}(t)$, traffic $\mathcal{T}(t)$, and metrics $\mathcal{M}(t)$. The network $\mathcal{G}(t)$ is covered in Section IV, the traffic $\mathcal{T}(t)$ is covered in Section V, and the metrics are covered in Section VI. Conducting an evaluation can be interpreted as running the dynamical system from time $t = 0$ until it ends at $t = T$ while the metrics are monitored.

B. Theoretical Modeling

Theoretical evaluations use mathematical modeling and analysis. Theoretical evaluations are based on abstractions and often simplify real networks to make analytical models tractable. The models of the network $\mathcal{G}(t)$ and traffic $\mathcal{T}(t)$ implicitly or explicitly embody assumptions that may not immediately be apparent. However, these assumptions have to hold in order to make theoretical results applicable to real networks. Since real networks often exhibit complex dynamics that are intractable and often random, this may not be the

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

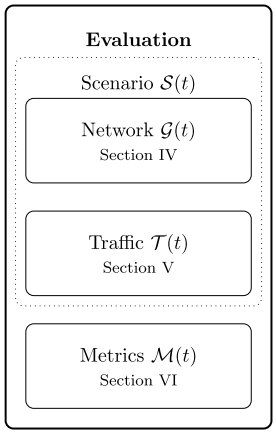


Fig. 2. Components of a CCA evaluation: The combination of a network model and a traffic model defines a CC evaluation scenario; in turn, the combination of a CC evaluation scenario and evaluation metrics defines a CC evaluation.

case. Showing that a CCA performs well theoretically can be considered as a strong argument for its deployment.

C. Experiments

Experiments that evaluate CCAs can be conducted to verify theoretical findings or for scenarios that are mathematically intractable. We differentiate between simulations, emulations, as well as testbed, Data Center (DC), and Internet experiments.

1) Experiment Steps: An experiment is first set up, then executed, and eventually monitored metrics are reported.

a) Setup step: In the setup step, all three evaluation components are configured. The network $\mathcal{G}(t)$ includes the topology and all network elements present in the experiment. For example, in a testbed experiment, all nodes (e.g., hosts and switches) are connected. The setup of the traffic $\mathcal{T}(t)$ includes configuring all applications that generate and receive traffic, e.g., setting up *iperf* [99], on senders and receivers. For the monitoring of evaluation metrics, tools are set up that can capture information at nodes and links, as further detailed in Appendix C-C.

b) Execution step: In the execution step, the actual experiment is conducted. While the experiment runs, evaluation metrics are gathered that are later used for the result reporting. Experiments can be run multiple times with different scenario parameters. Running experiments repeatedly may either be desirable to show how a parameter choice affects a CCA or to obtain statistically meaningful results.

c) Post-processing step: After an experiment ends, the evaluation metrics are processed and reported. Often, experiments are designed to evaluate how a CCA deals with prescribed network and traffic scenarios. Thus, an evaluation typically determines a set of performance metrics that are tailored toward the considered scenario.

2) Simulations: Simulation experiments typically use discrete-event simulators to evaluate CCAs, for example, ns-3 [100] or OMNET++ [101]. These simulators schedule events at simulation times that are processed sequentially. Both ns-3 and OMNET++ implement standard-compliant TCP/IP stacks with a level of detail that is comparable to implementations of OSs. Hence, they aim to minimize the abstraction level of their implementation in order to maximize the realism of the simulations. The traffic is generated by simulated applications. Network simulators typically provide ways to measure and log metrics while evaluations are running.

In contrast to emulations, which are conducted in real time, discrete-event simulators model a simulation time that represents the evaluation time t. Furthermore, the processing speed of the TCP/IP stack implementation of a simulator does not affect simulation results, i.e., a ns-3 simulation on a slow Central Processing Unit (CPU) should yield the same results as a ns-3 simulation on a fast CPU.

3) Emulations: Emulations use the TCP/IP stack implementation of OSs in combination with virtualization technologies to run experiments. Typically, Linux is used to emulate networks and evaluate CCAs as the OS implements advanced virtualization features. Network namespaces provide a way to create isolated virtual networks inside a single host [102]. Each network namespace uses its own isolated copy of a TCP/IP stack. Hence, each network namespace can be treated as an individual virtual host. Then, one or more virtual interfaces (most commonly virtual Ethernet interfaces) can be created in each network namespace. The virtual interfaces can be connected with each other to create virtual links. With *tc* and *netem* [103], [104] the links can be configured to schedule packet transmissions. By scheduling transmissions, data rate limits and artificial delays can be added to packet transmissions to emulate link properties (see Section IV). Traffic can be generated by a traffic generator program, such as *iperf* [99], by running the program in the network namespace of a virtual host. Emulations take place in real time and processing delays may alter evaluation results. With a careful emulation setup, the noise of the processing delays is negligible.

4) Testbed Experiments: Simulations and emulations have the advantage of being reproducible on commodity hardware by distributing software. In contrast, testbeds require specific hardware, e.g., hosts, routers, or switches, to be reproducible. Testbeds are controlled environments in which scenarios can be reproduced. The evaluation results depend on the hardware and software used in an experiment.

5) Data Center Experiments: DCs are large-scale networks, see Section X-H.

6) Internet Experiments: Internet experiments are harder to reproduce than other experiment types due to a lack of isolation from external influences. Experiment results are influenced by the traffic of third parties and other unobservable Internet dynamics that likely depend on the location and time. For instance, the traffic volume of access networks is usually higher in the evening [105].

IV. Background on Network Model

A. Nodes, Queues, and Links

A network $\mathcal{G}$ consists of nodes and links. Nodes are hosts, switches, or routers equipped with Network Interface Controllers (NICs) to transmit and receive communication data. Hosts are communication endpoints (senders or

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

receivers) running applications that open network sockets for communication, whereas switches and routers store-and-forward data packets originating from hosts. With a typical TCP/IP network socket, the application writes to and reads from transport layer socket buffers. The processing of packets by the TCP/IP stack adds processing delays to the data path. Incoming traffic is called ingress traffic, whereas outgoing traffic is called egress traffic.

Nodes and links have parameters that jointly constitute the network parameters. In general, we model these parameters as random processes. Treating the parameters as random processes that may change over time enables the modeling of the full gamut of network dynamics, including time-variable channels, link failures, and route changes.

Nodes use queues to store packets before the packets are fed to network interfaces and transmitted on a link l. Queues can enqueue, dequeue, drop, or mark packets. A packet that is enqueued and later dequeued experiences a queueing delay D^q [s]. Queues have a finite queue size (buffer size, capacity) B [bits] that limits the queue length (backlog size) b [bits] with $b \leq B$. If a queue is full, a newly arriving packet is dropped before enqueueing due to a queue overflow (tail drop). Queues may fill up temporarily and drain while servicing traffic bursts. A queue that is backlogged continuously (over a longer period of time) is called a standing queue. The packet scheduling algorithm that decides how packets are enqueued and dequeued is called the Queueing Discipline (qdisc) (we follow the widely used Linux terminology in this article). First In, First Out (FIFO) is the simplest qdisc that enqueues and dequeues packets one after another. A queue may be implemented in hardware, e.g., a ring buffer of a NIC, or in software, e.g., the Traffic Control (TC) system of Linux that adds software queues to network interfaces. More advanced qdiscs are covered in Section X-E.

A link l connects two network interfaces of a sender and receiver. The link is directed. A link may have a reverse link that transmits packets from the receiver to the sender. Each link has a finite capacity C_l [bits/s] that limits the transmission rate (bitrate). Furthermore, a link has a finite one-way propagation delay $\texttt{OWD}_l^{\text{prop}}$ [s] that any transmitted packet experiences. Links may induce random packet loss by corrupting or erasing packets. A packet loss model, which is a random process, decides if a packet is lost during its transmission. The loss model determines packet loss at the transport layer. Many link layer protocols may use forward error correction schemes to repair bit corruptions of frames. Such bit corruptions and repairs are concealed from the transport layer and are treated as successful transmissions by the loss model. Commonly, the packet loss model is an independent and identically distributed (i.i.d.) random variable with a packet error probability that is constant over time.

B. Paths

A path $P = (l_1, l_2, \ldots, l_H)$ of length H is an ordered sequence of links that connect a sender and receiver [106]. From the perspective of the sender, packets that are transmitted to the receiver traverse the forward path (forward direction), and packets that are returned from the receiver to the sender traverse the reverse path (reverse direction). A path is called symmetric if the properties of its reverse path are identical to the properties of its forward path; otherwise, the path is asymmetric. Packets may be lost, reordered, or duplicated on a path.

The transport layer essentially provides an end-to-end communication pipe between hosts that is agnostic of the actual network path between the hosts [4]. That is, the path (topology) between the hosts is a black box with prescribed properties. Hence, any path can be substituted with a single link that matches the path properties.

The path capacity (bottleneck rate) equals the capacity of the slowest link $C = \min_{l \in P} C_l$ of the path [107]. We refer to the slowest link as the bottleneck of a path. Since the bottleneck link has the slowest rate, the bottleneck queue is the location where congestion takes place on a path. Other queues tend to migrate to the bottleneck queue over time [77]. The bottleneck queue size and the bottleneck qdisc are the two path parameters that are particularly influential during congestion. We survey studies with small buffers in Section X-D, studies with deep buffers (bufferbloat) in Section X-C, and studies with different bottleneck qdiscs in Section X-E.

The One-Way Delay (OWD) [s] of a path is the sum of all nodal processing delays, queueing delays, transmission delays, and propagation delays of its nodes and links. For CC experiments, the nodal processing and transmission delays are often neglected, as they are typically much smaller than the queueing and propagation delays in high-speed networks. With empty queues, the OWD is approximately equal to $\texttt{OWD}^{\text{prop}}$.

The sum of the OWDs of the forward and reverse paths constitutes the RTT. The RTT is also called the Two-Way Delay (TWD), whereby the RTT is the more commonly used terminology in the CC literature. The two-way propagation delay $\texttt{RTT}^{\text{prop}}$ [s] is defined analogously.

The product of the bottleneck capacity C and the two-way propagation delay $\texttt{RTT}^{\text{prop}}$ is called the Bandwidth-Delay Product (BDP), i.e., $\texttt{BDP} = C \cdot \texttt{RTT}^{\text{prop}}$. In order to "fill the pipe," i.e., in order to avoid idle times at the sender, the sender needs to be permitted to transmit at the full (forward) bottleneck capacity C until the feedback comes back via the receiver, i.e., with a propagation delay $\texttt{RTT}^{\text{prop}}$. That is, the sender needs to transmit $C \cdot \texttt{RTT}^{\text{prop}}$ (the BDP) bits in order to continuously fully utilize the "pipe", i.e., to fully fill the forward path with packets in flight. From the perspective of the sender-based CCA, the packets corresponding to the BDP are transmitted-and-not-yet-acknowledged packets [108] when the sender fully and continuously utilizes the forward path bottleneck rate C, as further explained in Section IV-D.

As packets traverse a path they can be lost. Generally, there are two types of losses: i) random losses at links, and ii) packet drops at queues (congestion losses). The packet loss model of an entire path is a superposition of the packet loss models of all links on the path. Analogously to a link's loss model, a path's loss model is a random process that only accounts for random losses (i.e., the path loss model excludes congestion losses).

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

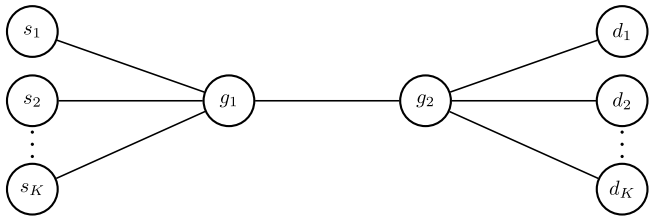


Fig. 3. Dumbbell network topology with K host pairs: Senders $s_1, s_2, \ldots, s_K$ transmit via K access links to a gateway node that forwards the K flows over one link to another gateway node, which forwards each of K flows to their corresponding receivers $d_1, d_2, \ldots, d_K$.

C. Dumbbell Topology

A common network topology to evaluate CCAs is the dumbbell topology depicted in Fig. 3. The dumbbell topology connects K host pairs through a single, shared bottleneck link. The K node pairs, which may have different propagation delays $\mathtt{RTT}^{\mathrm{prop}}$, compete for the available bottleneck capacity C.

D. Operating Point Model

Fig. 4 illustrates the different operating regions of CCAs [109] depending on the number of packets in flight Ω [segments] at the transport layer. If $\Omega < \mathtt{BDP}$, then the pipe is underutilized and the throughput is limited by the transmission rate of the sender. For $\mathtt{BDP} < \Omega < \mathtt{BDP} + B$, the pipe saturates and a proportional standing queue with backlog size $b \approx C(\mathtt{RTT} - \mathtt{RTT}^{\mathrm{prop}})$ forms (typically at the bottleneck) that inflates the RTT by adding queueing delay to the $\mathtt{RTT}^{\mathrm{prop}}$. Once $\Omega > \mathtt{BDP} + B$, then the limited buffer size B is exhausted and the bottleneck queue overflows. CCAs that continuously operate in this queue overflow region experience frequent packet drops and retransmissions due to congestion, leading to the so-called congestion collapse [110].

Ideally, CCAs should operate at Kleinrock's point of optimality $\Omega^* = \mathtt{BDP}$ [111], where the throughput is maximized while the delay and congestion losses are minimized, i.e., $\lambda^* = C$ and $\mathtt{RTT}^* = \mathtt{RTT}^{\mathrm{prop}}$. Equivalently, Ω^* is reached when the power metric [112], given as the (normalized) ratio between the throughput and RTT, is maximized [111]. Continuously operating at the optimal operating point is challenging because both C and $\mathtt{RTT}^{\mathrm{prop}}$ are unknown network path parameters for sender-side CCAs and network dynamics, e.g., time-variable channels and statistical multiplexing of competing traffic, vary the optimal number of packets in flight Ω^* over time.

Not all CCA algorithms are designed to operate at $\Omega^* = \mathtt{BDP}$. Loss-based CCAs require packet losses as their feedback signal; however, at Ω^*, no such congestion packet losses occur. On paths with a FIFO tail-drop qdisc at the bottleneck, a loss-based CCA detects congestion when the queue overflows, which occurs once $\Omega > \mathtt{BDP} + B$, but not any earlier. The loss-based CCAs blindly keep increasing their packets in flight, even if the path is saturated. This incrementally builds up a backlog and causes self-inflicted queueing delay.

In this section, the operating point is given as a throughput-RTT pair. Throughout this article, the term *operating point* broadly refers to an allocation-delay pair, whereby the exact allocation and delay metrics depend on the context. For instance, many evaluations use the OWD or queueing delay as the delay metric. Furthermore, the mean operating point is often reported, i.e., the allocation and the delay are averaged over time.

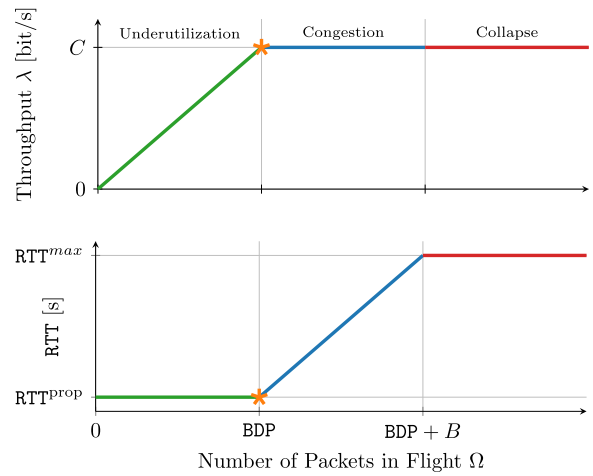


Fig. 4. Illustration of operating point regions of CCAs: Flow throughput [bit/s] and round-trip time (RTT) [s] as a function of the number of packets in flight.

V. Background on Traffic Model

The traffic model $\mathcal{T}$ consists of flows created by applications, which generate data according to a traffic generation model.

A. Flows

A flow carries packet traffic between two hosts. The packets belonging to a given flow are identified by a flow classifier based on header information (see Appendix B). There can be simultaneous parallel flows between a sender and a receiver by using different ports.

The lifetime $[t_s, t_e]$ of a flow depends on the duration of the data transmission, which begins at t_s and ends at t_e. During that lifetime, an application generates data according to a traffic model. Experiments with many flows of different lifetimes and sizes (whereby size is defined as the entire data amount carried by a flow during its lifetime) exhibit time-variable dynamics. As flows start and end, they join and leave the competition for the respective bottleneck bandwidths.

Generally, elephant and mice flows are distinguished depending on the flows' lifetimes and sizes [113]. A large-sized, long-lasting flow is commonly referred to as an elephant flow, while a small-sized, short-lived flow is commonly referred to as a mouse flow. Furthermore, elastic and inelastic traffic can be differentiated [114]. Elastic traffic stems from applications that tolerate fluctuations of the transmission quality, e.g., throughput and delay, whereas inelastic traffic has real-time constraints. Elastic traffic is more suitable for being rate-controlled by a CCA because of its less strict service requirements, while inelastic traffic often uses the User Datagram Protocol (UDP).

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

TABLE I
TRAFFIC GENERATION MODELS

Name	Description
Greedy source	Application always has data to send
CBR	Data is synchronously generated with a constant bitrate
On-off	Data is only generated during *on* periods
Poisson	Data is generated according to a Poisson point process
Bulk transfer	Constant Bit Rate (CBR) traffic to transmit files
Web	HTTP traffic that follows request-response cycles
Streaming	Real-time traffic to stream media

Many applications generate so little traffic that the CCA does not have to throttle the transmission rate. In that case, a sender is application-limited instead of network-limited. When evaluating CCAs, it is typical to configure applications to generate enough traffic for senders to be network-limited.

B. Traffic Generation Models

A flow's traffic generation model defines when the application feeds payload data at the application layer into the corresponding network socket. Table I lists the traffic generation models that are commonly used for evaluations in the CC literature. An application with a greedy source model always has data to send. The application keeps the transmission buffer of the network socket full at all times. Hence, a flow with greedy source traffic cannot be application-limited and sends as fast as the network socket allows. In contrast, a Constant Bit Rate (CBR) model generates data with a fixed rate. Data is typically fed into the network socket uniformly over time, e.g., a data block is transmitted every second. The on-off model has two periods (states). During the *on* period, the on-off traffic generation model generates traffic according to some other traffic generation model, whereas during the *off* period no traffic is generated.

Bulk transfer applications. e.g., Hypertext Transfer Protocol (HTTP) or File Transfer Protocol (FTP) downloads of large files, generate elephant flows. The bulk transfer flows are elastic and strive to minimize file transfer times. On the other hand, Web traffic follows the request-response cycles of the HTTP protocol. Web traffic flows can vary in size, but the majority of Web traffic flows are small and short-lived mice flows with the objective of finishing as quickly as possible. Media streaming applications, such as Dynamic Adaptive Streaming over HTTP (DASH), often produce long-lasting inelastic flows with real-time constraints. More details about traffic generation models are covered in Appendix B.

VI. SURVEY OF EVALUATION METRICS

This section comprehensively surveys the evaluation metrics, see Table II, that have been used in the existing literature to evaluate the performance of CCAs.

A. Taxonomy of Survey of Evaluation Metrics

Metrics can be categorized into three types: i) flow, ii) link, and iii) network metrics. Flow metrics capture the performance of an individual flow. Flow metrics are typically measured at the hosts of a network path at either the sender, the receiver, or both. Link metrics are recorded at the network links of interest. Naturally, bottleneck links at which scarce resources are shared by competing flows are of interest for CCA evaluations. The queue states of bottleneck links directly indicate the congestion level. Network metrics assess the global performance of the network as a whole: aggregated metrics, e.g., aggregated throughput, as well as fairness metrics belong to the network metric category.

Furthermore, we follow the notion of [106] to differentiate between three types of metrics: i) *sample* metrics, ii) *singleton* metrics, and iii) *statistical* metrics. A metric x that is sampled n times so as to give the vector (sequence) of metric samples $[x_1, x_2, \ldots, x_n]$ is called a sample metric. When the sampling is done within one evaluation at different measurement (sampling) times $t = 1, 2, \ldots, T$, we refer to the sample metric as a time series $x_t = [x_1, x_2, \ldots, x_T]$ for a single considered

TABLE II
SUMMARY OF SURVEY OF DEFINITIONS OF CC EVALUATION METRICS: THE METRIC TYPES ARE F) FLOW METRICS, L) LINK METRICS, AND N) NETWORK METRICS

Name	Symbol	Definition	Type
Byte or Packet Counts			
Sliding window	w	Section II-E	F
Congestion window	`cwnd`	Section II-E	F
Packets in flight	Ω	Unacked packets in flight	F
Bytes ACKed	–	Max. ACKed Seq. number	F
Queue length	b	# of packets in queue	L
Rates			
Transmission rate	r	Rate of transm. bits	F
Throughput	λ	Rate of received bits	F
Est. transmission rate	–	`cwnd`/`sRTT`	F
Est. delivery rate	–	Section VI-B2	F
Utilization (flow)	u	Norm. throughput λ/C	F
Utilization (link)	–	Norm. transm. rate r_l/C_l	L
Packet Delays and Jitter			
OWD	–	One-way delay	F
RTT	–	Round-trip time	F
Smoothed RTT	`sRTT`	RTT estimate (EWMA)	F
Jitter	–	Section VI-B4	F
Queueing delay	D^q	Sojourn time in queue	L
Packet Losses and Retransmissions			
Loss events	–	Section VI-B5	F
Packet losses	–	# of packet losses	F
Retransmissions	–	# of retransmissions	F
Timeouts (RTOs)	–	Section VI-B5	F
Queue drops	–	Dropped packets of queue	L
Utility Functions			
Prop. fairness	–	Eqn. (4)	F
Min. pot. delay fairness	–	Eqn. (5)	F
α-fairness fct.	–	Eqn. (12)	F
Network power	–	Section VI-B6	F
Flow Statistics			
Flow completion time	`FCT`	Section VI-B7	F
Responsiveness	–	Section VI-B8	F
Aggressiveness	–	Section VI-B9	F
Smoothness	–	Section VI-B10	F
Efficiency & Fairness			
Aggregated throughput	–	Section VI-D1	N
Jain's index	–	Eqn. (6)	N
β-fairness fct.	–	Eqn. (9)	N
Convergence			
Convergence speed	–	Section VI-D3	N
Stability	–	Section VI-B11	F
Usage of System Resources			
CPU usage	–	CPU usage of socket	F
Memory usage	–	Memory usage of socket	F
Power consumption	-	Power consumption of socket	F

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

flow; and as a (vector) time series $\boldsymbol{x}_t = [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_T]$ for a time series of vectors of sample metrics from K flows, e.g., for sample time $t = 1$, $\boldsymbol{x}_1 = [x_1(t = 1), x_2(t = 1), \ldots, x_K(t = 1)]^{\mathrm{T}}$. Theoretical evaluations often model dynamical systems with a continuous state function $\boldsymbol{x}(t)$ for $t \in [0, T]$ as we explained in Section II-A. Metrics are then derived from the state as continuous functions of time. A singleton metric $x \in \mathbb{R}$ is a single value that quantifies some measurable property of an evaluation. For example, the flow completion time quantifies the lifetime of a flow. A statistical metric $x \in \mathbb{R}$ condenses one or multiple sample or singleton metrics to a single real number. Elementary statistical metrics are the arithmetic mean $\overline{x}$, sample variance s_x^2, Coefficient Of Variation (CoV) $c_x = s_x/\overline{x}$, or accumulation $\sum_t^T x_t$ of a sample metric $[x_1, x_2, \ldots, x_T]$ captured in a single experiment. More complex statistical metrics can be tailored to quantify specific aspects of the CCAs. The experiment duration T influences the values of such statistical metrics [115]. More details about how to measure metrics are covered in Appendix C.

B. Flow Metrics

1) Packet or Byte Counts: The number of packets in flight Ω measures the amount (number) of segments in flight from the sender to the receiver, i.e., the number of packets in flight Ω corresponds to the so-called transmitted-not-yet-acknowledged segments [108]. Senders can estimate the number of packets in flight Ω by keeping track of unacknowledged data. Depending on the TCP implementation, the socket may count transmissions of segments or count transmitted bytes. In the case of byte counting, the number of packets in flight may be derived with a simple calculation at the transport layer: ceiling function of [number of bytes in flight divided by the Maximum Segment Size (MSS)]. Many studies and Linux use packets in flight (and not bytes in flight) and we follow the packets in flight Ω [segments] convention. The packets in flight Ω is limited by the congestion window $\texttt{cwnd}$, i.e., $\Omega \leq \texttt{cwnd}$. Instead of the number of packets in flight, $\texttt{cwnd}$ is often reported as a metric in experiments where Ω is limited by $\texttt{cwnd}$.

The number of transmitted bytes is another byte count that can be reported, either the progress over time or accumulated over the lifetime of a flow. For example, TCP tracks the sequence number of the highest unacknowledged byte of a connection at the sender.

2) Rates: In principle, data rates, which are often evaluated at the application layer, can be calculated by dividing the amount [number of bytes] of transmitted data over an interval by the duration [length in seconds] of the interval. Typically, periodically repeating (successive) measurement intervals are used for the calculation.

An important data rate is the transmission rate r of a sender. An alternative to measuring the transmission rate by counting transmitted bytes, is available for CCAs that use $\texttt{cwnd}$ as the primary control variable. With $\texttt{sRTT}$ denoting the estimated RTT at the sender (see Section VI-B3), the transmission rate can be estimated at the sender as $\hat{r} = \texttt{cwnd}/\texttt{sRTT}$. The estimate assumes that all segments of a sliding window are transmitted within one RTT.

Another commonly considered data rate is the throughput (delivery rate) λ, which can be measured at the receiver. Some studies report the goodput instead of the throughput by only considering the payload. The estimated delivery rate estimates the throughput at the sender by evaluating the progress of acknowledged sequence numbers [92], [116].

The throughput can be normalized with the path capacity to obtain the utilization $u = \lambda/C \in [0, 1]$. The utilization has the advantage that its value is independent of the path capacity C; thus, the utilization can meaningfully be compared across evaluations with different bottleneck rates C.

3) Delay: Delays can be measured either on a per-packet basis or as averages over sample intervals. For delay measurements over sample intervals, typically either the mean or a high percentile (tail latency), e.g., the 95th-percentile, is reported.

The OWD measures the end-to-end delay of a path, see Section IV-B. To measure the OWD, packets needs to contain sequence numbers and timestamps. Furthermore, the endpoints need to maintain time synchronization.

In contrast to the unidirectional OWD, the RTT is the sum of the OWDs from the forward and reverse paths, i.e., the two-way (roundtrip) delay. The per-packet RTT can be calculated based on acknowledgments with timestamps that are returned by the receiver to the sender. Alternatively, the per-packet RTT samples can be averaged over a short time scale. TCP uses an EWMA to estimate the RTT at the sender [117], that is also called the smoothed RTT and denoted by $\texttt{sRTT}$.

Both the OWD and RTT are end-to-end delays. The propagation delay and queueing delay are components of the end-to-end delays, as explained in Section IV-B. The two-way propagation delay $\texttt{RTT}^{\text{prop}}$ is either known or can be estimated. Vegas [118] introduced the idea to estimate $\texttt{RTT}^{\text{prop}}$ (also called the base RTT) by tracking the minimum of the per-packet RTT samples of a connection. Other CCAs, such as BBR [77], use the minimum RTT of samples within a short-term sliding window to account for $\texttt{RTT}^{\text{prop}}$ changes.

The queueing delay D^q can be estimated for each flow, for example with $\widehat{D^q} = \texttt{sRTT} - \texttt{RTT}^{\text{prop}}$. The estimation assumes that the difference between the end-to-end delay and the propagation delay originates from the queueing delay. If $\texttt{RTT}^{\text{prop}}$ is unknown, it may be replaced by an estimate. Another approach is to measure the queueing delay of the bottleneck queue as a link metric (see Section VI-C). Since queues may be shared, the traffic generated by other flows may alter the queueing delay for a considered flow. Hence, distinguishing self-inflicted queueing delay from shared queueing delay is more in line with the nature of flow metrics. In [14], latency-avoidance is introduced as a concept to measure how well a CCA avoids self-inflicted queueing delay.

4) Jitter: As RFC 5166 [10] suggests, congestion control should minimize oscillations of delay and throughput. This section only covers the metrics related to the delay oscillations; the throughput oscillations are covered in Section VI-B11. Oscillations of delay components, e.g., queueing delay, result in *jitter* of the RTT. Multiple approaches have been defined to measure these delay variations, namely evaluating: (i) the standard deviation of a set of measurements [119], (ii) the inter-packet delay variation (IPDV) [120], [121], which is the delay difference between consecutive packets, and (iii) the absolute IPDV [122], [123], or (iv) the smoothed interarrival *jitter* of [124]. However, jitter is rarely evaluated

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

in existing studies; the few studies that consider jitter usually examine the standard deviation of the RTT or queueing delay.

5) Losses, Retransmissions, and Timeouts: CCAs aim to avoid congestion losses and retransmissions. Different metrics exist to characterize loss events, mostly measured in bytes or packet units.

On a per-packet basis, packet loss is a binary signal, which equals zero if a packet is successfully received within a time threshold, and equals one otherwise [125]. A variation of this definition is used in [126], which treats a burst of losses within one RTT as a single loss event.

Lost packets can also be captured as sample metrics. For that, the number of losses with a sample interval is counted. Similarly, the number of retransmissions and RTOs can be counted. Packet losses or retransmissions may further be distinguished depending on the socket state, e.g., packet losses during slow start, CA, or LR. Spurious packet loss and RTOs, which are explained in Section IX-B, may be counted separately as well. Likewise, the number of bytes sent or the throughput during LR may be measured. Many loss metrics can be aggregated over the entire lifetime of a flow to yield statistical metrics, e.g., the total number of retransmissions by a flow. These numbers can be divided by the number of sent packets to obtain ratios, e.g., the packet loss ratio. The time spent in LR may be measured.

The number of loss events (LR is entered) differs from the number of lost packets, because packets may be lost while TCP is already in LR. Typically, the response of a flow to a single loss event and to multiple loss events within one RTT is the same. Therefore, [127], [128] define a *loss event fraction* metric that treats bursty losses within one RTT as a single loss event. Loss patterns are further discussed in [129]. The *loss period* metric captures the frequency and burstiness of loss events, while the *loss distance* metric measures the spacing between loss events.

In [14], a metric called *robustness to non-congestion loss* is introduced that measures whether a CCA continuously maintains an allocation that is larger than a threshold despite experiencing packet losses. Furthermore, a metric called *loss-avoidance* is introduced that measures the packet loss ratio that a CCA generates at steady-state (which ideally equals zero).

6) Utility: The utility of a single flow can be measured with a utility function. For the evaluation of a utility function, the resource allocation x_k is typically the congestion window `cwnd`, the transmission rate *r*, or the throughput λ; also, composed metrics, such as the network power [111] (throughput over delay), are feasible. Two common utility functions in the CC literature are proportional fairness given by

$$U(x_k; w_k) = w_k \log(x_k) \tag{4}$$

and minimum potential delay fairness [130] given by

$$U(x_k; w_k) = -\frac{w_k}{x_k} \tag{5}$$

with a weight w_k. Both utility functions can be expressed as special cases of an α-fairness utility function which we cover in Appendix C-E.

7) Flow Completion Time: The flow completion time `FCT` is the time duration between a connection establishment and termination. Approximately, the flow completion time is the duration between the time t_s when the first packet is transmitted and the time t_e when the last packet is received with $\texttt{FCT} \approx t_e - t_s$. Dukkipati and McKeown [131] argue that from an application perspective, the flow completion time is the most suitable metric to measure the performance of a CCA, because minimizing the flow completion time corresponds to maximizing throughput while avoiding queueing delay.

A metric that is similar to the flow completion time is the response time of Web servers. In a scenario with an application that generates Web traffic by exchanging HTTP request and response messages, the duration between the transmission of a request message and the reception of the response message at the sender measures the response time of the Web server. With HTTP/1.0, a new connection (flow) is created for each exchange of a request and response message (non-persistent connections). In that case, the response time equals the flow completion time. With HTTP/1.1, a single transport layer flow can be used for multiple request and response messages to avoid the overhead of connection establishment (persistent connections) [108]. With persistent connections, the flow completion times and response times of Web servers measure different durations. Both the flow completion time and the response times of Web servers are typically measured at the application layer.

8) Responsiveness: CCAs should be responsive to congestion by shrinking their transmission rate in a timely manner. That ability is measured by the responsiveness metric for which different definitions exist. In [132], [133], [134], responsiveness measures how fast [commonly in units of RTTs] a single flow can halve its transmission rate on a congested path that generates at least one packet loss per RTT. The definition is closely related to Reno, which is an AIMD scheme of Eqn. (3) with multiplicative decrease parameter $\beta = 1/2$. Reno halves its cwnd immediately after a packet loss is detected and therefore has a responsiveness of one RTT. However, this definition of responsiveness is tailored towards Reno and therefore outdated to some degree, since today Reno is not the predominant CCA anymore, see Section II. Today, other CCAs reduce cwnd (or the pacing rate) less strongly. For example, Cubic uses a multiplicative decrease factor of $\beta = 0.7$. By the definition of this responsiveness metric, these CCAs would not be responsive to congestion, which is an erroneous conclusion. Furthermore, note that this responsiveness metric only evaluates how fast a CCA reacts to a packet loss as a congestion signal. The responsiveness towards increased queueing delay is not examined.

Two alternative responsiveness definitions for evaluations with multiple flows are proposed in [135], [136]. In these definitions, the responsiveness equals the convergence speed of the throughput, which we cover in Section VI-D3.

9) Aggressiveness: Aggressiveness measures the ability of a CCA to acquire unclaimed spare bandwidth. In [132], [134], aggressiveness is defined as the maximum increase of the transmission rate [typically in units of packets per RTT] within one RTT. The aggressiveness of a CCA may depend on its

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

phase—CCAs are typically more aggressive during slow start compared with congestion avoidance. Instead of measuring aggressiveness on a time scale of one RTT, Zarchy et al. [14] propose to consider longer time scales to measure the growth of the transmission rate.

10) Smoothness: Smoothness evaluates how rapidly a CCA can change its transmission rate. In [132], [133], smoothness is defined as the maximum reduction of the transmission rate within one RTT. As an example, the theoretical smoothness of Reno equals the AIMD parameter $\beta = 1/2$ of Eqn. (3) by definition. Note that transmission rate reductions due to RTOs should not be considered for the smoothness metric.

11) Stability: The stability metric (also referred to as oscillation or smoothness metric) measures the variations of a flow's allocation over time. In [127], [134], [135], [137], [138], the CoV (or standard deviation) of the throughput is considered. Other stability metrics include the throughput ratio λ_t/λ_{t-1} of two consecutive time steps [132] and the average absolute deviation of the throughput [139]. We also note that [136] considers the CoV of the queueing delay as a stability metric. The stability metric is related to the stability theory in the context of dynamical systems, which is further explained in Section IX-C.

12) Usage of System Resources: To measure the computational footprint of a CCA, the CPU usage, memory usage, and power consumption can be tracked. A small footprint is especially of interest for devices with little resources, battery-driven devices, and for servers that maintain thousands of parallel connections. A general-purpose CCA has to scale up to high multiplexing levels, which can be a problem for complex algorithms, e.g., CCAs that use machine learning. Furthermore, high throughput requires fast packet processing, otherwise the processing speed of the CCA at the sender can become the bottleneck (CPU-limited). Hence, CCAs should require little system resources, otherwise their usefulness in production systems is limited. The power consumption of CCAs can also be examined [140].

C. Link Metrics

Link metrics can be captured for each link in a network. For the evaluation of CCAs, bottleneck links are primarily of interest. The link utilization is given by the normalized transmission rate $u_l = r_l/C_l$, where r_l is the transmission rate of the link and C_l is the capacity of the link. The queue length (backlog size) b_l counts the number of queued packets. Queued packets that are serviced before a given waiting packet cause a queueing delay D_l^q for the considered waiting packets in the queue. Each packet experiences an individual queueing delay, which is measured by the sojourn time (waiting time) of a packet passing through a queue.

Queues can drop or mark packets and thereby generate loss events, see Section VI-B5. The number of queue drops quantifies the congestion level of a queue.

D. Network Metrics

1) Aggregated Flow Metrics: Flow and link metrics are captured for individual flows or links, respectively. The metrics can be aggregated (sum over flows or links) to measure network-wide performance [10]. For instance, the transmission rates $[r_1, r_2, \ldots, r_K]$ of K, $K \geq 1$, senders in a network can be aggregated to the aggregated transmission rate $\sum_{k=1}^{K} r_k$. Other aggregated flow and link metrics are defined analogously. The aggregated allocation (in most cases, the transmission rate or throughput) has also been referred to as efficiency [14], [141].

2) Fairness: As the network resources are shared among competing flows, it is important to evaluate how evenly the resources are divided among all K, $K > 1$, flows. Fairness metrics evaluate the "evenhandedness" of this resource division by assessing the resource allocation vector vector $\boldsymbol{x} = [x_1, x_2, \ldots, x_K]^{\mathrm{T}}$. For this purpose, a fairness function f maps the allocation $\boldsymbol{x}$ to a scalar value for which higher values indicate a fairer allocation, i.e., the fairness function f grows towards 1 (with appropriate normalization, as needed for some metrics) as the K elements of the vector approach each other. A common fairness function in the CC literature is Jain's index [142]:

$$f_J(x_1, x_2, \ldots, x_K) = \frac{\left(\sum_{k=1}^{K} x_k\right)^2}{K \sum_{k=1}^{K} x_k^2}. \tag{6}$$

Fairness can be evaluated over different time scales. Commonly, the fairness of a mean allocation $f(\frac{1}{T}\int_{t=0}^{T} \boldsymbol{x}(t)\, dt)$ over the duration of an experiment is used. However, it is also possible to measure fairness over time by evaluating $f(\boldsymbol{x}(t))$ with $\boldsymbol{x}(t) = [x_1(t), x_2(t), \ldots, x_K(t)]^{\mathrm{T}}$. Additional fairness metrics that could be used in place of Jain's index are detailed in Appendix C-D. The viability of these throughput-centric fairness metrics has been debated, see Appendix C-G. Alternative notions of fairness may also consider delay and other metrics [143]. For example, in [144], a slowdown metric is introduced to measure fairness. The slowdown metric measures how much a CCA of a flow prolongs the flow completion times of competing flows.

3) Convergence: The convergence speed (also convergence time or rate) metric measures how fast an allocation converges to its equilibrium. Analytically, the convergence speed is measured by an order of convergence of the time series $\boldsymbol{x}_t$, e.g., $\boldsymbol{x}_t$ may converge linearly, quadratically, or exponentially to its equilibrium [80], [145], [146], [147]. Hence, such rate-of-convergence evaluations examine the asymptotic behavior of $\boldsymbol{x}_t$. In experiments the convergence speed t_c is measured in seconds or units of RTTs. The convergence speed requires a convergence criterion to decide when $\boldsymbol{x}(t)$ converged. The ϵ-convergence criterion is used in [135], [136], [139], [148], whereas the δ-fairness criterion is used in [97], [133], [138], [149]. Both convergence criteria are detailed further in Appendix C-F. Another convergence criterion is used in [133], which monitors the frequency of congestion losses. The criterion assumes that the duration between congestion losses (queue drops) converges at the same time as $\boldsymbol{x}_t$ converges. However, since some CCAs may not induce queue drops and the frequency of queue drops depends on many factors, e.g., the bottleneck qdisc, the criterion may not detect the convergence of $\boldsymbol{x}_t$ as intended.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

E. Taxonomy of Competition and Fairness

1) Intra- and Inter-Protocol Fairness: There are two types of fairness notions depending on the choice of CCAs at the senders. Intra-protocol fairness requires that all senders in an experiment use the same CCA. Hence, intra-protocol fairness assesses if a CCA is fair to itself. On the other hand, inter-protocol fairness is evaluated in experiments that feature at least two different CCAs. To make an (iterative) deployment of a new CCA feasible, the fairness to established CCAs is an important criterion.

2) Fairness Notions Depending on System Conditions: We differentiate three fairness notions depending on the evaluation scenario: i) synchronous fairness, ii) start-time fairness, and iii) RTT fairness.

a) Synchronous fairness: In a synchronous fairness scenario the path parameters and start times of all flows are identical. Even though such system conditions rarely exist in real networks, synchronous fairness can be understood as a baseline evaluation to examine whether a CCA is at least fair in an idealized competition scenario. Many studies evaluate the steady-state intra-protocol synchronous fairness in long-running experiments in which the competing flows converge. However, studies should not solely evaluate synchronous fairness for two main reasons. First, since all flows start simultaneously in synchronous fairness evaluations, the startup phases of flows are largely synchronized and the rates grow evenly. Hence, synchronous fairness may not properly test whether a CCA can yield bandwidth to or acquire bandwidth from competing flows. Second, the effect of different path parameters among flows cannot be studied.

b) Start-time fairness: Start-time fairness relaxes the condition that all flows start simultaneously. Instead, flows may start or end during an experiment at different times, but the path parameters remain identical.

c) RTT fairness: RTT fairness assesses if a CCA is fair independent of the two-way propagation delays of competing flows. Whether or not RTT fairness is a desirable property has been questioned in [150].

F. Most Commonly Used Metrics

Although there are numerous performance metrics in the CC literature, Table III shows the most commonly used metric types. Studies may differ with respect to the specific metrics they use; e.g., one study may measure the cwnd whereas another study may measure the throughput, however, both metrics quantify the allocation of flows.

TABLE III
MOST COMMONLY USED METRIC TYPES IN CCA EVALUATIONS

Name	Section	Comments
Allocation	VI-B1/VI-B2	$\boldsymbol{x}(t)$ of Section II-A
Delay	VI-B3	Packet delays
Loss	VI-B5	Packet loss
FCT	VI-B7	Flow completion time
Fairness	VI-D2	Fairness of mean allocation

VII. CCPERF: EVALUATION FRAMEWORK

We have developed *ccperf* as a modular framework to evaluate and benchmark CCAs [15], [151]. The framework defines a set of experiments that cover a wide range of scenarios. The set of experiments test CCAs thoroughly in order to obtain a holistic view of the performance of the CCAs. To quantify the performance, ccperf specifies a set of metrics.

A. Architecture

The ccperf framework consists of a frontend and a backend. The ccperf frontend defines, pre-processes, launches, and post-processes experiments. Furthermore, the frontend specifies a ccperf benchmark, which is a set of experiments that are launched to comprehensively evaluate a CCA under test in a single benchmark run. For the user, the frontend provides a Command-Line Interface (CLI) to run individual experiments as well as benchmarks with arguments. Experiments are determined by three components: network, traffic, and metrics, as illustrated in Fig. 2. An experiment can have parameters that can be varied, e.g., the *Loss Resilience Experiment* uses the packet error probability as a changeable parameter. The experiment parameters can be specified as arguments over the CLI. In a benchmark run, some experiments are repeated with different parameters to investigate how parameters affect the performance of CCAs. The execution of an experiment is done by the backend, which is launched by the frontend as a child process. After the backend finishes the execution, the resulting metrics can be post-processed.

The ccperf backend executes experiments in an evaluation platform. Presently, the backend is implemented by a ccperf module for ns-3. In principle, other suitable simulators, emulators, or testbeds can be used as evaluation platforms as well. In the ns-3 backend, the experiments are set up and simulated. The ns-3 backend also has a CLI to run experiments with arguments. Metrics are saved to an SQLite database at runtime. The SQLite database contains all simulation results, which are later used by the frontend for post-processing.

B. Topologies

All ccperf experiments use the dumbbell topology with a single bottleneck shown in Fig. 3. The network parameters of the dumbbell topology can be configured with the path parameters of Table IV and the number K of paths. The two-way propagation delay of each path in the dumbbell topology can be set individually. The packet loss model is given by an i.i.d. packet error probability.

Table IV shows the paths of ccperf. Many of the ccperf experiments in Table V use the default path parameters with some changes to configure the network, e.g., the loss resilience experiment may change the packet error probability. The capacity and queue size of the default path are proportional to the number of competing flows K to grant a proportional amount of resources that flows can compete for in experiments. The queue size is purposefully set to $B = K \times \texttt{BDP} = C \cdot \texttt{RTT}^{\text{prop}} = K \times 20$ packets, so that sufficient buffering is

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
PARAMETER SETTINGS OF PATHS OF CCPERF; MX = MEXICO, P(·) = POISSON WITH MEAN (DATA) RATE (OF LINK)

Name	Capacity [Mbits/s]	OWD [ms]	Loss [%]	Qu. Size [packets]	qdisc
default	$16K$	7.5	0	$20K$	FIFO
India to AWS India	100.42	27	0	173	FIFO
AWS Korea to China	77.72	51	0.06	94	FIFO
AWS CA to MX	114.68	45	0	450	FIFO
WINDS Satellite	42.00	400	0.74	2800	FIFO
Nepal to AWS India	P(0.57)	28	4.77	14	FIFO
MX cel. to AWS CA	P(2.64)	88	0	130	FIFO
AWS Brazil to Colombia. cellular.	P(3.04)	130	0	426	FIFO

provided for each flow [152]. The parameters of the paths named India to AWS India, AWS Korea to China, AWS California to Mexico, Nepal to AWS India, Mexico cellular to AWS California, and AWS Brazil to Colombia cellular are the calibrated path parameters of the Pantheon project [11]. The WINDS satellite path parameters are from [153].

Dynamic paths have network parameters that change over time. In ccperf, dynamic paths are implemented by replaying packet departure trace files as further explained in Section XI-B. The trace files replay packet departure timings that resemble Poisson traffic with different mean rates. For that, the ccperf ns-3 module implements a qdisc that schedules packet departures at the bottleneck link. At each scheduled time slot, the qdisc may send a quota of packets. For the dynamic paths, the quota is set to one and the departure times are read from the trace files such that the link generates the respective Poisson traffic.

C. Traffic

The traffic that is generated and consumed by applications can be specified on a per-flow basis. The parameters used for the configuration of flows are the start and stop times, transport layer protocols, CCAs, and traffic models. In Section V-B, different traffic models that are commonly used in CC evaluations are outlined. In ccperf, most experiments set up flows that use the greedy source traffic model to ensure network-limited evaluation scenarios. Other traffic models are implemented with on-off applications, e.g., Poisson traffic uses an exponentially distributed off-time between subsequent packets. The traffic model can be set for each flow individually by providing a signature, e.g., `Greedy(bytes=0)` lets a flow use a greedy source traffic model with an infinite number of packets to send.

D. Metrics

A subset of the metrics outlined in Section VI are used in ccperf. As explained in Section VI, we differentiate flow, link, and network metric types. Furthermore, depending on the sampling technique each metric is either a sample, singleton, or statistical metric. Each metric has its unique identifier in ccperf, which follows a naming scheme that includes the metric type, measurement protocol layer, and name of the metric. For example, the `flow_throughput_l4` metric measures the throughput of a flow at the transport layer. The sample type can be deduced from the definition of the metric. Statistical metrics aggregate other metrics and their identifiers start with the aggregation method, e.g., `mean_flow_x_l4` calculates the mean of a flow metric x at the transport layer. The mean, coefficient of variation, and total (sum of all elements) are the aggregation methods used in ccperf.

In the ccperf ns-3 backend, a tracing mechanism is used to calculate and save metrics while the simulation is running. The tracing mechanism allows the registration of callbacks for certain variables. Whenever a traceable variable changes, all registered callbacks are run by ns-3, which allows ccperf to extract state information required to calculate a metric. Based on this state information, metrics are calculated and saved to the database. Depending on the metric, the sampling is either periodic or event-based.

E. Experiments

Experiments in ccperf evaluate tailor-made scenarios with certain network conditions, traffic conditions, and evaluation metrics as depicted in Fig. 2. An experiment has a unique name and has parameters that change the specifics of the scenario that is created. One parameter that all experiments have in common is the CCA under test that is evaluated by the experiment. In ccperf, related experiments are bundled together in experiment groups, which correspond to the main evaluation categories in our taxonomy in Fig. 1. Table V shows the experiment groups, experiment names, experiment parameters, and a reference to the respective section of this article. The parameter that sets the CCA under test is omitted from the table.

F. Benchmarks

Benchmarks are a set of experiments that are run for a CCA under test. Each experiment type of Table V is evaluated multiple times with different experiment parameters in a single benchmark run. In total, more than 100 different experiments are run by ccperf to thoroughly test a CCA. The results of each benchmark are saved to separate database file. When benchmarks are repeated for different CCAs under test, the same random seeds may be used to ensure that the CCAs encounter the same network and traffic conditions in each experiment run.

G. Reporting of Results

The ccperf framework shows evaluation results in accordance with the CC literature. After a benchmark is run for a CCA under test, the stored metrics can be post-processed to create various figures and tables. The results are created for each experiment type separately and we distinguish experiment results, benchmark results, and comparative results.

1) Experiment Results: The experiment results show the metrics that were measured in a single experiment. Statistical metrics are shown in tables for flow, link, and network metrics in accordance with Section VI. The flow metrics table shows metrics for each flow separately, whereas the link metrics table shows metrics for the bottleneck link. The network metrics

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

TABLE V
EXPERIMENTS OF CCPERF, WHEREBY K DENOTES THE NUMBER OR FLOWS AND THE OTHER PARAMETERS ARE DEFINED IN THE LISTED SECTIONS. THE CCPERF EXPERIMENT GROUPS CORRESPOND TO THE EVALUATION CATEGORIES IN OUR TAXONOMY IN FIG. 1

Group	Name	Parameters	Section
Steady-state	Single Flow	path	VIII-A
	Synchronous Fairness	K, path	VIII-B
	Start-time Fairness	K, path, start times	VIII-B
	RTT Fairness	K, propagation delays	VIII-B
Transient-state	Step Function Response	target, period, direction, lower, upper	IX-D
	Pulse Wave Response	target, period, lower, upper	IX-D
Network Variation	Scalability	capacity, two-way propagation delay	X-A
	Loss Resilience	packet error probability	X-B
	Deep Buffers	K, queue size	X-C
	Shallow Buffers	K, queue size	X-D
	AQM	K, qdisc	X-E
	ECN	K, CE threshold	X-H
	ACK Compression	interval, quota	X-F
	Traffic Policing	policing rate, peak rate	X-G
Dynamic Networks	Replay Rate Trace	trace file	XI-B
	Random Walk	random walk parameters	XI
Competition	Reno Fairness	K	XII-B
	Cubic Fairness	K	XII-B
	Vegas Fairness	K	XII-B
	TCP On-Off Traffic	K, start and stop times	XII-A
	UDP Background Traffic	K_{TCP}, K_{UDP}, start and stop times, traffic gen. models	XII-C

table is only shown if more than two flows were present in an experiment because all network metrics of ccperf are aggregated flow metrics. Moreover, the operating point is depicted by a set of figures in different ways. The figures include a time series plot as shown in Fig. 6, a throughput-delay plane plot as shown in Fig. 10, an empirical Cumulative Distribution Function (eCDF) plot of the throughput and sRTT, and a plot that marks the mean operating point in the operating regions illustrated by Fig. 4.

2) Benchmark Results: The benchmark results summarize the metrics of repeated experiments of the same type with different experiment parameters. The results aim to evaluate how experiment parameters affect the performance of the CCA under test. Therefore, the figures are specifically tailored to the experiment type. For example, the loss resilience experiment is repeated for different packet error probabilities to measure the mean bandwidth utilization as a function of the packet error probability, as depicted in Fig. 8.

3) Comparative Results: Results of different benchmarks can be processed to compare different CCAs against each other. A common figure is to show the mean throughput and mean delay of CCAs comparatively in a throughput-delay plane as a scatter plot.

H. Use of ccperf in This Article

We have used our ccperf framework (with ns-3 as the backend) to generate all result figures in this article. We have used ns-3 as-is, i.e., we have not changed the ns-3 code. The ns-3 CCA implementations have been validated against the corresponding Linux implementations in [154]. Our evaluations confirmed that there are some discrepancies between the ns-3 and Linux implementations (as acknowledged in [154]); furthermore, we noticed some discrepancies to the CCA designs described in the respective original studies. Further investigating and resolving these ns-3 CCA implementation inconsistencies is an important direction for future research and development. Thereby, our ccperf framework can be used to validate CCA implementations and to uncover differences.

We show results for Reno, Cubic, Vegas, and BBR; we selected these CCA algorithms for the following reasons:

- Reno [4]: Standard TCP CCA (original, legacy); formerly most popular CCA; especially older studies evaluated Reno extensively.
- Cubic [76]: Widely-used CCA today (default in many OSs, e.g., in Linux, Windows, MacOS, Android). Whereby, we disable the TCP friendliness operating mode of Cubic so that the waveform is a cubic function.
- Vegas [118]: Widely-known representative of delay-based CCAs (first CCA to popularize delay as a congestion signal). Whereby, we do not use delayed ACKs to obtain results that are close to the original Vegas design.
- BBRv1 [77]: Has become popular lately; developed actively by Google (BBRv3 is in development [155], including its standardization at the IETF); uses different congestion signals (not only loss or delay).

Throughout, we present only key illustrative evaluation results that support the respective surveyed evaluation category. The comprehensive set of evaluation results is accessible at the affiliated website (*https://ccperf.net*) [151].

VIII. STEADY-STATE INTRA-PROTOCOL EVALUATION

The goal of steady-state intra-protocol evaluations is to examine the equilibrium $\boldsymbol{x}^e(t)$ that the allocation $\boldsymbol{x}(t)$ may converge to. For that, a steady-state evaluation sets up long-running flows that compete in a static scenario. A steady-state evaluation is designed to spend most of its time in the equilibrium state. Hence, the influence of the initial transient dynamics on metrics averaged over the duration of an experiment becomes negligible. In this section, only intra-protocol competition experiments are covered, in which all

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

flows use the same CCA. Inter-protocol competition is covered in Section XII.

Let $\boldsymbol{x}(t)$ be the allocation of a dynamical system of Eqn. (1) for $0 \leq t \leq T$. Steady-state experiments have a duration T that is much longer than the time when $\boldsymbol{x}_t$ converges. Furthermore, the system conditions determined by the scenario $\mathcal{S}(t) = (\mathcal{G}(t), \mathcal{T}(t))$ are chosen such that $\boldsymbol{x}(t)$ may converge to an equilibrium without disturbances. After some point in time, the network is static with $\mathcal{G}(t) = \mathcal{G}(t_1)$ for $t \geq t_1$, i.e., network parameters, such as link capacities or delays, are constant over time. Likewise, the traffic $\mathcal{T}(t) = \mathcal{T}(t_1)$ is constant for $t \geq t_1$, i.e., all applications have started and generate traffic with a constant data rate. Such experiments are also referred to as *traditional* in [156].

The equilibrium of a CCA depends on the conditions that are set by the scenario. In particular, the network topology and all path parameters influence the equilibrium. For instance, a larger bottleneck capacity provides more resources to allocate. Therefore, the results obtained for some choice of network parameters may not generalize to other parameters. Experiments normally use some choice of network parameters that are deemed suitable for studying the steady-state behavior of CCAs. However, another choice of network parameters may lead to significantly different results [150], [157]. We cover the influence of network parameters on the equilibria of CCAs in Section X.

A. Alone on a Path

1) Scenario: A static scenario is typically used to evaluate the steady-state behavior of CCAs. In the *"Alone on a Path"* evaluation, a single flow (i.e., the number K of flows is $K = 1$) operates in the static network. Only a single network path has to be set up for the experiment, e.g., a dumbbell network of Fig. 3 with $K = 1$. Since neither the network nor the traffic conditions change over time, the allocation $x(t)$ of the flow is expected to converge to its steady-state equilibrium $x^e(t)$. The equilibrium depends on the network and traffic parameters, e.g., a larger path capacity may lead to larger allocations. The network may use a nonzero i.i.d. packet error probability as a path loss model to induce random losses.

2) Metrics: The static scenario creates the ideal system conditions to evaluate the steady-state operating point of a CCA. All metrics of Table III are applicable, except for fairness as there is only a single flow. For the operating point, an allocation variable and a delay variable should be evaluated as shown in Fig. 4. The $\texttt{cwnd}$, packets in flight, the transmission rate, or the throughput can be used as the allocation variable. The sRTT, OWD, or queueing delay can be used as the delay variable. The waveform of the operating point may be evaluated, e.g., $\texttt{cwnd}(t)$ or $\texttt{sRTT}(t)$ may be evaluated as a function of time. Furthermore, statistical metrics may be used to quantify the average performance of a CCA. Besides the averages of the allocation and delay variables, the number of retransmissions and flow completion times are commonly reported.

3) Theoretical Modeling:

a) Waveform: For many CCAs, the equilibrium allocation $x^e(t)$ oscillates periodically. For some CCAs, the waveform can be modeled analytically; examples are the modeling of the sending window $w(t)$ for Reno [158], [159], Cubic [76], and Vegas [160]. Suppose Reno experiences periodic loss events with a loss event rate of p. Then, the window $w(t)$ of Reno approximately follows a periodic (cyclic) sawtooth function [158] as it implements an AIMD($\alpha = 1, \beta = 1/2$) scheme of Eqn. (3). The window $w(t)$ is increased by approximately one segment each RTT starting from $w_{\max}/2$ until $w_{\max}$ within a loss cycle of duration $T_c = 1/p$. The window is given by $w(t') = (w_{\max}/2)(1 + t'/T_c)$ as a function of the elapsed time t' since the last loss event. When a loss event occurs after the window reaches $w_{\max}$, Reno halves its window and a new cycle starts. The modeling assumes that the packet loss can immediately be repaired; the LR phase is neglected. The duration T_c of the loss cycle is proportional to the RTT; thus, the window growth of Reno is slower for larger RTTs.

In contrast, the window of loss-based Cubic follows a cubic function $w(t') = a(t' - b)^3 + w_{\max}$, whereby the parameters a and b are independent of the RTT [76]. Both Reno and Cubic blindly increase their number of packets in flight in CA until a new loss event occurs. Such CCAs are called buffer fillers. With FIFO queues, buffer fillers produce standing queues that eventually overflow. However, as explained in Fig. 4, the ideal operating point of a CCA avoids filling up buffers.

The sending window of Vegas is designed to converge to a constant size slightly above the BDP in CA. Vegas estimates the two-way propagation delay with $\texttt{RTT}^{\hat{\text{prop}}}$ and tracks the minimum RTT sample in a sending window with $\texttt{mRTT}$ to draw conclusions about how many packets are queued in the network. Furthermore, Vegas calculates the difference of the actual rate $r_a = \texttt{cwnd}/\texttt{mRTT}$ and expected rate $r_e = \texttt{cwnd}/\texttt{RTT}^{\hat{\text{prop}}}$ with $\delta = r_e - r_a$. Then, the cwnd is adjusted with

$$\texttt{cwnd}_{t+1} = \begin{cases} \texttt{cwnd}_t + 1 & \text{if } \delta < \alpha \\ \texttt{cwnd}_t - 1 & \text{if } \delta > \beta \\ \texttt{cwnd}_t & \text{else,} \end{cases} \tag{7}$$

where α and β are two parameters. The default values of α and β are chosen such that Vegas aims to keep two to four packets in the network. At steady-state, Vegas is designed to operate in the region $\alpha < \delta < \beta$, where $\texttt{cwnd}$ remains constant over time [160].

BBRv1 [77] is a hybrid CCA that uses the pacing rate r_p as its primary control variable. BBR cycles through an internal state machine to periodically probe and calculate the path capacity estimate $\hat{C}$ and two-way propagation delay estimate $\texttt{RTT}^{\hat{\text{prop}}}$. With these estimates, BBRv1 sets $r_p = \hat{C}$ and $\texttt{cwnd} = g \cdot \hat{\texttt{BDP}} = g \cdot \hat{C} \cdot \texttt{RTT}^{\hat{\text{prop}}}$, where g is some gain factor. Hence, BBRv1 relies on accurate estimates of the path capacity and two-way propagation delay. To probe for the path capacity, BBRv1 periodically increases the pacing rate slightly in the *ProbeBW* state. Each *ProbeBW* state is followed by a *Drain* state that reduces the pacing rate slightly so that queues can drain. To probe for $\texttt{RTT}^{\text{prop}}$, BBRv1 periodically reduces its number of packets in flight to a small value in order to drain standing queues and avoid inflated RTT measurements due to queueing delays in the *ProbeRTT* state.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

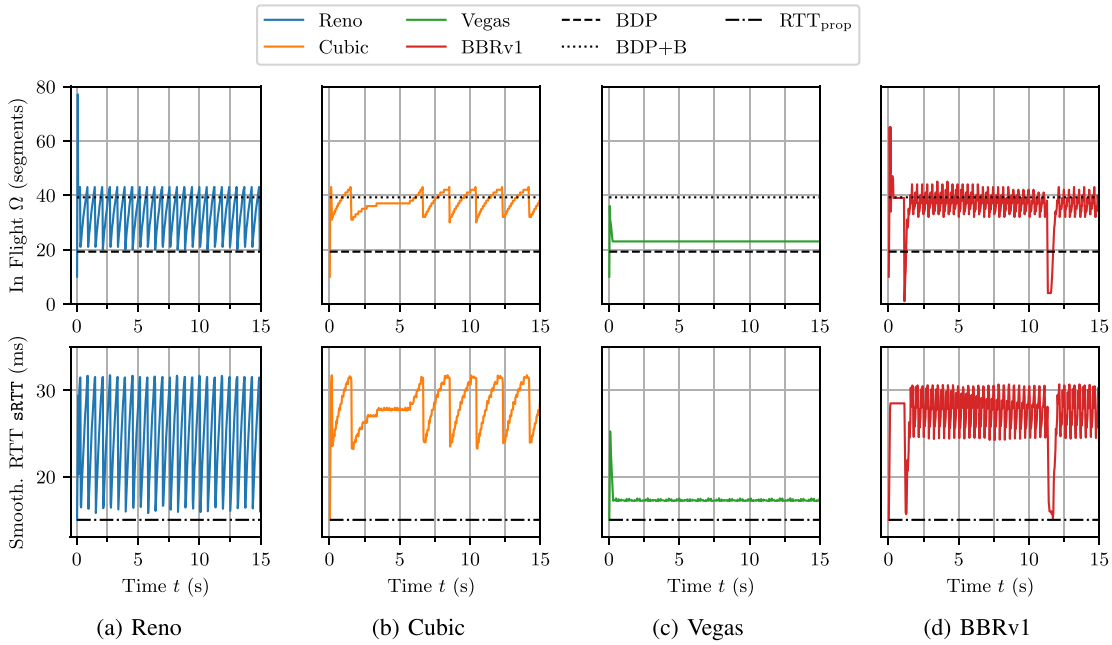


Fig. 5. Time series plots of the number of packets in flight Ω and smoothed round-trip time $\mathtt{sRTT}$ for Reno, Cubic, Vegas, and BBRv1 in a steady-state experiment with the default path parameters of Table IV.

b) Mean operating point: Based on the modeling of the waveform, the mean operating point of a CCA can be modeled as well. Recall that the mean operating point is an averaged allocation-delay pair, e.g., the throughput-sRTT pair $(\mathbb{E}\{\lambda(t)\}, \mathbb{E}\{\mathtt{sRTT}(t)\})$. The *response function* of classical TCP states that the mean window of Reno [158], [159], [161], [162] is a function of the loss event rate p with

$$\mathbb{E}\{w_t\} \approx \sqrt{\frac{3}{2}} \cdot \frac{1}{\sqrt{p}} \approx 1.2 \cdot \frac{1}{\sqrt{p}}. \tag{8}$$

The mean throughput can be approximated with $\mathbb{E}\{\lambda_t\} \approx \mathbb{E}\{w_t\}/\mathbb{E}\{\mathtt{sRTT}_t\}$, therefore, Eqn. (8) is also known as the *throughput formula* of TCP.

The response functions of other loss-based CCAs have been modeled in a similar way [76], [127], [135], [163], [164], [165], [166], [167]. The throughput of delay-based or hybrid CCAs can also be modeled; however, since they (also) use other congestion signals than loss events, their throughput formulas are not only functions of p [168]. A delay model may possibly be derived from the allocation in a network with a FIFO bottleneck queue. The expected bottleneck queue length is given by $\mathbb{E}\{b_t\} \approx \min(B, \max(0, \mathbb{E}\{\Omega_t\} - \mathtt{BDP}))$, the expected queueing delay by $\mathbb{E}\{D_t^q\} \approx \mathbb{E}\{b_t\}/C$, and the expected sRTT by $\mathbb{E}\{\mathtt{sRTT}_t\} \approx \mathtt{RTT}^{\mathrm{prop}} + \mathbb{E}\{D_t^q\}$.

4) Experiments: To validate the analytical modeling of the waveform or to evaluate the waveform of CCAs that cannot be modeled, it is common to evaluate time series from experiments. Typically, the allocation $x(t)$ is plotted, often in conjunction with a delay variable, to visualize the waveform of the operating point variables, e.g., in [4], [77], [115], [116], [118], [164], [169], as illustrated in Fig. 5 (which will be discussed in Section VIII-A5). Averages of the operating point variables of multiple CCAs are often shown comparatively in a throughput-delay plane, e.g., in [11], [137], [138], [149], [170], [171], as illustrated in Fig. 10. A less common way to report the operating point of a CCA measured in experiments is to show empirical Cumulative Distribution Functions (eCDFs) of the throughput [77], [135], [149] or the delay [172], [173].

5) Experiments From ccperf: The *Single Flow* experiment of Table V evaluates the operating point in a static scenario with a single flow ($K = 1$). Fig. 5 shows the number of packets in flight $\Omega(t)$ and the $\mathtt{sRTT}(t)$ of Reno [72], Cubic [76], Vegas [118], and BBRv1 [77] operating in isolation ($K = 1$) in a network with the default parameters of Table IV. An independent simulation was run for each CCA. Reno, Cubic, and Vegas use the cwnd to limit the number of packets in flight, whereas BBRv1 uses the pacing rate. All CCAs start with an slow start phase that rapidly ramps up the number of packets in flight. Reno, Cubic, and BBRv1 enter LR after detecting a loss event. Reno and BBRv1 overshoot $\mathtt{BDP} + B$ significantly. The CCAs retransmit lost packets until the loss event is resolved and the LR is exited. BBRv1 experiences an RTO during the process. Then, all three CCAs enter CA. In contrast, Vegas detects a growth of the sRTT and exits slow start before a loss event occurs. In CA, all CCAs operate in the congestion region where $\lambda_t \approx C$ and the sRTT is approximately proportional to the number of packets in flight with $\mathtt{sRTT}_t \approx \mathtt{RTT}^{\mathrm{prop}} + (\Omega_t - \mathtt{BDP})/C$. Reno's AIMD scheme produces the characteristic sawtooth function of the packets in flight.

Cubic follows the cubic function of Section VIII-A3a during CA that is most visible from $t \approx 2$ s to $t \approx 6$ s with an inflection point at around $t = 4$ s. The concave portion on the left side of the inflection point ramps up the number of packets in flight quickly to a target value where Ω_t plateaus. The convex portion on the right side of the inflection point probes for new bandwidth. The convex growth begins slowly

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

but accelerates quickly. After $t \approx 6$ s, Cubic enters its steady-state behavior. It repeatedly ramps up cwnd with concave growth to a target window of approximately $\texttt{BDP} + B$. At the end of this growth period, a packet is lost, the cwnd is reduced by a factor $\beta = 0.7$, and the oscillation repeats.

In contrast to Reno and Cubic, Vegas does not oscillate around its mean operating point and operates in the $\alpha < \delta < \beta$ region of Eqn. (7). Furthermore, Vegas has a low sRTT as it avoids queueing delays by not filling up the bottleneck buffer. The operating point of Vegas is close to the optimal operating point described in Fig. 4.

Similar to Vegas, BBRv1 is expected to also avoid queueing delays and to operate with a number of packets in flight near the BDP. However, the number of packets in flight Ω of BBRv1 oscillates around $\texttt{BDP} + B$ in our experiment. The simulation logs of our ns-3 experiment suggest that BBRv1 overestimates the bottleneck rate, which is used to set the pacing rate. Apart from the ($+B$) offset, the oscillations of Ω depict the expected waveform of BBRv1, which alternates between different gain factors to slightly increase or decrease Ω sequentially. At approximately $t = 11$ s, BBRv1 executes its *ProbeRTT* state that reduces Ω to a small value with the aim to drain the bottleneck queue and to measure the two-way propagation delay $\texttt{RTT}^{\text{prop}}$.

The modeling of the mean window of Reno in Eqn. (8) is validated in our simulation experiment shown in Fig. 5. Reno experiences one loss event every $w_{\max}/2$ RTTs and transmits $(w_{\max}/2) + (w_{\max}/2 + 1) + \cdots + (w_{\max}) \approx (3/8)\, w_{\max}^2$ segments between two consecutive loss events, where $w_{\max} = \texttt{BDP} + B$. Hence, plugging in the loss event rate $p \approx 1/[(3/8) w_{\max}^2]$ in Eqn. (8) yields a mean window of $\mathbb{E}\{w_t\} = \overline{w} \approx (3/4)\, w_{\max} = 30$ segments. The mean window can also be approximated with the AIMD decrease factor $\beta = 1/2$ of Eqn. (3) as the arithmetic mean of $w_{\max}$ and $\beta w_{\max}$ as $[w_{\max}(1+\beta)]/2$, which also yields 30 segments.

The average window $\overline{w} \approx 0.85 w_{\max}$ of Cubic is higher compared to Reno, mainly because Cubic uses a larger value for β. Vegas settles in on a constant number of packets in flight slightly above the BDP with $\Omega_t = \texttt{BDP} + 3$ segments. BBRv1 operates with $\overline{\Omega}$ slightly below $\texttt{BDP} + B$ due to an overestimation of the bottleneck rate.

The mean delay can be derived from the mean allocation in our experiment, which uses the default path parameters of Table IV. The mean queue length is given by $\overline{b} \approx \min(B, \max(0, \overline{\Omega} - \texttt{BDP}))$, the mean queueing delay by $\overline{D^q} \approx \overline{b}/C$, and the mean sRTT by $\overline{\texttt{sRTT}} \approx \texttt{RTT}^{\text{prop}} + \overline{D^q}$. For example, Reno approximately induces a mean queue length of 10 packets and a mean sRTT of 22.5 ms.

B. Compete Against Yourself

1) Scenario: This section covers intra-protocol competition evaluations in which multiple ($K > 1$) long-living bulk transfer flows compete against each other in a static scenario. Evaluations with short-lived flows are covered in Section XII. The scenario extends the scenario Section VIII-A, which evaluates the steady-state behavior of a single flow in a static scenario. The most commonly used topology is the dumbbell topology of Fig. 3.

We distinguish different competition scenarios as explained in Section VI-E: a synchronous fairness scenario, a start-time fairness scenario, and an RTT fairness scenario. A CCA may converge to different equilibria depending on the competition scenario.

2) Metrics: The scenario is mainly used to quantify the performance of the steady-state equilibrium $\boldsymbol{x}^e(t)$ of the $K > 1$ flows. All metrics of Table III are applicable. The equilibrium $\boldsymbol{x}^e(t)$ and related operating point metrics, e.g., the cwnd, throughput, and delays, are commonly evaluated over time. The avoidance of self-inflicted queueing delay has risen in importance over the last years. Furthermore, the fairness and utility metrics of Section VI-D are often evaluated for the equilibrium, both theoretically and experimentally. Jain's index, see Eqn. (6), is widely used as a fairness function. Synchronous, start-time, and RTT fairness are commonly evaluated in scenarios with different initial conditions.

3) Theoretical Modeling:

a) Waveform: The equilibrium $\boldsymbol{x}^e(t)$ can be modeled based on the modeling of waveforms of individual flows, which we explained in Section VIII-A. For some CCAs, the convergence to equilibria that maximize specific utility functions in the framework of Eqn. (2) can be shown analytically. In steady-state scenarios that fulfill some assumptions, AIMD algorithms, such as Reno, maximize Eqn. (5) and converge to an equilibrium that is minimum potential delay fair [80], [81], [141]. AIMD schemes converge regardless of the start times of flows; hence, Reno is fair in synchronous and start-time fairness scenarios. On the other hand, AIMD schemes do not achieve RTT fairness—they have a bias against flows with long RTTs [85], [135], [174]. Since the cwnd of Reno is approximately increased by one segment each RTT, flows with larger RTTs increase cwnd slower over time.

Vegas, FAST, and Compound TCP (CTCP) are proportionally fair as they maximize Eqn. (4) [135], [160], [167]. These proofs are based on theoretical network models that are abstractions of real networks. More details regarding this topic can be found in Section VIII-B3e.

b) Fluid models and game theory models: Fluid models are another method to model the operating point of CCAs over time. Such fluid models solve stochastic differential equations to obtain the operating point as a function of time and network parameters [175]. Another method uses game theory [138], [149], [170] to prove the convergence of CCAs. Such proofs formulate CC as a non-cooperative game, where each flow individually tries to maximize its utility by adjusting its transmission rate. Under certain conditions, such games can be shown to converge to stable equilibria.

c) Mean joint operating point: In Section VIII-A3b, we explained that the mean operating point of a single flow is given by the pair of the flow's mean throughput and mean sRTT. We can generalize this concept to a mean joint operating point for multiple competing flows that share a bottleneck queue. For example, the mean joint operating point can be defined as the pair of the mean aggregate throughput and mean queueing delay, i.e., as $(\mathbb{E}\{\sum_k \lambda_k(t)\}, \mathbb{E}\{D^q(t)\})$. This definition assumes that all flows share the same queue at the bottleneck and therefore experience the same queueing delay. With the mean joint operating point, CCAs can be compared

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

in terms of efficiency (bottleneck bandwidth utilization) and avoidance of queueing delay. CCAs whose mean operating point can be modeled for a single flow often also allow modeling of the mean joint operating point of multiple flows [141], [160], [170], [176].

d) Pareto-optimal solutions: Both minimum potential delay fair and proportional fair solutions are Pareto-optimal solutions in the framework of Eqn. (2). These solutions mark different tradeoffs between efficiency and fairness. Consider the following example for the parking lot topology with $K = 3$ [81]. In the example, s_1 competes for resources with both s_2 and s_3, whereas s_2 and s_3 do not compete with each other. The respective system conditions of the optimization problem are $x_1 + x_2 \leq 2$ and $x_1 + x_3 \leq 1$. Suppose a first equilibrium is $\boldsymbol{x}^a = [0.5, 1.5, 0.5]^{\mathrm{T}}$ and a second equilibrium is $\boldsymbol{x}^b = [0.1, 1.9, 0.9]^{\mathrm{T}}$. Both $\boldsymbol{x}^a$ and $\boldsymbol{x}^b$ are capacity-achieving Pareto-optimal solutions as they fully saturate both C_1 and C_2. However, $\boldsymbol{x}^b$ is more efficient in terms of the aggregated allocation ($\sum_k x_k^b = 2.9$ in comparison to $\sum_k x_k^a = 2.5$), but less fair than $\boldsymbol{x}^a$ as s_1 is almost starved ($x_1 = 0.1$). Whether $\boldsymbol{x}^a$ or $\boldsymbol{x}^b$ is preferable is a matter of choice in terms of a tradeoff between efficiency and fairness. Each utility function encodes a specific preference with respect to that tradeoff. In the CC literature, there is no consensus on the utility function that is most suitable to evaluate CCAs. Nevertheless, proving that a CCA converges to an equilibrium that maximizes a well-defined utility function is regarded as a theoretical proof of proficiency (which represents the quality that a CCA is optimal with respect to some utility function). The convergence of CCAs to a fair equilibrium is considered as a crucial component for the stability of the Internet in [177].

e) Theoretical assumptions of models: The proofs of convergence in Section VIII-B3a are based on theoretical network models that are abstractions of real networks. The proofs make assumptions that have to be fulfilled for the proof to hold. If any assumption is violated, these proofs make no statement about the convergence. In the following, we present a few examples and explain how these assumptions translate to an evaluation scenario.

The proof that an AIMD scheme converges to a fair equilibrium makes the assumption of synchronized feedback; that is, ACKs arrive at competing senders simultaneously. Synchronized feedback is a strong assumption as it implies that many system conditions are fulfilled. The RTTs of competing flows have to be the same, which disqualifies RTT fairness scenarios. Furthermore, packets are not randomly delayed or dropped, and congestion losses of competing flows occur simultaneously. For that to be approximately true, a static network with a FIFO queue and without random losses is required. Hence, these assumptions are approximately fulfilled by only a subset of the steady-state evaluation scenarios that we cover in this section.

The proof of convergence for TCP Vegas [160] assumes that all flows accurately know the $\mathtt{RTT}^{\mathrm{prop}}$ of the path they use. However, in reality, $\mathtt{RTT}^{\mathrm{prop}}$ is often unknown and has to be estimated, which can introduce an estimation error. The effect of the estimation error is quantified analytically in [160] and empirically in [178], [179]. An overestimation typically occurs in a situation where a new flow starts while the bandwidth is already saturated by other flows and a queue backlog exists. Then, the new flow overestimates $\mathtt{RTT}^{\mathrm{prop}}$ as it misinterprets queueing delay as propagation delay. Due to the overestimation, the flow then acquires a proportionally larger share of the bandwidth. Therefore, Vegas may discriminate against flows that start earlier. Vegas is start-time fair as long as $\mathtt{RTT}^{\mathrm{prop}}$ is estimated correctly by all flows; however, if only a single flow has an estimation error then this is not the case anymore.

These two examples illustrate that the convergence of a CC depends on the conditions a scenario determines, possibly in subtle ways. Hence, labeling a CCA as provably fair should be done with care, as it may not generalize beyond narrow assumptions that are often violated in real networks.

4) Experiments: The experimental evaluation of the steady-state behavior of CCAs is very common in the literature. Studies that propose new CCAs typically include steady-state experiments. The experiments provide empirical results for the steady-state behavior of a CCA and may also validate the theoretical modeling. Suppose that a series of experiments is conducted. Each experiment repetition may use other scenario parameters, e.g., different start times of flows. Depending on the steady-state behavior of the evaluated CCA, a few qualitatively different convergence properties can be distinguished.

- The CCA may not reliably converge to a steady-state behavior such that $\boldsymbol{x}(t)$ follows no pattern, i.e., $\boldsymbol{x}(t)$ is neither periodic nor constant. In this case, a CCA does not exhibit a steady-state behavior in the classical sense. Learning-based CCAs may fall into this category. The metrics of Section VIII-B2, such as the aggregated throughput or fairness, are still applicable for the evaluation.
- The CCA converges to a steady-state behavior, but the equilibrium $\boldsymbol{x}^e(t)$ changes when the experiment is repeated. For example, in a first experiment a CCA could converge to $\boldsymbol{x}^e(t) = (1, 2)$ and in a second experiment converge to $\boldsymbol{x}^e(t) = (2, 1)$. The differences may originate from the CCA itself, if random decisions are part of a CCA's design, or because the scenario parameters are changed for each repetition.
- The CCA reliably converges to the same equilibrium in the series of experiments.

a) Synchronous fairness: In a synchronous fairness experiment, all flows start simultaneously and the two-way propagation delays of flows are the same. Experiments that evaluate synchronous fairness are covered in [76], [116], [167], [169], [180], [181].

b) Start-time fairness: In a start-time fairness experiment, the start times of flows are varied. The two-way propagation delays of flows are the same. Start-time fairness experiments are covered in [11], [77], [118], [138], [149], [164], [180], [182], [183]. Since it is unlikely that flows start simultaneously in real networks, the start-time fairness scenario is more realistic than the synchronous fairness scenario. Most CCAs are designed to be start-time fair, i.e., have no biases against flows starting early or late. As a counterexample, the experiments

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

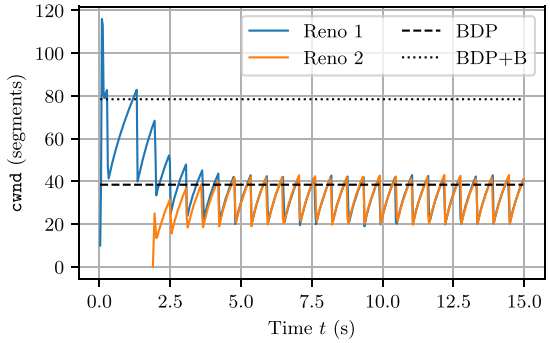


Fig. 6. Two Reno flows competing against each other in a steady-state experiment with the default path parameters of Table IV. The flows are synchronized from $t_c = 4$ s onward.

in [168] show that Vegas can have a bias against flows starting earlier, as explained in Section VIII-B3e.

c) RTT fairness: RTT fairness experiments can be found in [76], [138], [165], [167], [169], [170], [180], [182]. Many CCAs have a bias against flows with longer RTTs. Without countermeasures, the bias is a somewhat natural property of a CCA as the RTT determines the feedback delay of the control loop. Some CCAs, such as Cubic or Compound [167], are designed to reduce the bias and labeled as RTT-fair. BBRv1 has been observed to show a bias against flows with shorter RTTs [184]. However, a bias against shorter RTTs is rare.

d) Synchronization: The synchronization of flows with the same bottleneck has been observed in Wide Area Networks (WANs) that resemble our steady-state scenario [185]. When multiple flows reduce their transmission rates synchronously, their aggregate transmission rate may underutilize the path capacity. Furthermore, the aggregated transmission rate exhibits pronounced oscillations. The disadvantages of flow synchronization are discussed in [185], [186], [187]. AQM methods, such as Random Early Detection (RED) [85], are specifically designed to avoid flow synchronization.

e) Phase effects: The periodic nature of traffic can significantly impact the equilibrium—a phenomenon named phase effects [150]. Periodic traffic is created either directly by the application's traffic generation or by CCAs that shape traffic transmission timings. Suppose that two flows using loss-based CCAs share a bottleneck and generate periodic traffic. The bottleneck queue is almost full and about to overflow once the next packets arrive from either flow. Suppose the packets of the first flow are dropped due to an overflow, whereas the packets of the second flow arrive later in time and can be serviced. In the short term, only the first flow reduces its rate in response to the loss event. Due to the periodicity of the traffic, it is possible that this situation (first flow reduces its rate, the second flow does not) repeatedly occurs over time, i.e., the two flows synchronize to an equilibrium that heavily favors the first flow. Note that if the arrival times of packets would have been slightly different, the situation could be reversed. Hence, small changes of the network or traffic models may lead to different phase effects that can change the equilibrium in a qualitative manner, i.e., fundamentally change the equilibrium for the two flows. Whether or not certain phase effects will occur in an experiment is hard to predict beforehand. Phase effects can be mitigated by randomizing packet transmission timings or by AQM algorithms that avoid flow synchronization at queues [85].

f) Bursty traffic: Related to phase effects is the known bias against flows that generate bursty traffic [150], [174]. Bursty traffic is more likely to overflow a queue. Suppose a flow sends three packets in a burst and then stops sending packets due to being cwnd-limited. The bottleneck queue, however, has only two vacant slots to enqueue packets. As the packets arrive shortly after each other, the queue may have to drop the third packet (overflow). If the packets were spread out over time, the queue can dequeue packets for transmission before new packets arrive and thereby avoid the overflow. The example illustrates how flows that generate bursty traffic may experience more packet losses, which may trigger some CCAs to reduce the transmission rate. Hence, when a bursty flow competes with a non-bursty flow, the bursty flow is likely to reduce the transmission rate more frequently and acquire less bandwidth resources. Bursty traffic may either originate from the application's traffic generation pattern itself or after a sudden increase of the cwnd. Pacing transmissions can help to reduce the burstiness of traffic at the sender [95]. Furthermore, qdiscs which randomize packet drop events, such as RED, can remove the bias against bursty traffic [85].

g) Degree of multiplexing: The majority of studies have evaluated CCAs in experiments where the number K of competing flows (degree of multiplexing) is small. However, the degree of multiplexing can be high in the Internet. Some CCAs, such as Reno, exhibit decreased efficiency and fairness as K grows [127], [138], [188]. The accuracy of related theoretical models may depend on K as well, as shown in [189], [190].

5) Experiments From ccperf: The *Synchronous Fairness*, *Start-time Fairness*, and *RTT Fairness* experiments of Table V cover intra-protocol competition steady-state scenarios in accordance with the taxonomy explained in Section VI-E. Fig. 6 shows the cwnd from a simulation experiment of a *Start-time Fairness* experiment with $K = 2$ Reno flows. The flows start uniformly at random between zero and three seconds. At around $t_c = 4$ s the cwnd converges to an equilibrium. Then, the cwnd of each flow follows a sawtooth function that is synchronized (in-phase) with its competitor.

C. Summary

Steady-state evaluations have been conducted since CCs were proposed as a distributed control problem. Steady-state evaluations are closely related to the modeling of CCAs as a dynamical system as explained in Section II-A, in which long-running flows compete in a network. In such a scenario, CC is formulated as a resource allocation problem as explained in Section II-B. A CCA is supposed to converge to a stable, efficient, and fair steady-state operating point. Numerous studies have theoretically modeled CCAs as a dynamical system in steady-state scenarios. Experimental studies are commonly used to validate and augment analytical models.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

Steady-state studies have long focused on evaluating the throughput and fairness of CCAs, whereas latency was omitted. More recent studies evaluate packet delays as well, because the importance of achieving low latencies has risen. Furthermore, bufferbloat (Section X-C) is a known problem in which CCAs play a crucial role. In the past, CCAs were not designed to avoid self-inflicted queueing delay. Today, the expectation towards a CCA to keep the queueing delays small has grown.

For a long time, steady-state evaluations were the focus of CCA evaluations. This focus has reduced over time and a wider range of scenarios is deemed important today [156]. Scenarios with short flows and dynamic network or traffic conditions are some examples. Nevertheless, steady-state scenarios are a cornerstone of CCA evaluations.

IX. Transient-State Evaluation

Before a CCA converges to an equilibrium it goes through a transient-state phase. We also refer to the transient-state phase as the convergence phase. In this section, we cover evaluations that focus on assessing the transient phase of CCAs in intra-protocol competition evaluations. All inter-protocol evaluations are surveyed in Section XII. There are different situations that initiate and constitute the convergence phase.

A flow typically starts with a transient slow start phase, in which the transmission rate is ramped up to quickly utilize bandwidth resources. After the slow start phase finishes, it may take CCAs considerably more time until they converge to an equilibrium. They may complete several CA and LR phases before they eventually converge as illustrated in Fig. 6.

Another situation arises when the equilibrium itself shifts after a system condition of the experiment changes, e.g., a network parameter changes or a new flow joins the competition. Then, a new convergence phase begins in which the established flows respond to the change.

A. Slow Start

In the slow start phase, a CCA typically ramps up its transmission rate quickly. The slow start phase is typically initiated when a flow starts, after an RTO occurs, or after an idle period.

1) Scenario: Since each flow starts with a slow start phase, slow start algorithms can be evaluated in the same scenarios that also evaluate CA algorithms. For example, in the experiments shown in Fig. 5 each CCA begins with a slow start phase that can be separately evaluated. Alternatively, short flows that finish quickly can be set up. These short flows spend their entire or the majority of their lifetime in slow start. For such experiments, flow metrics may be averaged over the entire lifetime of a flow to quantify the slow start algorithm performance.

A TCP/IP stack may cache state information across connections, which can significantly speed up slow start [191], [192]. When a new socket initiates a connection to a known host, the socket can initialize state variables with cached values for that host, e.g., initialize `cwnd` with its cached value of the past. Reusing cached state information can significantly speed up flow completion times of short flows. However, most CCA evaluations use empty caches in experiments unless the experiments are specifically designed to evaluate caching mechanisms.

2) Metrics: Slow start algorithms aim to acquire bandwidth resources quickly without causing congestion losses that require retransmissions. The flow completion time is the metric that matches these goals precisely, especially for small flows that spend their entire or the majority of their lifetime in slow start. For small flows, avoiding packet loss is crucial to achieve short flow completion times, as retransmission may require multiple RTTs to repair [193] before the flow finishes.

The flow completion time of a large flow also depends on the slow start algorithm, even though the influence may be smaller. Once the slow start algorithm exits, a CA or LR algorithm takes over the adjustment of the control variable (assume cwnd is used). In that moment, the cwnd may either be in the underutilization, congestion, or congestion collapse region of Fig. 4. It is rare that the cwnd exactly matches the BDP right after slow start exits. If the cwnd is too small (underutilization), the flow may underutilize the bandwidth resources for a prolonged time. In contrast, if the cwnd is too large (congestion collapse), a burst of congestion losses may occur that deteriorate the flow's performance. Hence, slow start may affect the performance of a flow beyond the slow start exit. In scenarios with competing flows, the slow start influences the convergence speed to an equilibrium [97]. A CCA may fail to acquire its fair bandwidth share such that the CA algorithm has to correct for it afterward; or a CCA may aggressively acquire an unfairly large amount of bandwidth resources that are yielded by other flows [194]. Both situations likely increase the time until the equilibrium is reached.

3) Theoretical Modeling: The classical slow start algorithm of Reno doubles the `cwnd` per RTT, which results in an exponential growth over time. Analogously to the waveform modeling of CA algorithms in Section VIII, the slow start algorithm can be modeled with an equation similar to Eqn. (3). Assuming no packets are lost and delayed ACKs are disabled, the cwnd is given by $\texttt{cwnd}_t = \texttt{cwnd}_0 \cdot 2^t$ after t RTTs, where $\texttt{cwnd}_0$ is the initial cwnd (defaults to 10 segments in Linux) [2]. Hence, $n = \log_2(\texttt{cwnd}_n/\texttt{cwnd}_0)$ RTTs are required to reach $\texttt{cwnd}_n$. The model can be extended to account for packet loss and the possibility of RTOs [195]. Modeling how quickly a slow start algorithm reaches a certain throughput is of interest since rapid growth is one of the main slow start objectives. Furthermore, the expected flow completion time of small flows can be deduced from the model. The flow completion time is proportional to the RTT, which is problematic in networks with large propagation delays, such as satellite networks [196]. An alternative slow start algorithm is proposed and modeled in [183].

4) Experiments: Various experiments are conducted in [97], [196] to evaluate slow start algorithms. The evaluations examine time series plots and show statistics about the flow completion times for different slow start algorithms. A challenge for slow start algorithms is to decide when to exit the slow start phase. Overshooting and premature exits are two common problems of slow start algorithms.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

a) Overshooting: The exponential growth during the slow start phase can severely overshoot the path capacity C [97], [127], [197], [198]. Window-based schemes can overshoot by up to a factor of two before they detect a loss event as we observe for Reno in Figs. 5 and 6.

Rate-based schemes are at risk of overshooting the path capacity even more because they are not ACK-clocked. After such an overshooting event which causes many packet losses, LR may take several RTTs to retransmit lost packets. In the worst case, overshooting may cause an RTO and a repeated slow start phase. Overshooting can significantly increase the file completion time, especially in networks with large propagation delays.

b) Premature exit: Contrary to the overshooting problem is the problem of premature exits of the slow start phase [79], [97], [199]. Exiting slow start too early with a low transmission rate is problematic because CA is then responsible for growing the transmission rate to utilize resources efficiently. However, CA algorithms typically grow the transmission rate slowly and therefore may underutilize bandwidth resources for a long time.

B. Loss Recovery (LR)

The classical TCP algorithms that constitute the LR algorithm are fast retransmit and fast recovery [1]. The LR algorithms of the classical TCP were iteratively refined to improve various shortcomings.

Fast recovery algorithms control or adjust the transmission rate during LR. The goal of fast recovery is to maintain a high throughput at the transport layer while retransmissions repair losses. The TCP specification explicitly allows the usage of advanced fast recovery algorithms instead of the classical algorithm defined in [1]. The NewReno modifications [72] add a window inflation mechanism to the classical fast recovery algorithm. Linux uses the PRR algorithm as its default for fast recovery [98].

A distinct class of LR algorithms handles the processing of ACKs, loss detection mechanisms, and retransmission timings. The main goal of these LR algorithms is to recover losses as quickly as possible—ideally within one RTT [193]. Reliable transport layer protocols starve applications during LR when a loss prevents in-order delivery. Recovering from losses quickly is important for real-time applications that need to stay responsive and for achieving short flow completion times.

With the fast retransmit algorithm, a sender immediately retransmits a segment after receiving the third duplicate ACK (dupACK) without waiting for a retransmission timeout. The Selective ACKnowledgement (SACK) option [200] added a selective repeat request mechanism to TCP. Furthermore, the Duplicated Selective ACKnowledgement (D-SACK) extension [201] allows senders to track the packet reception order at the receiver. With the extension, a sender can detect if a packet was falsely retransmitted more than once. By using Forward ACKnowledgements (FACKs) [202] the tracking of the number of packets in flight is improved when SACKs are used and fast retransmit is triggered earlier.

1) Scenario: Experiments to evaluate LR algorithms either rely on queue overflows or random losses to produce packet loss, or use mechanisms to drop specific packets purposefully [98], [196], [203]. The purposeful packet dropping has the advantage of creating the same reproducible packet loss scenario in independent evaluations, which facilitates a direct comparison of LR algorithms.

2) Metrics: Metrics that are related to packet loss are essential to evaluate LR scenarios. The number of lost packets, retransmissions, and RTOs are commonly reported in tables. Furthermore, the time spent in LR and flow completion times (or response times of Web traffic) quantify how quickly losses can be recovered. Packet losses and retransmissions also increase the total usage of system resources and power consumption of flows due to increased flow completion times. The consumption of system resources depends on the CCA [140]. The throughput at the transport layer is the main metric to measure the performance of fast recovery algorithms.

3) Theoretical Modeling: The theoretical modeling of LR algorithms is not very common in the CC literature, because packet loss is treated as a random event, which prevents analytical modeling to a large degree.

4) Experiments:

a) Fast recovery algorithms: Time series plots, e.g., packets in flight over time, may be used to visualize how the fast recovery algorithm adapts the transmission rate during LR [98], [196]. Various metrics, such as the flow completion times, time spent in LR, number of retransmissions, and the fraction of flows that experience slow start after LR are used in [98], [193] to examine fast recovery algorithms. Reducing flow completion times and minimizing the time spent in LR is especially important for Web or streaming traffic. A fast recovery algorithm that is too aggressive may produce additional packet loss, which prolongs LR and flow completion times. On the other hand, a fast recovery algorithm that is not aggressive enough may reduce the throughput during LR, which also prolongs flow completion times.

b) Tail-loss problem: The tail-loss problem arises when a sender transmits a burst of packets and one of the last packets in that burst is lost. When less than three segments arrive at the receiver after the loss, the receiver cannot generate three dupACKs to trigger LR at the sender. Hence, the sender falls back to RTO-based recovery. Such behavior was observed in Web traffic generating packet bursts [204] and with connections that operate with small window sizes [205]. The limited transmit algorithm [206] tries to mitigate that issue by allowing senders to transmit new data after receiving the first two dupACKs, whereby the new data can extend to two segments beyond the current congestion window. Similarly, the early retransmit algorithm [207] lowers the dupACK threshold at the sender if less than four outstanding segments are in transit. Another option is to use redundant or probe packet transmissions to trigger the generation of ACKs at the receiver [193], [208].

The flow completion time (or Web server response times, in case of Web traffic) and retransmission statistics are used in [193], [204], [205] to quantify performance in tail-loss scenarios.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

c) Spurious time outs and retransmissions: Spurious timeouts occur when a sender does not wait long enough for an ACK to arrive and an RTO is triggered. A spurious timeout may either be a consequence of an underestimation of the RTT at the sender or because the receiver experiences a connection outage, in which ACKs are held back. Spurious retransmissions, on the other hand, are a consequence of packet reordering. Each out-of-order segment that arrives at the receiver triggers the transmission of a dupACK. Hence, more than three (dupACK threshold) out-of-order segments will eventually cause an LR phase at the sender, even though packets were not lost but rather reordered. Spurious timeouts and retransmissions can be detected in retrospect with the Eifel detection algorithm [203], Forward-RTO-Recovery [209], and with the D-SACK extension [210]. The Eifel response algorithm [203] undoes cwnd reductions caused by spurious retransmissions or timeouts. In [203], packets are delayed or reordered purposefully to cause spurious retransmission or timeouts in experiments. The time series of the highest acknowledged sequence number is used to analyze how TCP responds to such events.

C. Convergence Phase

To evaluate the convergence properties of a CCA, we formulate a CCA evaluation as a dynamical system as explained in Section II-A. The allocation $\boldsymbol{x}(t)$ starts with an initial state $\boldsymbol{x}(t_0)$ and may converge to its steady-state equilibrium $\boldsymbol{x}^e(t)$ at the time of convergence $t_c \geq t_0$. The convergence phase starts at the time t_0 with the initial state and ends at the time t_c when the state converges. Convergence phase evaluations examine convergence properties, such as the existence of an equilibrium, the convergence speed, or the stability of the equilibrium.

1) Scenario: The convergence of a CCA is evaluated in an intra-protocol competition scenario. In inter-protocol competition scenarios, the convergence properties cannot be attributed to a single CCA. Any steady-state evaluation scenario of Section VIII may be used to evaluate the converge phase. Scenario parameters may be varied to evaluate how the change of a parameter changes the convergence to the equilibrium.

2) Metrics: The convergence phase is a transient phase that occurs before a CCA reaches its steady-state behavior. The allocation $\boldsymbol{x}(t)$ may operate at a worse operating point compared to the steady-state equilibrium. Hence, the efficiency, fairness, and delay metrics may be used to quantify the transient-state system. If only two flows compete, the convergence phase of the system $\boldsymbol{x}(t) = [x_1(t), x_2(t)]^{\mathrm{T}}$ can be visualized in a phase portrait [116], [141], [211].

The convergence speed measures how much time $\boldsymbol{x}(t)$ requires to converge to the equilibrium $\boldsymbol{x}^e(t)$. Commonly, studies plot time series data and visually infer when convergence has occurred. However, to rigorously evaluate when $\boldsymbol{x}_t$ converges in experiments, the convergence speed metric of Section VI-D3 may be used.

Although the stability metric of Section VI-B11 is often discussed in connection with other convergence properties of a CCA, the metric is typically not used for the convergence phase. Instead, the stability metric assesses the variation of the equilibrium after convergence.

3) Theoretical Modeling: Theoretical studies may evaluate if an equilibrium for a given dynamical system exists, i.e., if a CCA reaches its steady-state behavior. Many studies prove convergence for a specific CCA as explained in Section VIII-B3a; for example: Reno [80], [141], Hamilton TCP (HTCP) [146], Illinois [169], BIC [165], Vegas [118], [160], [212], FAST [135], CTCP [167], and Copa [170], [213]. Furthermore, the convergence speed is analyzed in [80], [81], [145], [160], [164], [166], [169]. For which scenarios these proofs are applicable is defined by the respective dynamical system models. A single proof may be valid for many scenarios and a range of scenario parameters. Suppose a static scenario is used with $\mathcal{S}(t) = \mathcal{S}_0$. The scenario parameters $\mathcal{S}_0$ are typically function parameters of the equilibrium solution $\boldsymbol{x}^e(t; \mathcal{S}_0)$. A small change $\delta \neq 0$ of a scenario parameter may also lead to a small change of the equilibrium such that $\boldsymbol{x}^e(t; \mathcal{S}_0 + \delta) = \boldsymbol{x}^e(t; \mathcal{S}_0) + \varepsilon$ for some small ε, e.g., a small change of the bottleneck rate leads to slightly different allocations. In contrast, if a small parameter change leads to a qualitatively different equilibrium, a bifurcation occurs [150], [157]. If a closed-form solution for $\boldsymbol{x}^e(t; \mathcal{S}_0)$ is known, bifurcations can be deduced. In that case, the number of bifurcations and the different equilibria can precisely be predicted. However, since theoretical models are only abstractions of real networks, the models may lack the complexity to include some bifurcations that occur in real networks. As [157] suggests, a system of CCAs may exhibit chaotic behavior.

The stability of a dynamical system evaluates how the system behaves when $\boldsymbol{x}(t)$ is perturbed by a small amount. An equilibrium is stable if $\boldsymbol{x}(t)$ stays close to its equilibrium after the small perturbation. In contrast, the system is unstable if $\boldsymbol{x}(t)$ diverges away from the equilibrium. In this context, the stability metric of Section VI-B11 quantifies how close $\boldsymbol{x}(t)$ stays to the mean value of the equilibrium. A formal definition of different stability notions for a dynamical system and a rigorous stability analysis can be found in [80], [81], [84]. In the context of CCA evaluations, the initial state $\boldsymbol{x}(t_0)$ represents the initial allocations of the competing flows in Eqn. (1). A stable dynamical system implies that regardless of initial allocation the system remains close or converges to the same equilibrium—or in the context of Section VI-E, a stable system implies start-time fairness.

4) Experiments: The studies that conduct steady-state experiments, which we covered in Section VIII, often also discuss the convergence of CCAs.

The convergence speed of CCAs is examined in experiments in [76], [97], [138], [149]. Different convergence criteria are used in these experiments. The experiments show that the convergence speeds of CCAs differ between a few seconds and a few minutes, depending on the exact scenario. A slow convergence speed is undesirable because the allocation is likely less efficient and fair during the convergence phase.

Experiments may be repeated to gather evidence that a CCA converges for different initial conditions $\boldsymbol{x}(t_0) = \boldsymbol{x}_0$

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

and scenario parameters $\mathcal{S}_0$. These experiments may provide evidence for or against the stability of a CCA and uncover possible bifurcations. In real experiments that vary scenario parameters, bifurcations can be common. Within some interval of scenario parameters, a large number of bifurcations can exist and the equilibrium of a CCA may show chaotic behavior [157]. The chaotic behavior has the properties that small changes to the initial state $\boldsymbol{x}_0$ may significantly change the equilibrium in seemingly unpredictable ways. However, in contrast to experiments that involve randomness, a chaotic experiment produces the same results when repeated with the same initial conditions and system parameters.

5) Experiments From ccperf: The convergence phase evaluation can be conducted as part of the steady-state experiments of Table V. Fig. 6 shows the cwnd of a steady-state simulation experiment using the default path parameters of Table IV with $\texttt{BDP} = 40$ packets and $K = 2$ Reno flows that start at different times. Initially, the experiment goes through a transient phase. The first flow (Reno 1) starts at $t = 0$ s and overshoots the BDP during slow start, which inflicts a queue overflow and multiple congestion losses. Once the second flow (Reno 2) starts at around $t = 1.9$ s, the allocation is efficient $(\texttt{cwnd}_1(t) + \texttt{cwnd}_2(t) \geq \texttt{BDP})$ but unfair $(\texttt{cwnd}_1(t) > \texttt{cwnd}_2(t))$. At approximately 4 s, the allocation converges and the transient phase ends. Then, the allocation of each Reno flow follows a periodic sawtooth wave with the same phase and period, which constitutes an efficient $(\texttt{cwnd}_1(t) + \texttt{cwnd}_2(t) \geq \texttt{BDP})$ and fair $(\texttt{cwnd}_1(t) \approx \texttt{cwnd}_2(t))$ equilibrium.

D. Responsiveness

CCAs should be able to respond quickly to sudden changes. A rapid change can quickly produce congestion, e.g., a sudden decrease in bandwidth in a wireless link requires a CCA to back off. On the other hand, a bandwidth increase adds new resources that could be utilized. A sudden change in bandwidth is only one possibility—in theory, any path parameter can change. For example, after a route change, many parameters, such as the bandwidth and the two-way propagation delay, may change simultaneously. This can be a problem for CCAs that rely on the estimation of such parameters, e.g., Vegas and BBR [214].

Three traits are often the focus of evaluations that assess the responsiveness: i) how quickly a CCA responds to a congestion signal, ii) how much a CCA backs off after receiving the congestion signal, and ii) how aggressively a CCA increases its transmission rate to acquire spare bandwidth.

1) Scenario: The scenario to evaluate the responsiveness of a CCA typically uses a single flow and changes one or more path parameters suddenly. Typically, the bottleneck rate, the two-way propagation delay, or the error model is changed. The sudden change is either i) carried out once during an experiment, or ii) repeated periodically. For i), the path parameter follows a step function; for ii) it follows a pulse wave.

2) Metrics: The most typical way to examine the responsiveness of CCAs is to plot the allocation over time. The time series plots show in a qualitative manner if a CCA responds appropriately to congestion or newly created bandwidth resources. A more quantitative approach is possible with the responsiveness, smoothness, and aggressiveness metrics of Section VI-B.

3) Theoretical Modeling: The allocation $\boldsymbol{x}(t)$ of responsiveness scenarios has not been explicitly modeled by studies. However, the theoretical responsiveness, smoothness, and aggressiveness of AIMD schemes are covered in Section VI-B.

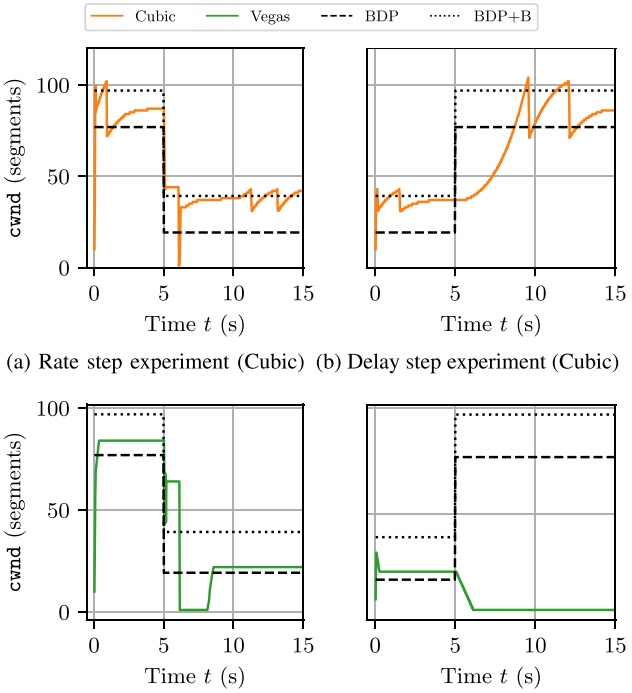
4) Experiments: Many studies evaluate the responsiveness of CCAs in experiments that match our step function responsiveness scenario. In [134], the packet error probability and bottleneck rate are changed by a step increase in two different scenarios that intend to assess the responsiveness and aggressiveness of CCAs. In the first scenario, the packet error probability is increased to measure the responsiveness by counting the number of RTTs until a CCA halves its transmission rate. The scenario imitates sudden congestion that CCAs should respond to. In the second scenario, the bottleneck rate is increased to measure the aggressiveness by assessing how fast a CCA grows its transmission rate to utilize the new capacity. The scenario resembles a situation where capacity is freed, e.g., by an ending flow.

Various studies examine how CCAs deal with channel conditions that change periodically. Commonly, either the bottleneck rate [215], [216], [217] or the two-way propagation delay [218] follows a pulse wave that alternates between a high and low level. The studies typically show time series plots of the allocation to visualize how quickly a CCA backs off or acquires resources. In [133], a similar scenario is achieved with a background flow. The background flow does not use a CCA and generates pulse-wave traffic. Thereby, the available capacity of competing flows also follows a pulse wave. In [211], the packet error probability is alternated between short periods of high loss (25 %) and long periods of low loss (0.5 %). Instead of alternating between two levels, [138], [149], [219], [220] randomly change the bottleneck rate, the two-way propagation delay, or the packet error probability every 5 s. They draw the values of the parameters randomly from bounded uniform distributions with lower and upper bounds.

5) Experiments From ccperf: The *Step Function Response* and *Pulse Wave Response* experiments of Table V set up responsiveness scenarios in ccperf. The two experiment types implement scenarios with a single parameter change and periodic parameter changes, respectively. Either the capacity, the two-way propagation delay, or the packet error probability may be changed; the choice can be specified by the `target` experiment parameter. In the *Step Function Response* experiment, the network parameter specified by `target` is either increased from a lower value specified by `lower`) to an upper value specified by `upper` or vice-versa be decreased as specified by `direction`. The change occurs at a time specified by `period`. The *Pulse Wave* experiment changes the network parameter value periodically with a period specified by `period` between the `lower` and `upper` values.

Fig. 7 shows the cwnd time series of responsiveness experiments that evaluate Cubic and Vegas. The experiments set up a single greedy-source flow on a network path with a controlled

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.



(a) Rate step experiment (Cubic) (b) Delay step experiment (Cubic)

(c) Rate step experiment (Vegas) (d) Delay step experiment (Vegas)

Fig. 7. Responsiveness experiments that change a network parameter abruptly at $t = 5$ s. In (a) and (c), the bottleneck rate decreases from 64 Mbits/s to 8 Mbits/s. In (b) and (d), the two-way propagation delay increases from 15 ms to 60 ms.

delay (CoDel) qdisc at the bottleneck. Each experiment either changes the capacity or propagation delay of the path at $t = 5$ s. All other path parameters are set to the default path parameter values of Table IV.

In Figs. 7(a) and 7(c), the capacity is decreased from 64 Mbits/s to 16 Mbits/s. The sudden capacity decrease immediately causes congestion as both Cubic and Vegas operate with a cwnd above the BDP. The congestion results in packet loss to which both CCAs respond by decreasing cwnd as they enter Loss Recovery. However, the cwnd reduction is not large enough and both CCAs keep operating with $\texttt{cwnd} > \texttt{BDP} + B$. At approximately 6 s both CCAs experience an RTO, which resets the cwnd to one segment. Eventually, Cubic manages to exit the following Loss Recovery phase faster than Vegas and converges to the new operating point first.

In Figs. 7(b) and (d), the one-way delay of the path is increased such that the two-way propagation delay $\texttt{RTT}^{\text{prop}}$ increases from 15 ms to 60 ms. Cubic and Vegas respond to this increase of $\texttt{RTT}^{\text{prop}}$ differently. Cubic increases cwnd as the *pipe has become longer*. The increase stops once a congestion loss occurs, at which point Cubic fully utilizes the capacity. In contrast, Vegas misinterprets the growth of the RTT samples as queueing delay, decreases cwnd and reduces the transmission rate. As a consequence, the throughput of Vegas deteriorates and the capacity is underutilized. After the step increase of the delay, Vegas' estimate of $\texttt{RTT}^{\text{prop}}$ is not updated and remains approximately equal to 15 ms. The experiment illustrates how the estimation of network parameters by CCAs can be wrong, which may degrade performance. Other CCAs that incorporate network models to some degree, such as Westwood or BBR, face similar challenges.

E. Summary

The transient-state evaluation of CCAs covers slow start, LR, the convergence phase, and the response of CCAs to sudden changes. A variety of scenarios are used to cover these aspects; for example, scenarios that purposefully drop packets to enforce packet loss or scenarios in which a network parameter changes abruptly. These scenarios are more dynamic and diverse in comparison to the steady-state scenarios. Therefore, the theoretical modeling of the transient-state behavior is more challenging and less common. Instead, many evaluations focus on experiments to evaluate CCAs in the transient-state phase.

The methods to evaluate slow start, LR algorithms, and the convergence phase have not changed significantly over time, except that the flow completion time has become the primary performance metric. Flow completion times of short flows are entirely or largely determined by the transient-state phase. Responsiveness scenarios, in which a network parameter is changed abruptly, have become more prevalent in recent studies. CCAs are expected to react quickly to parameter changes, such as changes of the bottleneck capacity or the propagation delay of a path.

A significant percentage of flows in real networks today are small. In contrast to long-running flows, small flows end before a CCA converges to steady-state behavior. Hence, transient-state evaluations are important for real network traffic and are required to thoroughly examine the performance of a CCA.

X. Network Variation

The operating point of a CCA depends on the network $\mathcal{G}(t)$. Studying how the operating point of CCAs changes by varying the network topology and path parameters extends the operating point evaluation. In this section, we exclude experiments with time-variable network parameters; the time-variable parameters are covered in Section XI. More specifically, this section examines the operating point for a range of values of the path parameters, whereby the considered path parameter values are held fixed for a long time (i.e., do not vary with time). For example, this section includes the evaluation of the throughput as a function of the packet error probability ϵ to examine the loss resilience of a CCA. In particular, for the duration of one experiment run, the packet error probability ϵ is fixed and constant, but experiment runs are conducted for a range of error rate values, i.e., the ϵ values are swept.

A. Scalability

High-speed networks are characterized by very high data rates. With modern fiber technologies, data rates of 1 Gbits/s or beyond are common. Some CCAs struggle to saturate a path with a large capacity C. Besides high speeds, a large two-way propagation delay $\texttt{RTT}^{\text{prop}}$ may also lead to underutilization. Long Fat Networks (LFNs), that have a large capacity and a large two-way propagation delay, can be especially challenging for CCAs if random losses are present. A scalability experiment tests under which conditions a CCA can saturate a path

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

with a large BDP—a trait that is also referred to as scalability. The bandwidth utilization of a scalable CCA should be equally high irrespective of the BDP when all other path parameters are kept constant.

1) Scenarios: The scenario of a scalability experiment sets up a network with a large BDP. The network may use a large bandwidth (fat network), a long RTT (long network), or both (Long Fat Network) to have a large BDP. The experiment may be repeated for different BDPs to measure the connection between the BDP and the throughput of a CCA. Typically, only a single flow using the CCA under evaluation is set up. Loss-based CCAs may have scaling problems in networks with a nonzero packet error probability. Showing the connection between packet error probabilities and the throughput has been the primary method to attest that a CCA is scalable in the past. A CCA would be deemed scalable if it achieves a high throughput despite experiencing random losses. However, today many CCAs are not loss-based CCAs and are designed to withstand random losses to some degree. Hence, other approaches may be more adequate to show that such CCAs are able to acquire resources quickly. In [134], a step increase of the bandwidth is used to measure the ability of a CCA to acquire spare bandwidth. The scenario matches one of the responsiveness scenarios that we covered in Fig. 7.

2) Metrics: The main metric to measure the scalability of a CCA is the throughput or any related metric that quantifies the bandwidth allocation. A scalable CCA achieves a high throughput irrespective of the BDP. Furthermore, for loss-based CCAs the response function (allocation as a function of the packet loss ratio) may be modeled or measured in experiments. The aggressiveness metric of Section VI-B9 measures how quickly a CCA grows its allocation over time. A scalable CCA grows its allocation quickly to avoid long durations of underutilization.

3) Theoretical Modeling: The theoretical scalability of a CCA can be derived from the modeling of its mean throughput. For loss-based CCAs the response function, which we covered in Section VIII-A3b, expresses the expected sending window as a function of the loss event rate. Reno's response function is given by Eqn. (8). To saturate a path, the expected window needs to be equal to or larger than the BDP. For large BDPs Reno has scaling problems because the required loss event rate is unrealistically low [163]. It can take hours for Reno to recover from a single random loss event with its AIMD algorithm during CA. The reason is that AIMD is not aggressive in acquiring spare bandwidth—it grows its window by one segment each RTT as measured by the aggressiveness metric. This gives rise to a scaling problem for large-BDPs networks that can have thousands of segments in transit.

A set of loss-based CCAs have been proposed to mitigate the scaling problem of Reno, namely HighSpeed TCP (HSTCP) [163], Scalable TCP (STCP) [166], HTCP [164], FAST [135], Binary Increase Congestion Control (BIC) [165], and Cubic [76]. These scalable CCAs implement a more aggressive window growth than Reno during CA to recover more quickly after losses. Furthermore, these CCAs reduce their window less severely in comparison to Reno. The theoretical scalability of delay-based or hybrid CCAs can also be derived from the respective throughput models, e.g., [168] models the dependence of Vegas' throughput on the two-way propagation delay.

4) Experiments: The scalability of CCAs has also been evaluated in experiments. For loss-based CCAs, the packet error probability ε of a path can be swept to measure the response function $f(\varepsilon)$. In [139], [159], theoretical and empirical response functions are compared. Similarly, [148] sweeps the packet error probability and shows boxplots of the utilization. In [182], [221], the packet error probability is swept in a scenario where multiple flows compete and it is investigated how the number of flows affects the aggregated utilization. In [222], the two-way propagation delay is swept to measure the dependence of the throughput on the BDP. Experiments on real paths with different BDPs are conducted in [223]. In [97], three different experiments evaluate how different slow start algorithms affect the scalability by sweeping the packet error probability, two-way propagation delay, and capacity.

5) Experiments From ccperf: The *Scalability* experiment of Table V sets up a single flow and a network with the default path parameters of Table IV; in particular, a capacity C_0 and a two-way propagation delay $\mathtt{RTT}_0^{\mathrm{prop}}$ such that $\mathtt{BDP}_0 = C_0 \times \mathtt{RTT}_0^{\mathrm{prop}}$. At $t = T/4$, with T being the experiment duration, the capacity is increased to $C_s \geq C_0$ and the two-way propagation delay is increased to $\mathtt{RTT}_s^{\mathrm{prop}} \geq \mathtt{RTT}_0^{\mathrm{prop}}$ such that $\mathtt{BDP}_s = C_s \times \mathtt{RTT}_s^{\mathrm{prop}} > \mathtt{BDP}_0$. The experiment parameters are the capacity C_s, and the two-way propagation delay $\mathtt{RTT}_s^{\mathrm{prop}}$. The experiment evaluates if a CCA can quickly utilize the created spare bandwidth to avoid long durations of underutilization.

B. Loss Resilience

Random losses may degrade the allocation of CCAs, and this degradation commonly leads to bandwidth underutilization. As explained in Section X-A, underutilization due to random losses is especially prevalent on paths with large BDPs—a problem referred to as scalability. The key issue of affected CCAs is that they cannot withstand random losses. In contrast, some other CCAs are designed to be resilient towards random losses.

1) Scenario: The scenario to evaluate the loss resilience of a CCA typically sets up a single flow in a network with a single path. The path is set up with an error model that has a positive packet error probability. Typically, a constant packet error probability is used that does not change over time. Experiments may be repeated with different packet error probabilities.

2) Metrics: The throughput, or a related allocation metric, is the main metric to quantify the loss resilience. A loss-resilient CCA achieves a high throughput irrespective of the probability of random losses. The response function (mean throughput as a function of a loss probability) can either be modeled or measured in experiments. However, the response function gives only a partial picture, because the mean allocation is averaged over an entire evaluation. We may refer to this type of loss resilience as long-term loss resilience.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

In contrast, short-term loss resilience evaluates the degradation of the allocation on short time scales. A CCA that backs off after each loss event is not short-term loss-resilient, but may still be long-term loss-resilient if the allocation recovers from the reduction quickly. The *robustness to non-congestion loss* metric of Section VI-B5 quantifies short-term loss resilience. Experiments typically evaluate the short-term loss-resilience by plotting the allocation over time.

3) Theoretical Modeling: Many CCAs, e.g., BBR [77] or Performance-oriented Congestion Control (PCC) [138], are by design short-term and long-term loss-resilient as long as the frequency of loss events does not exceed a threshold. These CCAs do not back off after loss events. Furthermore, the long-term loss resilience of many CCAs can be modeled with the respective response function, which we covered in Sections VIII and X-A.

4) Experiments: The typical way to evaluate long-term loss resilience experimentally is to measure the empirical response function. For that, a series of experiments with varying packet error probabilities is conducted. As a result, the mean throughput as a function of the packet error probability is plotted [76], [77], [138], [139], [149], [165], [167], [170], [182], [220].

The short-term loss resilience of a CCA is typically evaluated by plotting the time series of an allocation variable from a single experiment.

5) Experiments From ccperf: The *Loss Resilience* experiment of Table V evaluates a CCA on the default path of Table IV for a specifiable packet error probability. Fig. 8 shows the mean utilization (bandwidth) of Reno, Cubic, Vegas, and BBRv1 from benchmarks that repeated the experiments for packet error probabilities ranging from 0.001 % to 5 %. As expected, the utilization of the loss-based Reno and Cubic decreases as the packet error probabilities grow as the CCAs falsely interpret the random losses as congestion. The results are in line with the analytical modeling of the response function in Eqn. (8). The delay-based Vegas decreases cwnd less strongly in response to a loss event and shows some resilience to random losses. Reno, Cubic, and Vegas use the PRR fast recovery algorithm during Loss Recovery to adjust cwnd. In contrast, BBRv1 does not use PRR and continues to use its Congestion Avoidance algorithm. By design, BBRv1 does not interpret loss events as a signal of congestion and does not back off. Our experiments confirm the loss resilience of BBRv1. However, as the packet error probability exceeds 0.02 % in our experiments, BBRv1's utilization degrades due to RTOs. More advanced LR algorithms that prevent RTOs could increase the mean utilization of all featured CCAs.

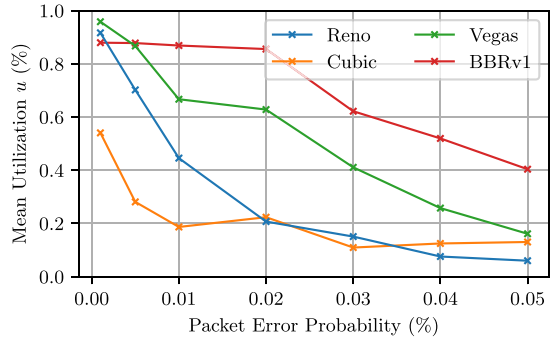


Fig. 8. Mean (bandwidth) utilization of Reno, Cubic, Vegas, and BBRv1 for different packet error probabilities.

C. Deep Buffers—Bufferbloat

Buffer-filling CCAs are designed to continuously increase their number of packets in flight over time during CA. With FIFO tail-drop queueing, the operating point of a buffer filler steadily approaches the collapse region of Fig. 4 over time. Hence, buffer fillers inflict backlogs and queueing delays that are proportional to the buffer size, which is the so-called bufferbloat problem [109], [224] of deeply-buffered paths. The main symptom of bufferbloat is increased delay, which is problematic for the feedback loop of CCAs, and for applications requiring low latency [225], [226], [227], [228], [229].

To counteract bufferbloat, CCAs should target an efficient operating point that avoids self-inflicted queueing delay irrespective of the buffer size, which is achieved by many delay-based and hybrid CCAs. Furthermore, AQM algorithms, which we cover in Section X-E, can assist sender-side CCAs by providing congestion signals earlier than tail-drop queues. The objective of a bufferbloat experiment is to examine how the operating point of a CCA changes as the bottleneck queue becomes deep ($B \gg \texttt{BDP}$).

1) Scenario: A bufferbloat scenario sets up a deep buffer at the bottleneck. Typically, a static scenario with a single or multiple flows and a bottleneck queue size that is larger than the BDP are used. Experiments are often repeated with different queue sizes.

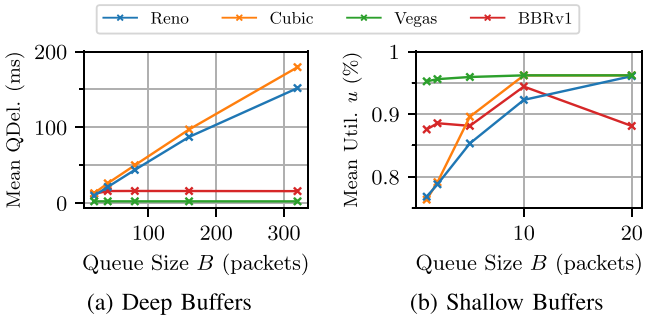
2) Metrics: The main metrics to measure if a CCA is affected by bufferbloat are the queue length (number of packets in queue) and the queueing delay. The packets in flight Ω (or cwnd if applicable) may be examined to show that a CCA operates with $\Omega > \texttt{BDP}$.

3) Theoretical Modeling: Based on the steady-state modeling, which we covered in Section VIII, the operating point of some CCAs can be expressed as a function of the queue size B [14], [81], [194]. For example, in a scenario without random losses and a FIFO queue, Reno's expected mean window size is proportional to the queue length with $\mathbb{E}\{w\} \approx 3/4(\texttt{BDP} + B)$.

4) Experiments: Experiments that are run a single time may show the time series of the operating point [109], [194] or its distribution [230]. These experiments are used as examples to explain how a CCA is affected by bufferbloat. Alternatively, the queue size B may be swept in multiple experiments to approximate the expected operating point as a function of the queue length B [77], [137], [138], [149], [231], [232].

5) Experiments From ccperf: The *Deep Buffers* experiment of Table V evaluates if a CCA is affected by bufferbloat. For that, the experiment sets up a network using the default parameters of Table IV with a specifiable bottleneck queue size. The queue size and the number of competing flows K are experiment parameters. Fig. 9(a) shows the mean queueing

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.



(a) Deep Buffers (b) Shallow Buffers

Fig. 9. Mean queuing delay (a) and mean utilization (b) for different queue sizes from the *Deep Buffers* and *Shallow Buffers* experiments.

delay from experiments with a single flow $K = 1$ that varied the bottleneck queue sizes ranging from 20 packets $(1 \times \texttt{BDP})$ to 320 packets $(16 \times \texttt{BDP})$. The queueing delay is estimated with $\texttt{sRTT} - \texttt{RTT}^{\text{prop}}$. The experiments confirm that Reno and Cubic are buffer fillers that inflict a queueing delay that is proportional to the bottleneck queue size. Cubic inflicts the highest queueing delays, as we have already witnessed in Fig. 5. In contrast, Vegas and BBRv1 are not buffer fillers and are designed to avoid self-inflicted queueing delays. The mean sRTTs of both CCAs remain the same for all queue sizes. Thereby, Vegas and BBRv1 are immune to the bufferbloat problem in the experiments.

D. Shallow Buffers—Bufferless

Paths with shallow-buffered $(B \ll \texttt{BDP})$ bottleneck queues are a challenge for CCAs. Shallow buffers are limited in their ability to absorb traffic bursts. As a consequence, queue overflows that produce congestion losses are more likely. These congestion losses trigger flows to back off even though the queues drain automatically after the traffic burst ends. This backing off can cause underutilization [152]. Furthermore, many loss-based CCAs require—by design—a sufficiently large buffer to reach full utilization. The operating point of such a loss-based CCA depends on the buffer size and the CCA's efficiency decreases as B falls below a threshold. For example, a single Reno flow needs a buffer at least the size of the BDP for full utilization [152].

1) Scenario: The evaluation typically uses a static scenario with a small bottleneck queue size that is less than the BDP. One or more flows are set up in the scenario.

2) Metrics: A CCA may underutilize the bandwidth resources in a shallow-buffered network. Furthermore, the number of lost packets may be high. Hence, the main metrics are the throughput (or aggregated throughput of all flows) and the number of lost packets or retransmissions.

3) Theoretical Modeling: For some CCAs the influence of the buffer size on the operating point can be modeled. In [164], [168], [233], the expected throughput of different CCAs in a shallow-buffered scenario is modeled explicitly and distinguished from scenarios with larger buffer sizes. Similarly, [234], [235], [236] use fluid models to model the aggregated transmission rate and bottleneck queue backlog over time in a shallow-buffered scenario.

4) Experiments: Experiments visualize the impact of the buffer size B on the throughput either with a sweep plot [97], [115], [138], [139], [149], [167], [215], [232] or show tables with different values for B [179], [214], [237]. These experiments verify that loss-based CCAs exhibit reduced throughput and an increased number of retransmissions with small buffers, whereas delay-based or hybrid CCAs are less affected. However, CCAs that do not necessarily back off after a loss are at risk of ignoring legitimate congestion losses. In [231] and [194] it is shown that BBRv1 fails to converge to its targeted operating point near the BDP on shallow-buffered paths where $B < \texttt{BDP}$. Instead, [231] and [194] have found that BBRv1 operates with a number of packets in flight beyond $\texttt{BDP} + B$ in the collapse region of Fig. 4 in shallow-buffered paths. This shortcoming of BBRv1 has been addressed by its successor BBRv2 [238] and by other BBR variants [239].

5) Experiments From ccperf: The *Shallow Buffers* experiment of Table V evaluates if a CCA can handle networks with shallow buffers that are smaller than the BDP. The experiment sets up a network with the default path parameters of Table IV with a specifiable bottleneck queue size. The queue size and the number of competing flows K are experiment parameters. Fig. 9(b) shows a sweep plot of the mean bandwidth utilization u from experiments with $K = 1$ that varied the bottleneck queue sizes ranging from 1 packet to 20 packets $(1 \times \texttt{BDP})$. As expected, Reno and Cubic underutilize the capacity of the path for small queue sizes. Reno is more affected by shallow buffers because its response to a congestion drop is more severe—Reno halves the cwnd after a loss event compared to Cubic's reduction by 30 %. Reno requires $B = \texttt{BDP}$ to fully utilize the capacity [152]. The utilization of Vegas and BBRv1 is not reduced due to shallow buffers. However, with small queue sizes, BBRv1 causes frequent congestion drops and operates with a number of packets in flight slightly above $\texttt{BDP} + B$. For example, with $B = 1$ packet, BBRv1 causes 1495 retransmissions in total. In each experiment, except with $B = 10$ packets, BBRv1 experiences an RTO after slow start. Consequently, with $B = 10$ packets, BBRv1's mean utilization is slightly higher than for the other B values. We conjecture that this inefficient BBRv1 behavior is due to discrepancies of the ns-3 BBRv1 implementation from the original BBRv1 design [77]. Ordinarily, we would expect that BBRv1 does not incur RTOs and we would expect the mean utilization curve to be flat (similar to the Vegas curve). As noted in Section VII-H, we leave detailed investigations into these apparent discrepancies for future research.

E. Active Queue Management (AQM)

AQM algorithms are packet schedulers that define policies for enqueueing and dequeueing packets by a queue. Functionally, AQM algorithms provide alternatives to the classical FIFO tail-drop policy. In Linux, AQM algorithms are implemented as qdiscs in the traffic control layer. AQM facilitates sender-side CCAs by dropping or marking packets with ECN in a proactive manner before queues overflow. The objective is to control the queue length at the bottleneck where congestion takes place. Hence, AQM tackles the bufferbloat

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

problem from within the network. AQM can be very effective in Local Area Networks (LANs) or data center networks. On the other hand, AQM algorithms may run into scaling problems in the Internet [240].

RED [85] was the first widely used AQM policy at routers or switches. RED randomly drops packets with a probability that grows with the queue length. In comparison to FIFO tail-drop qdiscs, RED reduces the expected queue length and discourages flow synchronization of loss-based CCAs. CoDel [230] controls the queueing delay directly by dropping packets when the queueing delay is above a threshold for a prolonged period. Combined with Fair Queuing (FQ) [241], the FQ-CoDel [242] qdisc serves competing flows with a round-robin scheme while controlling the queueing delay. Hence, the FQ-CoDel qdisc also implements a flow-level fairness mechanism. FQ-CoDel is the default qdisc in the Linux operating system today. An overview of the variety of AQM algorithms is given in [86]. AQM expects responsive senders to back off after the AQM communicates congestion by marking or dropping packets. Hence, it should be evaluated how a sender-side CCA interacts with and benefits from AQM. The handling of unresponsive flows (e.g., UDP flows) by the AQM algorithm is another topic for evaluations.

Furthermore, qdiscs may provide priority queueing by classifying packets into different traffic classes. In contrast to FIFO queues, priority queueing dequeues packets in an order that depends on the assigned traffic class. Differentiated Services (DiffServ) [243] and similar mechanisms rely on such priority-aware packet schedulers to provide their service.

1) Scenario: Experiments that evaluate CCAs in conjunction with AQM algorithms typically use a static scenario. The bottleneck queue is set up to use a qdisc that is evaluated. One or more flows that use CCAs are set up, typically in an intra-protocol competition scenario. UDP flows may be set up in addition to that.

2) Metrics: The operating point of CCAs may change if AQM algorithms are used. Throughput, delay, and fairness are the most common metrics used in evaluations, but all metrics of Table III may be used.

3) Theoretical Modeling: Theoretical modeling of CCAs in scenarios with AQM is less common, likely because of a lack of generalization and intractable complexity. Each pair of a CCA and an AQM algorithm would require its own modeling because the two algorithms interact in a unique way. Furthermore, many AQM algorithms are stateful and implement rules that are hard to model analytically. A counterexample to that is the theoretical modeling of Reno in a scenario with a RED qdisc [81], [85].

4) Experiments: Typical experiments let competing flows generate bulk or Web traffic in a topology with a single bottleneck that deploys a qdisc with an AQM algorithm [85], [230], [244], [245], [246], [247]. Then, typical experiments monitor the queue length, queueing delay, and loss events at the bottleneck qdisc and the throughput and latencies of the flows. The metrics are either shown as time series or as distributions. In [232], the operating points (throughput and delay pairs) of different combinations of CC and AQM algorithms are shown in the throughput-delay plane.

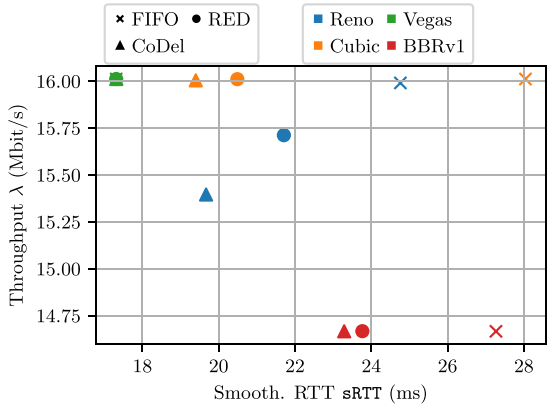


Fig. 10. The operating point plane for Reno (blue), Cubic (orange), Vegas (green), and BBRv1 (red) evaluated on the default path of Table IV with a FIFO (cross), RED (circle), and CoDel (triangle) bottleneck Queueing Discipline (qdisc), respectively.

Studies [230] and [245] also run experiments with dynamic bottleneck capacities that change abruptly every few seconds. Then, they evaluate how quickly the CCAs respond to the sudden congestion caused by a decrease of the bottleneck capacity. We cover such responsiveness experiments in Section IX-D in more detail. The AQM algorithms CHOose and Kill for unresponsive flows (CHOKe) [248], and Stochastic Fair BLUE (SFB) [249] are designed to enforce fairness between responsive and unresponsive flows. Therefore, these studies evaluate how the AQM algorithms limit the resource allocation of unresponsive UDP flows.

Common Applications Kept Enhanced (CAKE) implements priority queueing and a fair queueing between hosts instead of flows [250]. Instead of isolating each flow in its own queue (as done by FQ-CoDel), CAKE can isolate multiple flows of the same host (sender or receiver) in their respective queue. The evaluation in [250] includes a scenario that compares the throughput of flows depending on the different isolation types supported by CAKE. Furthermore, a scenario with priority queueing is covered that shows how with CAKE time-sensitive traffic (high priority) achieves low latency when competing against bulk transfers (low priority, best effort).

5) Experiments From ccperf: The *AQM* experiment of Table V sets up a network with the default path parameters of Table IV with K flows and a specifiable qdisc at the bottleneck. Fig. 10 shows the throughput-delay plane of experiments featuring Reno, Cubic, Vegas, and BBRv1 and different qdiscs. Each experiment evaluates a CCA operating in the default scenario with either a CoDel, RED, or FIFO qdisc. The default qdisc parameters are used and ECN is disabled. The throughput-delay plane shows the mean throughput and mean sRTT. With a FIFO qdisc (crosses), the experiment replicates the steady-state experiments of Fig. 5. The buffer fillers Reno and Cubic generate standing queues that inflict large queueing delays and inflate the sRTT. The ns-3 implementation of BBRv1 also inflicts a large queueing delay as it fails to accurately estimate the bottleneck capacity. In contrast to Reno and Cubic, however, BBRv1 should avoid the queueing delay by design. Only Vegas succeeds in avoiding filling up the buffer and operates with a mean sRTT that is

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

close to the two-way propagation delays. Since Vegas already avoids queueing delays with a FIFO qdisc, the RED and CoDel qdiscs do not change Vegas' operating point. For Reno, Cubic, and BBRv1, the RED and CoDel qdiscs reduce the sRTT by dropping packets earlier than a FIFO qdisc. Reno's throughput drops below the bottleneck capacity, especially for CoDel, because CoDel signals congestion earlier than RED. The reason for the underutilization is that Reno halves its cwnd after a loss event. With a FIFO qdisc, Reno halves its cwnd approximately when $\texttt{cwnd} = 2 \times \texttt{BDP}$. With a RED or CoDel qdisc, Reno reduces cwnd earlier to a value that is less than the BDP. Cubic's response to a congestion event is less strong with a cwnd reduction of 30 %. As a consequence, Cubic's mean throughput remains approximately equal to the bottleneck capacity, while its queueing delay is reduced. Likewise, BBRv1's throughput is not changed by the choice of the qdisc. The experiments demonstrate how AQM can mitigate bufferbloat problems. As shown, such in-network assistance from AQM algorithms is especially valuable for buffer filling CCAs.

F. ACK Compression and Aggregation

The pacing of ACK arrivals at the sender ideally matches the pacing of delivered packets at the receiver as explained in Section II-E. Then, the ACK arrival rate matches the bottleneck rate of the forward path, which is the ideal case for a cwnd-limited sender that is ACK-clocked. A phenomenon called ACK compression changes the pacing of ACKs on the reverse path in a way that the ACK arrival rate no longer matches the bottleneck rate of the forward path. Hence, ACK compression negatively interferes with ACK-clocked packet transmissions.

ACK compression causes the progress in the ACK sequence number space to be discontinuous with jumps. Consider a cwnd-limited sender that does not use rate pacing. During some time periods, the sender may not receive any ACKs. Then, the cwnd stalls the sending window and the sender does not send out any new data. At other times, a burst of ACKs may arrive at the sender that rapidly advances the ACK sequence number space almost instantaneously. As a consequence, the sender also advances its sending window by many segments at once and transmits a burst of packets. Such packet bursts may lead to congestion loss events followed by window reductions and underutilization. In extreme cases, flows with ACK compression can be starved entirely by competitors [251]. Furthermore, senders may use ACKs to estimate path parameters, such as $\texttt{RTT}^{\text{prop}}$ and the bottleneck rate. The unsteady arrival of ACKs complicates such path parameter estimations [92]. Erroneous estimations can make CCAs operate at undesirable operating points. For example, the bottleneck rate estimation of Westwood [116] overestimates the true bottleneck rate when ACK compression is present—an issue that was tackled in Westwood+ [252]. Similarly, BBR uses techniques to detect and filter ACK compression to improve estimates of path parameters [77].

1) Scenario: There are several different scenarios that produce ACK compression. First, some access networks, e.g., cable, WiFi, and cellular networks, use Medium Access Control (MAC) schemes that may delay the transmission of ACKs. The ACKs are held back in a transmission queue until medium access is granted. These networks often aggregate ACKs and transmit a single cumulative ACK that acknowledges multiple segments at once.

Second, TCP/IP stacks may implement batch processing algorithms such as TCP Segmentation Offload (TSO) and Large Receive Offload (LRO) as explained in Section A-B3. These algorithms may process and release a burst of packets at once. Hence, the algorithms may aggregate multiple segments or ACKs in a single burst and thereby erase the spacing between the ACKs.

Third, traffic may congest a link on the reverse path. To set this scenario up, one or more flows have to saturate the bandwidth resources on the reverse path. As a consequence, ACKs are queued up at that link before they are forwarded with a rate equal to that link's capacity. Therefore, the original pacing of the ACKs that matched the bottleneck rate of the forward path is lost during the forwarding of the ACKs on the reverse path. Additionally, the pacing is further altered because the ACKs are interleaved with other traffic waiting in the queue [253], [254]. Bursty traffic peaks may congest the reverse path only shortly causing temporary ACK compression that automatically dissipates.

Fourth, on highly asymmetric paths, the bottleneck capacity of the reverse path may be saturated by the ACK traffic [255], [256]. The scenario can be set up by choosing a relatively small bottleneck capacity on the reverse path. The small capacity leads to a bottleneck on the reverse path for the connection (in addition to the bottleneck on the forward path). The pacing of the forwarded ACKs is stretched out and ACKs may get dropped at the bottleneck queue of the reverse path [253]. The combination of an asymmetric path with a slow bottleneck link on the reverse path and (bursty) reverse traffic has been observed to cause ACK compression in real networks and in simulations. There are several network technologies, such as Asymmetric Digital Subscriber Line (ADSL), cable networks, and satellite networks, that are asymmetric and where ACK compression has been observed [256].

An ACK compression scenario can also be created by artificially delaying ACKs on the reverse path. This approach is implemented by the scheduled links of the Mahimahi emulator [257]. The scheduled link can be configured to transmit some number of ACKs at once in periodic intervals. The link resembles the behavior of a medium access control scheme.

2) Metrics: The main symptom of ACK compression is a discontinuous progress of the highest unacknowledged sequence number at the sender. The discontinuous progress may lead to low throughput (underutilization), especially for cwnd-limited senders. Hence, the throughput (or bandwidth utilization) is the main metric to measure a CCA's performance in an ACK compression scenario.

3) Theoretical Modeling: The window size of Reno in scenarios with asymmetric bandwidths of the forward and reverse paths is modeled in [255]. Besides that, ACK compression has not been modeled theoretically to the best of our knowledge.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

4) Experiments: Typically, experiments examine whether a CCA can maintain high throughput when ACK compression is present. Experiments with congested reverse paths are conducted in [59], [85], [211], [253], [258], [259], [260]. The bandwidth asymmetry in terms of forward and reverse path capacities is varied in [255]. The study expresses the throughput of Reno as a function of the bandwidth asymmetry and validates the theoretical modeling with simulations. Pantheon [11] conducts experiments with a scheduled link that purposefully generates ACK compression on the reverse path. Different ACK compression intervals are evaluated.

5) Experiments From ccperf: The *ACK Compression* experiment of Table V sets up a network with the default parameters of Table IV for $K = 1$ and a bottleneck qdisc on the reverse path that purposefully delays ACKs to produce ACK compression. The qdisc releases a quota of ACKs in one burst in periodic intervals. The quota and interval are experiment parameters.

G. Traffic Policing

Traffic policing is common in today's Internet. Internet Service Providers (ISPs) use traffic policing to enforce data plans of their customers [105], [261]. Rate limiters, such as the Token Bucket Filter (TBF), are used to limit individual data rates at access networks. A TBF drops packets if the traffic continuously exceeds some configured policing rate. Packet bursts that only temporarily exceed the policing rate are not policed.

1) Scenario: The scenario sets up a qdisc that implements traffic policing, e.g., a TBF, to limit the bandwidth of flows. The policing rate may be varied in experiments.

2) Metrics: Traffic policing may lead to bursty packet loss, retransmissions, RTOs, and long LR phases. As a consequence, CCAs may underutilize bandwidth resources and may struggle to converge to a transmission rate that matches the policing rate. Throughput, loss, and flow completion times are the main metrics to examine the performance of CCAs in this scenario.

3) Theoretical Modeling: To the best of our knowledge, the impact of traffic policing on CCAs has not been modeled.

4) Experiments: A series of experiments has been conducted in [198] to study the challenges of traffic policing scenarios. CCAs may overshoot the policing rate by a lot and experience massive packet loss as a consequence. Multiple rate reduction steps may be necessary to correct the overshooting. In the meantime, even retransmitted packets may be dropped and the policing may result in an RTO. Furthermore, in contrast to packet loss caused by congestion at a queue, packet drops due to traffic policing are not preceded by growing queueing delays. Therefore, CCAs do not observe an RTT increase as a congestion signal that indicates that the transmission rate exceeds the policing rate. Decreased throughput in traffic policing scenarios has also been observed in [262], [263].

The Pantheon framework [11] includes a scenario that uses a TBF for rate policing. The policing rate of the TBF is an experiment parameter. The Pantheon framework compares the mean operating points of the evaluated CCAs in the throughput-delay plane. The results indicate that many CCAs significantly underutilize the policing rate.

From the perspective of a CCA, a TBF is similar to a sudden change of the path capacity. At first, a CCA can transmit data with a transmission rate above the policing rate. Once the TBF starts enforcing its rate limit, the path capacity suddenly drops to the policing rate. CCAs that estimate the path capacity, e.g., BBRv1 with its estimation algorithms specified in [92], may overestimate the path capacity after the TBF starts enforcing the policing rate. To mitigate these estimation issues, BBRv1 implements a policer detection and policer model [77].

5) Experiments From ccperf: The *Traffic Policing* experiment of Table V sets up a TBF qdisc at the bottleneck. Two experiment parameters configure the TBF: the policing rate sets the target rate for the rate limiting and the peak rate sets the unlimited rate.

H. Data Center

DC networks are large-scale LANs. Similar to WANs, DC networks connect thousands of nodes in large topologies, but with the key difference that a DC network is a controlled environment often owned by a single organization. Furthermore, DC networks use links (typically fiber) with high bandwidths (typically more than 10 Gbits/s) and low RTTs (significantly less than 250 μs). The controlled environment of DC networks enables vendors to run CCAs that are specifically tailored towards DC networks. Following the classification in [264], we differentiate the following CCA types for DC networks:

- ECN-based CCAs rely on switches to set ECN marks to notify hosts about congestion.
- Explicit feedback CCAs let switches provide feedback to hosts that exceed the binary signals of ECN marks.
- Credit-based CCAs require the receiver to issue credit packets to senders.
- Packet scheduling CCAs use Quality of Service (QoS) classes to implement priority queueing at switches.
- Delay-based CCAs rely on accurate latency measurements as a congestion signal.

The unique characteristics of DCs significantly impact performance. Both bandwidth and RTTs deviate several orders of magnitude from the non-DC world (WANs or LANs). The extended control over hardware, software, and their interplay enables DC operators to exploit these characteristics [265], [266], [267]. The large operational scales of DCs make it feasible and profitable for DC operators to tune their TCP/IP stack beyond the point of diminishing returns for ISPs. The literature on CCA evaluation for DCs is very extensive. Due to space constraints, we limit this DC section to the evaluation scenarios and metrics that are most commonly used in studies on DC CC.

1) Scenario: The evaluation scenarios in DC networks are diverse and depend on the capabilities of the deployed switches, routers, and hosts. Each CCA type for the DC has its own set of hardware and software requirements to be deployable. The scenarios covered in this section provide an overview of the different topologies, hardware, and software

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

components that are commonly used to evaluate CCAs in DC networks. These components are not mutually exclusive, e.g., a scenario may evaluate CCAs in a certain topology in combination with switches that provide advanced features.

a) Topologies: Many DC scenarios use the dumbbell network for single-bottleneck scenarios. A single-bottleneck scenario may be sufficient for certain evaluations. However, real DC networks have different topologies that are also used in CCA evaluations. The topologies of DC networks often form regular patterns that arrange nodes in interconnected clusters. Hosts are typically located in server racks that use a Top of Rack (ToR) switch to connect to the rest of the DC network. A ToR switch aggregates the ingress and egress traffic of the hosts in a rack. The ToR switch is a typical point of congestion [264], [268], [269].

A common topology of DC networks are Clos networks, in particular the special case of a fat tree topology [270], [271]. The fat tree topology arranges nodes in a hierarchical tree structure. Nodes that are near the top, are connected by links with large bandwidths, whereas nodes at the bottom have lower bandwidths. The nodes at the top are typically switches or routers where traffic is aggregated, whereas the leaves of the tree are hosts.

b) Shallow-buffered switches: In DC networks it is common to use shallow buffers, especially at ToR switches [268], [269]. As covered in Section X-D, shallow buffers can be challenging for CCAs. In DC networks, where traffic can be bursty, shallow buffers may overflow quickly.

c) Traffic models in DCs: The traffic patterns present in DC networks are different from traffic patterns of other networks. A variety of distributed applications are running that access services located on other hosts, for example storing and retrieving data from databases, calling remote Application Programming Interfaces (APIs), exchanging periodic control messages, and executing Remote Procedure Calls (RPCs). Two traffic types may be differentiated: query and background traffic [268]. Overall, more than 99 % of traffic uses TCP.

Query traffic follows a request and response pattern. Flows carrying query traffic are typically small and latency-sensitive. Applications that generate query traffic either access databases, query search engines, or access remote APIs. A single application may issue many queries in parallel, which leads to the so-called incast problem that we elaborate in Section X-H1e.

Background traffic originates from many different application types for tasks, such as copying, replicating, or updating data, as well as control messages. Flow sizes vary depending on the types of applications. Control messages are typically exchanged with small, latency-sensitive flows. Flows that copy data are bulk transfers with large flow sizes.

The number of concurrent flows in DC networks can be high, especially in incast scenarios explained in Section X-H1e. However, the number of concurrent flows with large flow sizes (above 1 MB) is surprisingly small. Empirical distributions of flow sizes and inter-arrival times of flows have been reported in [268] for the different traffic types. Synthetic DC traffic can be generated by setting up flows, whose inter-arrival times and flow sizes are sampled from empirical distributions [268], [272], [273], [274].

d) WAN traffic: Flows that originate and terminate inside the DC network can use CCAs designed for the DC. External traffic, which originates in the WAN outside of the DC network, likely uses other CCAs [264], [268], [275]. Furthermore, the WAN traffic changes more slowly compared to DC traffic and the RTTs of WAN flows are much longer. For example, a DC that uses the ECN-based Data Center TCP (DCTCP) may have to handle external flows using Cubic that do not support ECN. Such a scenario that sets up mixed WAN and DC flows may be evaluated.

e) Incast problem: A problem that is specific to DC networks is the so-called *incast* problem [276], [277], [278]. The incast problem arises when a large number of flows simultaneously arrive at a switch. Often, these flows share the same receiver, because the receiver itself launched a distributed computing task. In such a task, the computation is offloaded to parallel worker nodes (partitioning). These worker nodes then finish their computations almost simultaneously and transmit the results of the task back to the receiver (aggregation). As a consequence, many small short-lived flows containing these results arrive almost simultaneously at a switch near the receiver. The switch may overflow due to the burst of traffic, which can lead to costly timeouts that dramatically increase flow completion times. Hence, the incast problem is a consequence of common distributed computing workloads and traffic patterns that are specific to DC networks.

An incast scenario uses a static network and Web traffic with request-response cycles. Two incast scenarios can be differentiated. A many-to-one incast scenario sets up a single client that requests data from n servers. The n servers transmit their response almost or exactly at the same time. In contrast, a many-to-many incast scenario sets up n servers and m clients in a fat tree topology. The m clients request data from the n servers simultaneously, and the responses are transmitted almost or exactly at the same time (typically, $n = m$). The number of servers, clients, and flows may be varied in experiments. Typically, the incast problem becomes more severe with a larger number of simultaneous transmissions.

f) On-off traffic patterns: In an on-off traffic scenario, which we also cover in Section XII-A, flows start and stop every few seconds. Such on-off traffic pattern scenarios are typically used to show how quickly CCAs converge [264], [268].

g) Link layer CCAs: DC networks may use link layer protocols that are designed to provide a lossless and reliable transport at the link layer. For example, data center bridging (also known as lossless Ethernet) is a set of protocols that enhance Ethernet. In particular, Institute of Electrical and Electronic Engineers (IEEE) 802.1Qau [279] and IEEE 802.1Qbb [280] are used by the link layer CCA named Quantized Congestion Notification (QCN) [281]. Scenarios with link layer CCAs require switches and hosts to support the link layer protocols. Link layer CCAs are combined with simple transport layer CCAs because it can be assumed that congestion is avoided and losses are rare at the transport layer.

h) Hardware timestamps: The propagation delays in DC networks can be as low as 1 μs per hop. Delay-based CCAs for DC networks require timestamps to be precise on

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

a microsecond scale. OSs may fail to provide timestamps of this precision. TIMELY has shown that modern NICs provide hardware timestamps that are precise enough to make delay-based CCAs feasible for DC networks [172]. Hence, delay-based CCAs require neither support from switches nor changes to conventional TCP/IP stacks, but to operate effectively they require hardware timestamping of NICs. Scenarios that evaluate delay-based CCAs should state whether or not NICs that provide hardware timestamps are used.

i) Offloading TCP/IP stacks to NICs: High link speeds and small propagation delays in DC networks require fast TCP/IP stacks. OSs may struggle to meet these requirements, as their TCP/IP stacks are implemented in software, which may be too slow for modern DC networks [282]. Too low processing speeds at hosts can lead to host congestion—congestion in the data path between the NIC, CPU, and memory at the host [275], [283]. To overcome these challenges, DC networks may use TCP/IP stacks that are implemented in NICs, leveraging high-speed data transfer technologies, such as Remote Direct Memory Access (RDMA). RDMA bypasses the OS kernel and thereby avoids the overhead of packet processing in software. The RDMA over Converged Ethernet (RoCE) implementation [284] uses QCN [281] as a link layer CCA. Scenarios to evaluate such OS-bypass technologies require testbeds with NICs that implement efficient TCP/IP stacks. Simulation models of the RDMA-based TCP/IP stacks are also available [285]. Other kernel-bypassing technologies, such as DPDK, have also been used to implement CCAs on NICs [286].

j) ECN: In contrast to the Internet, ECN (see Section II-D) is a practical and beneficial option in DC networks because administrators can ensure that all switches and routers support ECN marking. DCTCP [268] is a prominent CCA for the DC, which pioneered using ECN marking.

For ECN to work effectively, queues have to be configured with parameters that decide when a queue marks a packet with the CE codepoint. The original DCTCP used an algorithm that marked every arriving packet if the queue length exceeded a threshold. Other qdiscs, such as RED or CoDel, mark packets differently. For example, CoDel marks packets if the expected queueing delay is above a CE threshold parameter (in [s]) to control the queueing delay.

In an ECN scenario, all senders and queues are configured to use ECN. Queue parameters may require further configuration, e.g., the CE thresholds of CoDel queues have to be configured. The scenario may be repeated with different numbers of flows, queue parameters, or traffic models. The process of finding beneficial threshold parameters for queues in real DC networks is called ECN tuning. In heterogeneous DC networks with different types of switches and link speeds, ECN tuning can be challenging [264].

k) Explicit feedback: XCP [84], Rate Control Protocol (RCP) [287], and D^3 are CCAs that rely on explicit feedback from switches or routers, which communicate additional congestion signals to the hosts that is useful for rate control. In contrast to the qualitative binary signals of ECN, explicit feedback informs the sender about the congestion level in a quantitative way and possibly even contains instructions for CCAs how to update their rates. Similarly, QCN, Data Center Quantized Congestion Notification (DCQCN), and High Precision Congestion Control (HPCC) use explicit feedback at the link layer in conjunction with TCP/IP stacks offloaded to the NIC. Scenarios that evaluate CCAs based on explicit feedback require the hosts, switches, and routers to implement the respective feedback protocols, which may include the support of NICs at hosts.

l) Credit-based CCAs: pHost [273], ExpressPass [288], NDP [289], and Homa [290] are credit-based CCAs that require receiver-side modifications. The receiver of a flow issues credit packets to the sender to limit the sender's rate. A sender can transmit packets as long as the credits allow it. Switches may discard a credit packet issued by a receiver to limit the rate further. Scenarios that evaluate credit-based CCAs require the support of hosts to generate and consume credit packets, and possibly the support of switches to process credit packets.

m) Packet scheduling CCAs: Preemptive Distributed Quick (PDQ) [291], pFabric [292], Fastpass [293], QJump [294], and Karuna [274] are CCAs for the DC that use packet scheduling at queues. These CCAs use priority queueing at queues to provide different service levels to flows. Each flow is mapped to a priority level, which is considered by queues when scheduling transmissions. For example, packets belonging to latency-sensitive flows may always be scheduled first. Priority queueing is a viable approach for CCAs in DC networks, because of the controlled environment. Scenarios that evaluate such CCAs have to set up priority queueing algorithms at queues and flows with different priority levels.

2) Metrics: The most common metric for DC network scenarios is the flow completion time. The eCDF of flow completion times is often shown, alternatively the mean value or percentiles are reported. Many DC applications are sensitive towards tail latencies quantified by the 95th or the 99th percentile [294], [295]. Flow completion times are especially important for query and short message traffic.

CCAs for the DC are expected to operate closely to the optimal operating point of Fig. 4. Hence, throughput, fairness, inflicted queueing delay, and packet losses are evaluated. Furthermore, the stability (variation of the throughput or queue length) is evaluated as CCAs are expected to have an almost constant steady-state operating point with little fluctuations.

The consumption of system resources on hosts may also be evaluated [272], [294]. In particular, CPU and memory usage are important metrics. A single host may open thousands of simultaneous flows and thus the host requires a small consumption of system resources from CCAs. Since the RTTs in DC networks are so small, processing delays are interesting performance metrics at high speeds [272].

3) Theoretical Modeling: The steady-state behavior of CCAs for the DC can be modeled with the same methods described in Section VIII [170], [268], [272], [281], [281], [285], [287]. Shallow-buffered scenarios with DCTCP in DC networks are analyzed in [296]. Mostly fluid models are

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

used to prove that the CCAs reliably converge to efficient, fair, and delay-minimizing operating points. Other scenarios with dynamic traffic patterns are intractable to model and the evaluation is limited to experiments.

4) Experiments:

a) Steady-state experiments: Steady-state experiments of CCAs for DC networks for a single bottleneck can be found in [170], [171], [264], [268], [281] and for topologies with multiple bottlenecks or DC topologies in [268], [269], [272], [287], [291]. Often, a CCA's waveform from a single experiment is shown in a time series to analyze the CCA's behavior. Theoretical models are validated with experiments in [272], [281], [285].

The experiments may be repeated for different numbers K of flows. In [264], a steady-state scenario is evaluated where the number of competing flows K is larger than the BDP. In that case, the fair operating point of each flow is achieved when the number of packets in flight is between zero and one, i.e., each flow may only send a single segment every $1/\Omega$ RTTs. Many CCAs are not designed to operate at transmission rates less than one segment per RTT. As a consequence, these CCAs fail to converge to an efficient operating point that avoids congestion.

b) On-off traffic: Experiments with on-off traffic patterns can be found in [170], [171], [264], [268]. Exemplary time series of the throughput are typically shown to visually analyze how CCAs react to joining or departing flows. Furthermore, the convergence speed and stability are examined. Fairness among flows is evaluated with Jain's index.

c) Traffic mixtures: Experiments with traffic mixtures of DC traffic mixtures are covered in [264], [268], [269], [272], [273], [274], [285], [287], [289], [291], [292], [294], [297]. The flow sizes are typically drawn randomly from distributions extracted from real traces (synthetic traffic). The ratio between short and large flows may be varied but is typically chosen such that there a few large and many short flows. Supporting the coexistence of the two traffic patterns is challenging, especially in combination with incast scenarios, and an ongoing research topic. The experiments evaluate the coexistence of the different flows primarily by measuring flow completion times.

The effects of mixed DC and WAN traffic are studied in [264], [275]. The evaluation focuses on the coexistence between the two traffic types, which are different in particular in terms of flow sizes and RTTs. Flows that originate and terminate within the DC may start and finish on time scales that are less than the RTT of a WAN flow. Furthermore, the congestion signals and the different CCA schemes used in DCs are not used by WAN flows. Regardless, CCAs designed for the DC have to coexist with WAN flows. The study [275] suggests that different rate control schemes could be used depending on whether or not WAN traffic competes at a bottleneck. The study evaluates a scenario with a DC and WAN traffic mixture and examines the flow completion times, throughput, delays, and packet loss.

d) Incast problem: Experiments with incast problems are covered in [138], [172], [264], [268], [269], [272], [273], [285], [289], [292], [295], [298]. Incast experiments are often repeated with a different number of flows, different traffic loads, and in shallow-buffered scenarios. The flow completion time is the most important metric reported for incast experiments, typically its mean or 95th percentile (tail latency) is reported. The mean operating point may deteriorate as the number of flows grows. Incasts may be repeated with different flow sizes and optionally with added background traffic.

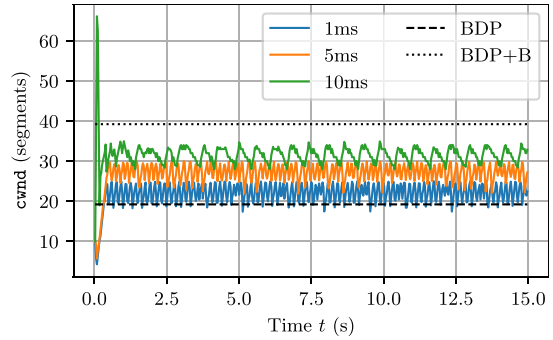


Fig. 11. Time series of the cwnd of DCTCP for different CE thresholds in experiments with ECN enabled and a CoDel qdisc at the bottleneck.

5) Experiments From ccperf: The *ECN* experiment of Table V sets up a network with K flows and the default path parameters of Table IV, but with a CoDel qdisc whose CE threshold is a specifiable experiment parameter. Fig. 11 shows time series of the cwnd over time of a DCTCP flow from three experiments with different CE thresholds. The experiments use our default scenario with a CoDel qdisc at the bottleneck. All three time series follow a similar pattern, i.e., oscillate around a mean value. However, the mean value is proportional to the CE threshold. With a CE threshold of 1 ms, the cwnd is close to the BDP because the CoDel qdisc quickly responds to growing queueing delay. In contrast, with a CE threshold of 10 ms, the CoDel qdisc allows larger queues to build up. Hence, the operating point of DCTCP depends on the CE threshold. In DCs, the marking threshold is a tunable parameter set by the operator. As we have seen in our experiments, the choice of the marking threshold may significantly affect the operating point of a CCA that uses ECN. In heterogeneous DC networks that deploy multiple types of switches that may differ from each other in terms of link speeds, marking algorithms, and other aspects, the tuning of marking thresholds can be challenging [264].

I. GEO Satellite Networks

Geosynchronous Earth Orbit (GEO) satellite networks are wireless access networks to the Internet for applications such as shipping, satellite telephony, and for limited connectivity in rural areas [299]. These applications typically require low data rates and can tolerate long propagation delays. The network conditions of GEO satellite networks are different from the conditions in Low Earth Orbit (LEO) satellite networks, which are further detailed in Section XIII-C13b. CCAs that are designed for or at least with GEO satellite networks in mind can be found in [196], [300].

1) Scenario: GEO satellite networks are characterized by long propagation delays, bandwidth asymmetry, and nonzero loss probabilities [299]. GEO satellite scenarios typically use topologies with a single bottleneck and *long-thin leaky pipes*,

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

i.e., the paths have large propagation delays, small bandwidths, and exhibit random losses. End-to-end path parameters that match the real-world WINDS satellite system can be found in [153]. The satellite link has a two-way propagation delay of $\mathtt{RTT}^{\text{prop}} = 800$ ms, a capacity $C = 42$ Mbits/s, and a constant packet error probability $\varepsilon = 0.74$ %.

A single-bottleneck topology is used in [196], in which K senders have a wired link to a first ground station and K sinks have a wired link to the second ground station. The ground stations are connected via a satellite, so each ground station has a wireless satellite link. The network is similar to the dumbbell network in Fig. 3, but the gateways have links to a satellite node. In a homogeneous scenario, all paths have the same path parameters; whereas, in a heterogeneous scenario, the propagation delays and packet error probabilities of the paths differ. A similar single-bottleneck topology is used in [300], in which one path includes a satellite link whereas all other paths only contain wired links. The satellite link directly connects a satellite with a sink. Furthermore, the satellite link has a time-variable propagation delay and packet error probability. The bottleneck link that limits the path capacities, however, is a wired link that connects two gateways. The scenario intends to evaluate the competition on paths with and without a satellite link.

2) Metrics: Long propagation delays paired with random losses may result in low throughput for many CCAs. Hence, throughput, flow completion times, and resilience towards random losses are the most commonly evaluated metrics in GEO satellite network scenarios.

3) Theoretical Modeling: The steady-state behavior of CCAs in GEO networks can in theory be modeled with the methods of Section VIII. For example, recall that the expected throughput of Reno is proportional to $1/(\sqrt{p}\,\mathtt{RTT})$ according to the response function of Eqn. (8), where p is the loss event rate. In GEO networks with large propagation delays and high loss event rates, the expected throughput of Reno is low. Note that RTOs, which are not considered in the formula, decrease the throughput even further. However, since the propagation delays are so high, the slow start of many CCAs may last significantly longer than in wired networks. The assumption that long-lived flows reach their steady-state behavior may be false and flows may spend the majority of their lifetime in slow start. Therefore, theoretical models that include the slow start phase are regarded as more realistic [196], [300], [301].

4) Experiments: The experiments set up satellite network scenarios and evaluate the operating point of CCAs. It is common to show time series plots of the allocation, e.g., $\mathtt{cwnd}(t)$, and analyze how CCAs can cope with long RTTs and random losses [196], [300]. The experiments in [196], [300] are repeated for different RTTs to evaluate how the waveform of a CCA changes.

Conducting a series of experiments that vary a single parameter to examine the parameter's influence is also common. For example, [300] varies the two-way propagation delay for different packet error probabilities, [196] varies the packet error probability and the capacity, and [138], [149] vary the queue size of paths using the WINDS satellite parameters. The results are reported as a sweep plot that shows the throughput with respect to the varied parameter.

In [170], [220], different CCAs are evaluated in separate experiments on paths with WINDS satellite parameters. The results are shown comparatively in an operating point plane (mean throughput vs. mean or 95th percentile of the queueing delay).

RTT fairness is evaluated in [300] by setting up satellite paths with different two-way propagation delays. In addition, [196] also varies the packet error probabilities of paths.

5) Experiments From ccperf: Experiments with GEO satellite networks are included in the *Synchronous Fairness* or *Start-time Fairness* experiments of Table V with the `path` parameter set to select the WINDS path parameters of Table IV.

J. Summary

Network variation evaluations challenge CCAs with a variety of network conditions. Different BDPs, packet error probabilities, buffer sizes, and AQM algorithms are a few examples that affect the performance of a CCA. In the past, loss-based CCAs were predominantly designed and evaluated. Reno's susceptibility towards random losses was especially prevalent in high-speed networks with large BDP. Hence, many proposed CCAs tried to address this scalability problem; and the evaluations were focused around that aspect. Low throughput was the primary concern of these evaluations. Afterward, bufferbloat was widely recognized as a problem; and avoiding self-inflicted queueing delay was set as a goal. The interaction of CCAs with AQM algorithms designed to mitigate bufferbloat, such as CoDel, became important. The effect and importance of buffer sizes for CCAs gained more attention. Today, scenarios that involve traffic policing or ACK compression extend the range of evaluations. Some scenarios can be modeled theoretically to a certain degree.

Evaluations of CCAs for DC networks differ from evaluations of CCAs designed for the Internet. DC networks are controlled environments that are less diverse and more centralized than WANs. Hence, CCAs can exploit advanced congestion signals, hardware capabilities, and other properties of DC networks. Furthermore, traffic patterns that exist only in DC networks have to be evaluated. The evaluation scenarios for DC networks reflect these unique characteristics of DC networks.

Network variation scenarios are essential to evaluate if a CCA behaves beneficially in a wide range of networks. The performance of CCAs may vary widely for different network parameters. For example, the same CCA may underutilize the bandwidth with small buffers, inflict a large queueing delay with deep buffers, and converge to qualitatively different equilibria when the buffer size is changed slightly between experiments. Therefore, a thorough evaluation of a CCA should include the network variation scenarios that we covered in this section.

XI. Dynamic Networks

In a time-variant network $\mathcal{G}(t)$, at least one path parameter varies over time; typically, the capacity, propagation delay, or packet error probability varies. This parameter variation introduces dynamics that are challenging for CCAs. First, CCAs

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

may have to continuously adapt their operating point as a reactive measure. For example, a path with a dynamic bottleneck rate directly varies the target value of the CCAs. Secondly, time-variable networks increase the uncertainty of congestion signals, e.g., RTT samples may quickly become outdated. In this section, we cover experiments with time-variant networks. We restrict ourselves to two network technologies for which CCAs are commonly evaluated: WiFi and cellular networks.

A. 802.11 LANs

The IEEE 802.11 (WiFi) protocol standard specifies data link and physical layer protocols for implementing wireless LANs that use unlicensed frequency bands. The dynamics of WiFi networks mainly stem from the time-variant wireless channel and medium access contention. Transport layer protocols have to cope with a number of challenges.

First of all, the wireless link is accessed by nodes through a medium access control mechanism in order to avoid radio collisions. Nodes run link adaptation algorithms that select suitable Modulation and Coding Schemes (MCSs) depending on the radio link quality of the wireless link. As a consequence, both the bandwidth and the transmission delays of these wireless links vary over time. At the transport layer, WiFi networks exhibit RTT variations and spikes, which complicate RTT estimations and increases the chance of spurious RTOs.

We briefly note that Mobile Ad hoc Networks (MANETs) that use the WiFi technology for radio transmissions introduce additional challenges, such as asymmetric multi-hop paths [302]. Also, node mobility leads to route changes, link, and route failures, as well as packet reordering. The evaluation of CCAs for MANETs [303], [304], [305], [306], [307], [308], [309] is outside the scope of this article and is left for future work.

1) Scenario: Several WiFi network scenarios have been used to evaluate CCAs. WiFi devices are often equipped with large buffers to avoid dropping packets during periods when the channel is busy, but this introduces the bufferbloat problems of Section X-C. Furthermore, random packet loss is more common compared to wired networks, which can degrade the throughput of CCAs. Packet loss may be caused by random bit errors, interference, radio collision, or short outages (link failures) due to node mobility [310]. Moreover, ACK aggregation is commonly used to reduce the number of transmissions.

In [311], a LAN is set up in which one sender is connected to an access point with a wired Ethernet link and one receiver is connected to the same access point with a WiFi link. Similar single-user and multi-user LAN scenarios are used in [312]. The link qualities are varied purposefully by periodically changing the MCS index of the WiFi access point. In two different experiments, the indices are either alternated or set by a Brownian motion model, respectively.

A network topology that is a variant of a dumbbell topology of Fig. 3 with $K = 1$ is emulated in [313]. In the topology, the gateway g_1 (sender side) is connected to an access point. The client (sender) is connected to that access point with an emulated WiFi link with static parameters. A key limitation of [313] is the emulation of WiFi without time-variant links. The Mahimahi emulator [257] is used in [314] to replay real WiFi packet departure traces. How packet traces are replayed with Mahimahi is elaborated in Section XI-B.

Scenarios with outages are covered in [315]. During an outage, the IP checksums of packets are deliberately corrupted. Corrupted packets are dropped automatically by the TCP/IP stack before reaching the transport layer.

Scenarios with split-TCP connections are evaluated in [316]. In a split-TCP scenario, a connection is split up into a wired portion and a wireless portion. A node that is connected to the wireless link, in this case, the WiFi link, serves as a proxy. The proxy node has to maintain two connections and redirect traffic between the connections for end-to-end transport. The approach has the advantage that the wireless channel cannot impair both connections. A different CCA may be used for each connection; for example, a CCA designed to cope with the challenges of WiFi networks may be used for the WiFi connection.

2) Metrics: As time-variant links with nonzero packet error probabilities and outages are challenging for CCAs, the throughput is the most used performance metric. The number of lost packets, RTOs, and retransmissions are also of interest. Furthermore, as WiFi networks may use deep buffers, delays may be measured to evaluate the bufferbloat problem. Since mobile devices are often battery-powered, low energy consumption is desirable.

3) Theoretical Modeling: The performance of classical TCP over wireless links with random packet loss is modeled in [317]. However, models of WiFi networks are abstractions that simplify the complex dynamics of real networks. Hence, we do not cover the theoretical modeling in more detail in this article.

4) Experiments: The studies [311], [318], [319] discuss the results of individual simulation and testbed experiments to emphasize the challenges that CCAs face in WiFi networks. The time series of the cwnd or the number of acknowledged segments is shown and the response to loss events, outages, and RTOs is analyzed. The mean throughput and 95th percentile of the OWD of multiple CCAs are shown comparatively in an operating point plane. In [315], outages are examined further by measuring the flow completion times in a series of experiments that vary the duration of the outage. A long outage likely causes an RTO, which severely reduces the transmission rate and lengthens flow completion times. The influence of packet loss is further evaluated in [311], [313], [318], [319] by repeating experiments with different packet error probabilities (see Section X-B). The mean throughput and file completion times of Internet measurements using WiFi access links are examined in [149], [176], Performance evaluations of different CCAs on a commercial cloud gaming platform with hosts using WiFi to access the Internet are covered in [181]. Similarly, emulation, testbed, and Internet experiments were conducted in [320] for different WiFi standards, for which the throughput, delay, and network power (Section VI-B6) are evaluated.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

B. Cellular Networks

Cellular networks (mobile networks) are wireless networks specified by the 3rd Generation Partnership Project (3GPP). Besides WiFi, cellular networks are the most widely used wireless networks to access the Internet. The bottleneck of cellular networks is often the wireless link that connects a user device with a base station [321]. Cellular networks use scheduling algorithms that arbitrate the access of all competing users to the wireless medium. Furthermore, the MCS is adapted depending on the link quality. Consequently, the capacity and transmission delays vary over time. ARQ and forward error correction algorithms at lower layers conceal packet losses at the transport layer to a certain extent [322], [323].

1) Scenario: The scenario sets up mobile hosts (users) equipped with cellular network interfaces to connect to a cellular network. The cellular network provides access to a WAN. Typically, the hosts connect to a server located inside the WAN. Cellular network scenarios are diverse and a large array of different topologies and node configurations may be set up. The numbers and interconnections of base stations, cells, antenna capabilities, frequency bands, mobility patterns of mobile users, scheduling and handover algorithms, and traffic patterns are a few examples of setup options. Furthermore, in simulations or emulations, the wireless channel needs to be modeled with radio propagation models.

Outages either due to signal loss or failed handovers between cells are possible. Cellular networks often maintain a queue for each active user instead of shared queues. Therefore, the flows stemming from a single user compete only against themselves and queueing delays are self-inflicted. To absorb the link variability, cellular networks often use deep buffers that create bufferbloat problems [324], [325].

Cellular networks may also be emulated in an abstracted way. The Mahimahi emulator [257] allows replaying trace files of packet departures captured in cellular networks. A link then schedules one or more packet departures at each slot (every millisecond), which imitates the packet scheduler of a cellular network. Furthermore, nonzero packet error probabilities that possibly vary are often used.

2) Metrics: Throughput, delay, loss, and flow completion times are the most widely used metrics in evaluations with cellular networks.

3) Theoretical Modeling: As cellular networks are complex systems, theoretical models are essentially intractable. Evaluations of CCAs in cellular networks are typically limited to experiments.

4) Experiments:

a) Simulations: Packet-level simulations of cellular networks can replicate the system dynamics of cellular networks with a high level of detail. A scenario with a mixture of video-streaming and bulk transfer traffic generated by multiple users that move in an area that is covered by 21 cells is evaluated in [326]. The evaluation examines the throughput, OWD, packet error probabilities, and tail latencies (90th percentile) of the packets containing video frames.

Channel traces of real cellular networks are used in [219] for a packet-level simulation. In multiple experiments, the mobility models (pedestrian, slow driving, fast driving) are varied and flows compete for the resources in cellular networks. The results are shown comparatively in an operating point plane and the fairness is evaluated.

b) Emulations: Many studies use the Mahimahi emulator to replay trace files stemming from real cellular network measurements to emulate cellular networks [11], [149], [232], [312], [314], [327], [328], [329]. Experiments that use emulations only mimic cellular network paths in an abstracted way. Commonly, the operating point (throughput and delay) or file completion times are examined to measure the performance. The operating point is either shown over time or in the operating point plane.

Another approach that uses emulations with an even higher level of abstraction is used in [330]. The experiments use *netem* to change the bandwidth, loss, and two-way propagation delay every second to evaluate the flow completion times of a bulk transfer.

c) Internet experiments: Internet experiments that involve real cellular networks are conducted in [11], [219].

5) Experiments From ccperf: The *Replay Rate Trace* experiment of Table V sets up a network that configures the bottleneck link to replay a packet transmission trace file. The Pantheon project provides three trace files recorded in cellular networks that contain packet departure times that match Poisson processes with average transmission rates of 0.57 Mbits/s, 2.64 Mbits/s, and 3.04 Mbits/s, see Table IV. Furthermore, the *Random Walk* experiment changes the network parameters according to bounded correlated random walk models. The network parameters that can be changed are the capacity, the two-way propagation delay, and the packet error probability. In one experiment, all three network parameters may be changed simultaneously. For each network parameter, a lower bound $x_{\min}$, an upper bound $x_{\max}$, a correlation parameter ρ, and a noise level σ can be specified as experiment parameters to configure the random walk models. Each random walk is bounded between $x_{\min}$ and $x_{\max}$ to retain realistic values. Also, each random walk is correlated because network parameters often exhibit correlations over time, e.g., the capacity deteriorates due to slow fading in a wireless channel. The unbounded random walk model is given by $X_t = X_{t-1} + \Delta X_t$ with the step size $\Delta X_t = \rho \Delta X_{t-1} + \epsilon_t$, for a given static user-specified correlation parameter $\rho \in [0, 1]$ and a random ϵ_t drawn from a normal distribution $\mathcal{N}(0, \sigma^2)$. The bounded random walk is then obtained by enforcing the boundary constraint with $X_t = \min[\max(X_t, x_{\min}),\ x_{\max}]$.

Fig. 12 shows the cwnd over time of Cubic and Vegas from two separate *Replay Rate Trace* experiments. The experiments set up a single flow in a network with the path parameters of the Mexico (cellular) to AWS California path of Table IV. The trace generates a time-varying capacity of 2.64 Mbits/s on average. The time-varying capacity is reflected by the changes of the BDP, which oscillates around the mean value of 38 packets. Cubic's slow start algorithm HyStart ramps up the cwnd quickly and overshoots ($\texttt{cwnd} > \texttt{BDP} + B$) until a loss event occurs. Due to the overshooting, Cubic sets a $\texttt{ssthresh}$ that is above $\texttt{BDP} + B$. As it exits LR, the first CA phase is brief and ends with a RTO, which resets cwnd to one segment. A second slow start phase follows that ends at approximately $t = 5.8$ s as cwnd reaches the updated

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

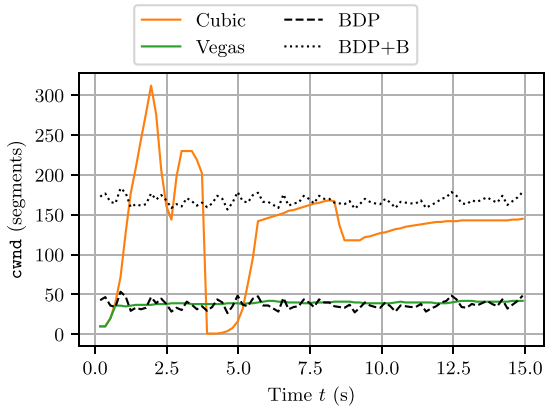


Fig. 12. Cellular network: Time series of cwnd for Cubic and Vegas (from two separate experiments).

ssthresh. By design, Cubic should now follow its steady-state behavior during CA, where the cwnd is concavely grown until it reaches a window target. The window target should be set to $\mathtt{BDP} + B$ to inflict a loss event. A steady-state cycle consists of concave growth to the target window to inflict a loss event, an LR phase, and a cwnd reduction. However, as $\mathtt{BDP} + B$ fluctuates over time, Cubic's second CA cycle uses a target window that is below $\mathtt{BDP} + B$. Cubic surpasses the inflection point of its concave window growth and starts a phase of convex growth after $t = 14$ s.

Vegas manages to converge to the mean BDP quickly. Vegas's cwnd also does not follow the BDP fluctuations accurately. Recall that to set cwnd, Vegas uses the sRTT and an estimate of the two-way propagation delay $\mathtt{RTT}^{\mathrm{prop}}$. The $\mathtt{RTT}^{\mathrm{prop}}$ is updated once per RTT and set to the minimum RTT sample of any segment. With a packet scheduler that replays the trace file at the bottleneck, packets may be delayed which adds noise to the RTT samples. However, the sRTT smoothes out the fluctuations and Vegas' $\mathtt{RTT}^{\mathrm{prop}}$ estimation remains accurate despite the noise. In contrast to Cubic, Vegas underutilizes the path during some time periods when the BDP is high. The experiments demonstrate that paths with dynamically changing capacities pose challenges for CCAs designed for wired networks. In general, the results indicate that in dynamic networks, CCAs may underutilize path capacities, generate larger queueing delays, and/or are more likely to suffer from RTOs, which, in turn, will likely reduce the quality of experience for a wide range of applications.

C. Summary

Dynamic networks have links that change over time and force CCAs to adjust to these changes to remain efficient. In this section, we covered evaluations in 802.11 LANs and cellular networks as examples. The complexity and dynamics of these networks render theoretical models infeasible to a large degree. Hence, evaluations rely heavily on experiments that use simulations, emulations, testbeds, or Internet measurements.

Two approaches for the setup of the scenarios can be differentiated. Simulations, testbeds, and Internet measurements evaluate CCAs in networks that generate realistic dynamics with none or a low degree of abstraction. In contrast, more recent studies increasingly rely on emulations that mimic the time-variant link conditions with packet schedulers. This emulation approach has a high degree of abstraction but is deemed sufficient as CCAs are part of the end-to-end transport layer.

Dynamic networks often use deep buffers to avoid packet losses as link qualities vary. This design choice comes at the cost of the bufferbloat problem. Recent studies evaluate more thoroughly if a CCA can avoid inflicting queueing delays despite links that vary over time.

Dynamic networks are a challenge for CCAs and will remain to be one in the future. A CCA that is efficient and fair in static networks, may perform poorly when link conditions are varying. Hence, a thorough CCA evaluation should include scenarios with dynamic networks.

XII. Competition

This section extends Section VIII, which covered intra-protocol competition evaluations that evaluate the steady-state behavior of CCAs. All evaluations covered in this section are competition evaluations with $K \geq 2$ flows competing for resources.

First, we cover transient-state intra-protocol competition evaluations, where flows join and leave the competition. As a result, the level of competition is time-variant, which interrupts the steady-state behavior of CCAs.

Second, we cover inter-protocol competition evaluations that let different CCAs compete for resources. In general, a newly proposed CCA could be tested against a large number of existing CCAs. However, since only a subset of existing CCAs is commonly used, newly proposed CCAs are typically only tested against established CCAs. We follow this approach and explain the most common inter-protocol competition experiments in this section.

Third, we cover experiments with background traffic that disturb CCAs. With background traffic, CCAs may struggle to converge to their steady-state behavior and the performance may suffer. Often, UDP flows are used to generate the background traffic.

A. On-Off Traffic

A common way to evaluate how a CCA responds to a change of competition is to use on-off flow patterns. With the on-off flow pattern, flows start and end throughout the duration of an evaluation. Whenever a new flow starts in the slow start phase, the flow quickly ramps up its transmission rate in order to establish itself among the competitors. The ramp-up typically creates sudden congestion, to which the other established flows react by backing off. On the other hand, when a flow leaves the competition, bandwidth is freed. Then, the remaining flows can increase their transmission rate in order to utilize the available bandwidth.

1) Scenario: An on-off scenario sets up hosts competing with traffic generated by applications that start and end throughout an evaluation. The scenario typically uses a single-bottleneck network. When switched on, an application running

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

on a host typically transmits CBR traffic with a constant data rate r. As a consequence, the aggregated data rate that arrives at the bottleneck follows a staircase function. Whenever a new flow starts or ends, the aggregated data rate is increased or decreased by r, respectively.

2) Metrics: Throughput, efficiency, fairness, flow completion times, the number of losses, and RTOs are useful metrics for the scenario. Furthermore, the stability of the allocation (Section VI-B11) can be examined to quantify how gracefully flows react to the dynamic traffic.

3) Theoretical Modeling: The dynamic traffic pattern of on-off traffic evaluation makes the theoretical modeling essentially intractable.

4) Experiments: On-off traffic evaluations with a periodic pattern, e.g., a flow starts and another flow stops every second, are conducted in [138], [149], [170]. In contrast, the on-off times may also be drawn from random distributions to create non-periodic traffic patterns [127], [171]. The studies examine how CCAs react to flows joining or leaving the competition by showing the throughput of each flow over time. Furthermore, long-term fairness and stability are often evaluated. For example, experiments with a very long duration (up to 90000 s) for various CCAs are conducted in [135]. The periodic interval of new flows joining or leaving is 1800 s. The fairness measured by Jain's index is reported for each interval separately. Furthermore, the stability of the throughput and queueing delay are analyzed visually with time series plots.

5) Experiments From ccperf: The *TCP On-Off Traffic* experiment of Table V sets up a network with K flows in a network with the default path parameters of Table IV. The start and stop times of flows can be specified, as well as the traffic generation model of each flow. Fig. 13 shows the number of packets in flight $\Omega(t)$ over time of six Cubic flows competing in an experiment. The flows start consecutively each second and its applications generate greedy-source traffic for a duration of 5 s. The experiment uses the default scenario with a bottleneck capacity of 90 Mbits/s. The first flow, Cubic 1, operates alone in the network without competitors for the first second. The flow quickly ramps up its cwnd to increase the number of packets in flight $\Omega(t)$ during slow start and acquires all bandwidth resources. Afterward, every second a new flow starts with a quick growth of its cwnd until the bottleneck queue overflows. A queue overflow happens each second for t<5s. With a shared bottleneck queue, multiple flows may experience a loss event and back off during one queue overflow. Furthermore, when flows end, the remaining CCAs require some time to acquire the freed bandwidth. As a consequence, the aggregated throughput of all flows may underutilize the bottleneck capacity. The experiment demonstrates that frequent flow startups and exits can generate disturbances that degrade performance.

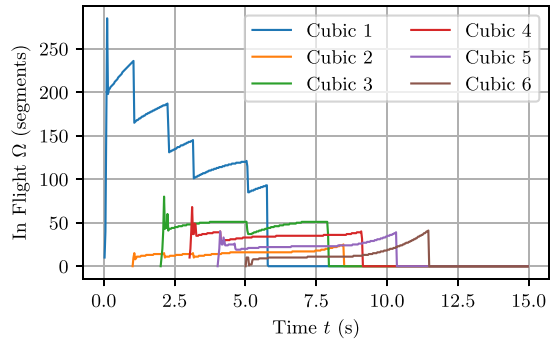


Fig. 13. Time series of the number of packets in flight $\Omega(t)$ for six competing Cubic flows in TCP on-off experiment.

B. TCP-Friendliness

To deploy a new CCA in existing networks and drive forward its adoption, it should be reasonably fair to competitors. CCAs that are overly aggressive may starve competitors that use other popular CCAs. The original definition of TCP-friendliness states that the expected transmission rate of a CCA should be at most twice the expected transmission rate of a Reno flow that experiences the same congestion signals [128]. Therefore, the term TCP-friendliness rather means Reno-friendliness. Since Cubic has established itself as the most popular CCA [79], fairness towards Cubic, i.e., Cubic-friendliness, represents an up-to-date version of TCP-friendliness. In general, a newly proposed CCA should be tested against the most popular CCAs of the time.

1) Scenario: TCP-friendliness is typically evaluated in a steady-state scenario. One or more flows that use the CCA under evaluation are set up to compete with one or more flows that use an established CCA, e.g., Cubic (Cubic-friendliness). The number of flows and the assignment of the CCAs to the flows may be varied.

2) Metrics: All metrics of Table III are applicable for the scenario. The deployability of a CCA in connection with the fairness to established CCAs is examined in more detail in [143].

3) Theoretical Modeling: TCP-friendliness towards Reno can be shown theoretically. The response function of Eqn. (8) models Reno's expected transmission rate. By modeling or measuring the response function of other loss-based CCAs, TCP-friendliness can be shown [116], [127], [163], [164], [165], [170], [331]. This approach is used by Cubic to continuously estimate Reno's transmission rate with Eqn. (8). Cubic may cap its own transmission rate to remain fair to Reno (TCP-friendly operating region) [76]. The TCP-friendliness can also be modeled in other ways, e.g., [332] models the interaction between BBR, Reno, and Cubic.

4) Experiments: Experiments with at least one Reno flow (classical TCP-friendliness) can be found in [59], [118], [135], [182]. The evaluations primarily focus on the fairness (Jain's index) towards Reno in steady-state scenarios. In [167], the experiments are repeated for different packet error probabilities. More recent studies rather test Cubic-friendliness [77], [115], [138], [149], [171], [183], [232], [320], [329]. Recently proposed CCAs struggle to be fair towards Cubic in many scenarios. In scenarios with random losses, recently proposed CCAs aim to be loss resilient and maintain a high throughput. On the other hand, Cubic backs off after each loss event.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

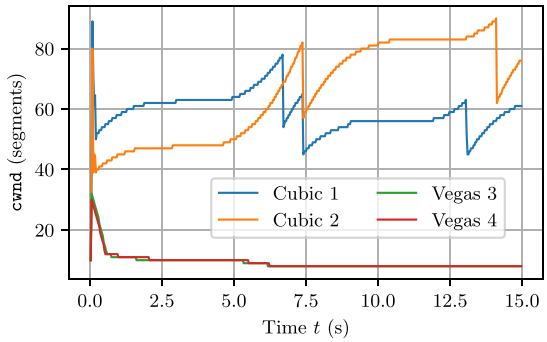


Fig. 14. Time series of the cwnd of a steady-state inter-protocol competition experiment with two Cubic and two Vegas flows.

Hence, loss resilience and fairness towards Cubic are contradictory design decisions to some degree. Loss-resilient CCAs may dominate Cubic in periods with random losses.

CCAs that are designed to not fill up queues and avoid self-inflicted queueing delays also conflict with Cubic's behavior. Cubic is a buffer-filler that blindly increases the number of packets in flight each RTT. When sharing a bottleneck queue with Cubic, a CCA's targeted operating point may be unreachable as Cubic generates a standing queue. A CCA that decreases its transmission rate in response to queueing delay, yields resources to buffer fillers and eventually starves itself. This is the case for Vegas [260], which by design decreases cwnd when it observes an increase in queueing delay, see Eqn. (7). Its inability to claim bandwidth against buffer fillers has been a challenge for the deployment of Vegas. TCP-friendliness is typically concerned about whether or not a CCA is too aggressive and unfair towards the established CCA, Vegas shows that the opposite may also be a concern. Experiments may evaluate whether or not a CCA is starved when competing against buffer fillers [115], [170]. Vice versa, the fairness of a newly proposed CCA towards CCAs that practice latency-avoidance may also be evaluated.

5) Experiments From ccperf: The *Reno Fairness*, *Cubic Fairness*, and *Vegas Fairness* of Table V set up inter-protocol competition experiments with K flows using the default path parameters of Table IV. The experiments focus on evaluating the efficiency and fairness of a CCA against the respective competitor. Half of the flows (rounded down) use the CCA under test, whereas the remaining flows use the competitor CCA. Fig. 14 shows the cwnd over time of an experiment where two Cubic and two Vegas flows compete with each other. The experiment sets up a network with the default path parameters of Table IV for $K = 4$. Both Vegas flows are starved as their cwnd is reduced to less than 5 packets over time. The experiment validates that Cubic is unfair towards Vegas. A similar degree of unfairness towards Vegas has been observed in experiments with Reno or BBR as the competitor [151].

C. Background Traffic

In real networks, bursty on-off traffic can disturb flows that use CCAs. The dynamics caused by such cross-traffic can influence the performance of CCAs and hamper convergence. Experiments exist in which background flows generate cross-traffic that consumes bandwidth. Hence, the available capacity of the bottleneck changes over time for other non-background flows. The background flows themselves are treated as noise; they are not evaluated in terms of performance.

1) Scenario: Scenarios with background traffic typically generate traffic on the forward path that shares a bottleneck with the flows that are evaluated. Background traffic can also be generated for the reverse path. Such background traffic can congest the reverse path and cause ACK compression as explained in Section X-F1. The data rate of the background traffic may vary over time and on-off traffic patterns may be used. Furthermore, the background flows often use UDP and are not responsive towards congestion.

2) Metrics: All metrics of Table III are applicable for this scenario.

3) Theoretical Modeling: Background traffic scenarios have not been modeled theoretically.

4) Experiments: The experiments are typically conducted in simulations, emulations, or testbeds. Experiments featuring on-off UDP traffic with manually set or randomly drawn transmission rates as well as on and off times are conducted in [116], [127], [167], [172], [319]. Similar experiments generate Web traffic as background traffic [133], [166]. The effect of reverse traffic on the throughput is evaluated in [259], [260]. Forward and reverse traffic (two-way traffic) in combination is used in [135] with on-off traffic, in [97] with Web traffic, and TCP background traffic in [165], [259]. The studies evaluate how different intensities and traffic patterns of background traffic affect the throughput and fairness of CCAs. The findings are that background traffic can severely impair the performance and stability of CCA. The background traffic adds noise to the system, which prevents the CCAs from converging to an equilibrium. Furthermore, bursty background traffic may cause random packet losses, which may lead to underutilization and temporary unfairness.

5) Experiments From ccperf: The *UDP Background Traffic* experiment of Table V sets up a scenario with $K = K_{\mathrm{TCP}} + K_{\mathrm{UDP}}$ flows in a network using the default path parameters of Table IV. The number of the TCP and UDP flows, the traffic generation models, and the start and stop times can be configured as experiment parameters. Fig. 15 shows the throughput over time of two Cubic and two UDP flows that compete against each other. All four flows start simultaneously at $t = 0$ s. The two Cubic flows use greedy-source traffic, whereas the UDP flows use Poisson traffic with a mean transmission rate of 1 Mbits/s. The UDP background traffic is supposed to disturb the two Cubic flows as their transmission rates randomly fluctuate over time. Sudden traffic bursts of the UDP flows lead to queue overflows, which create loss events for the Cubic flows. At approximately $t = 6$ s, the throughput of the second Cubic flow (Cubic 2) starts shrinking, whereas the throughput of the first flow (Cubic 1) starts growing by the same amount. Only Cubic 2 experiences a loss event and reduces its cwnd in response. In the following seconds, the throughput allocation becomes increasingly unfair. The experiment demonstrates that UDP traffic is a disturbance

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

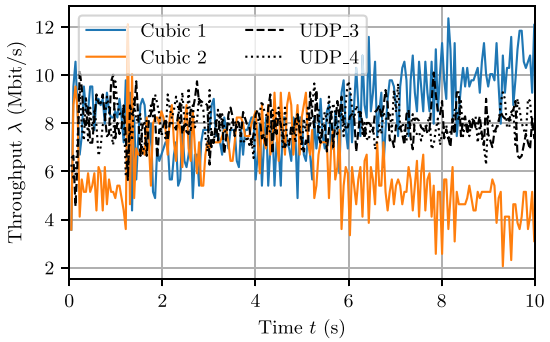


Fig. 15. Time series of the throughput λ for Cubic in UDP background traffic experiment.

for Cubic, which may negatively impact the efficiency and fairness.

D. Summary

Competition scenarios cover intra- and inter-protocol competition with a focus on efficiency and fairness. The scenarios complement the intra-protocol steady-state competition scenario in Section VIII-B. In on-off traffic scenarios, the traffic load changes over time either because flows start or end, or because applications generate time-variant traffic. In contrast to static scenarios, the time-variant traffic may prevent a CCA from converging to a steady-state behavior. Due to the complex traffic dynamics, such scenarios are predominantly evaluated experimentally. Although on-off traffic scenarios have been evaluated in studies for a long time, more recent studies have a stronger focus on delay metrics and flow completion times.

In the past, Reno was the predominant CCA used in the Internet. New CCAs were supposed to be fair towards Reno to a certain extent—a property known as TCP-friendliness. Many evaluations studied theoretically or experimentally to what degree other CCAs are TCP-friendly. As Reno is now largely superseded by Cubic, newer studies omit the classical scenarios in which other CCAs compete with Reno (Reno-friendliness). Instead, fairness towards Cubic is typically evaluated (Cubic-friendliness).

The coexistence of CCAs with unregulated UDP flows is another aspect worth evaluating. UDP flows are unresponsive to congestion and cause bandwidth variations over time. Scenarios with mixed TCP and UDP flows have been studied for a long time experimentally and the methodology is largely unchanged.

Inter-protocol competition and dynamic competition scenarios are essential to evaluate if a CCA is deployable in real networks. A CCA that is unable to cope with the challenges imposed by different competitors cannot be deployed, even if the CCA is efficient and fair towards itself.

XIII. Future Research Directions

The research area of evaluation of CCAs has existed since the first CCAs were proposed. Over the years, the evaluation methodologies shifted to match the evolution of network technologies. In this section, we outline future research directions for the evaluation of CCAs.

A. Theory vs. Practice: Math vs. Simulation, Emulation, and Measurements

Theoretical studies model CCAs analytically by formulating system models. As explained in Section VIII, the modeling of CCAs often focuses on the steady-state behavior after convergence. Furthermore, the modeling requires assumptions about the system conditions, i.e., network and traffic conditions. These assumptions are implicitly embedded in the system model. While system conditions that match the theoretical assumptions can be found in practice, they only cover a subset of real network conditions. Theoretical models often assume that traffic is generated by network-limited long-living flows, whereby all flows use the same CCA. However, in real networks, bursty on-off traffic is common, many flows are short-lived, application-limited, and a mixture of transport layer protocols and CCAs are used. Moreover, theoretical studies often assume that network conditions are static. Dynamic network conditions and random influences are not considered.

These differences in the system conditions express a gap between theory and practice. With that gap in mind, we ask the question: how applicable are theoretical findings for real networks? Is it a valuable property if a CCA *provably converges* in a theoretical system model despite the described gap? Two opposing positions regarding this question of theoretical convergence are presented in [156]. The first position is that theoretical models focusing on steady-state convergence "optimize in wonderland". These models would prove that a CCA converges in a scenario that rarely exists in real networks. Hence, the models are largely irrelevant for practical deployments.

The opposing position is that the property of theoretical convergence remains valuable because the steady-state scenario is essential for congestion control. The main purpose of a CCA is to avoid continuous congestion, i.e., congestion collapse that persists for multiple RTTs, inflicted by long-lasting flows. Hence, proving that a CCA converges to a suitable operating point serves as a guarantee for the stability of the Internet [333]. Furthermore, showing that the operating point is efficient and fair provides additional performance guarantees, e.g., flows are not starved.

The gap between theory and practice could also be reduced by improving theoretical models. This approach was taken by many studies in the past that added incremental changes to system models. Despite these efforts, system models remain an abstraction of real networks to keep mathematical analysis tractable. In this article we have pointed out that many transient-state, network variation, dynamic network, and competition scenarios have not been modeled adequately. The complexity of real networks exceeds the current system models by far. Hence, the theoretical modeling of CCAs would benefit from further studies that analyze which aspects should be abstracted away or added in detail to system models.

Recently, many proposed CCAs use learning techniques, such as machine learning [329], [334], [335], [336], [337], [338], [339], [340], [341], [342]. For these CCAs, the learnt rate update algorithm is a black box, which makes modeling the CCA infeasible. Further research on how to model such

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

CCAs is required; otherwise, learning-based CCAs are limited to experimental evaluations.

B. Evaluation and Benchmarking Frameworks

Frameworks to holistically evaluate and benchmark CCAs are another important direction for future work. We have strived to address this issue with our *ccperf* framework that we have introduced in Section VII. As explained in Section VII-F, a benchmark is a set of experiments conducted for a CCA under test with the aim to holistically evaluate the performance in a variety of scenarios using a comprehensive set of metrics. Finding a set of experiments and metrics for holistic test coverage is an ongoing activity at the IETF [9]. Pantheon [11] periodically ran Internet measurements and emulation experiments to compare the performance of CCAs before it was discontinued. Transperf [343] is an emulation framework to test and evaluate a CCA and was used for testing during the development of BBR. However, Transperf is not widely used in the CCA community and features only a small subset of the evaluation scenarios surveyed in this article. Future work can improve existing frameworks by adding new experiment types to increase the test coverage, improve the ease of usage for CCA designers, and improve the process of generating comparative results to facilitate comparisons of CCAs.

Today, the evaluation of CCAs is a task mainly left to the designers of new CCAs. As a consequence, the coverage of these evaluations is often limited for different reasons. First, there are many scenarios and metrics to consider for a holistic evaluation as attested by this survey article. However, designers of new CCAs focus on the design of their new CCA. The designers may be unaware of the range of existing evaluation experiment scenarios. Second, designers may be incentivized to cherry-pick evaluation scenarios with results that are favorable for their new CCA. In contrast, benchmarking frameworks can provide comprehensive evaluations featuring a wide range of scenarios. Furthermore, the frameworks can be designed to perform reproducible evaluations that allow a comparison of CCA in a fair manner. CCA designers would benefit from evaluation and benchmark frameworks as well. The frameworks can support designers in prototyping CCAs, finding bugs and shortcomings, and obtaining evaluation results.

C. Scenarios (Network and Traffic Setups) and Metrics

An essential future research direction is to examine which scenarios and metrics are necessary for a holistic performance evaluation. In the past, steady-state scenarios with static networks and constant traffic were typical. The main performance metrics have been throughput and fairness. Today, dynamic scenarios are commonly evaluated in addition to steady-state scenarios and the importance of delay has increased [225], [226], [227], [228], [229]. Hence, the scenarios and metrics of interest change over time as network technologies evolve.

1) What Is TCP-Friendliness Today?: From the large pool of CCA evaluation scenarios that we covered in this article, some evaluations may be outdated today. Section XII-B covered TCP-friendliness, which originally meant fairness towards the standard TCP using Reno according to the specification (Reno-friendliness). However, Reno is rarely used today in real networks [79]. That raises the question: should a newly proposed CCA be fair towards Reno and if so, to which degree? Many studies that introduce new CCAs do not evaluate Reno-friendliness anymore, i.e., the consensus leans towards deprecating Reno-friendliness as a desirable goal.

On the other hand, should newly proposed CCAs use Cubic-friendly operating modes similar to how Cubic has a TCP-friendliness operating mode to account for Reno? Is it desirable to enforce such backwards compatibility despite knowing that Cubic is not loss resilient and is negatively impacted by bufferbloat?

2) Improving Simulation and Emulation Setups: Sections X and XI covered scenarios that set up a variety of networks. Many evaluations use simulations or emulations to recreate the network conditions of the respective scenarios. An open question is how the setup of simulation and emulation experiments could be improved to enhance the realism of the evaluated scenarios. In the following, we give a few examples where we believe there is room for improvement.

Cable networks are used by many users worldwide to access the Internet [105], [344]. Often, studies only consider ACK aggregation as the primary concern for CCAs operating in cable networks. Other network conditions, such as the asymmetric time-variable bandwidth and traffic policing, have not been considered in experiments. Future work could improve the evaluation scenarios for cable networks by including the aforementioned and other aspects. A more thorough evaluation of CCAs operating in cable networks would be a viable contribution, as many Internet users use cable networks. The same applies for scenarios with ADSL Internet access, which are understudied in CCA evaluations.

ISPs use traffic policing that throttles the transmission rate of users to enforce data plans for both cable and ADSL access networks [198]. As we covered in Section X-G, traffic policing can degrade the performance of CCAs. Despite the prevalence of traffic policing in the Internet today, only a few studies have evaluated CCAs on policed paths [11], [262], [263], [345]. Future work should examine more thoroughly how CCAs cope with traffic policing and include traffic policing in cable and ADSL network scenarios.

To evaluate CCAs in wireless networks with emulations, a common abstract approach is to replay packet departure trace files as explained in Section XI. This approach could be supplemented by emulations that model real wireless networks, such as WiFi or cellular networks, more closely. For that, accurate emulators for these networks are required; otherwise, realistic CCA experiments have to resort to testbeds.

3) Relevance of Networks With Multiple Bottlenecks: CCAs have been primarily evaluated in single-bottleneck networks. The dumbbell topology of Fig. 3 is by far the most-used network topology in the surveyed evaluation studies. Otherwise, only Clos topologies for DC networks and the parking lot topologies [81] were used more than once in the surveyed studies. The parking lot topology has been used

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

by theoretical studies to analyze the efficiency and fairness of resource allocations as explained in Section VIII-B3d. However, to the best of our knowledge, there is no study that examines if parking lot topologies exist in real networks. Hence, it is unclear if CCA evaluations should include scenarios with parking lot topologies and if the evaluation results have a practical significance. Is it sufficient to optimize and evaluate CCAs in single-bottleneck networks? Do results from single-bottleneck network scenarios translate to multiple-bottleneck network scenarios? Future work should address these questions by examining if multiple-bottleneck topologies exist in real networks. Furthermore, a thorough evaluation in scenarios with multiple bottlenecks could uncover shortcomings and tradeoffs of CCAs that are yet understudied.

4) In-Network Congestion Control: Running algorithms at routers or switches in the network to avoid congestion is an idea that has became prominent since the RED qdisc was proposed in [85]. A success story of network-aided congestion control is the usage of ECN marking in DCs.

In-network CCAs are continuously being researched today. L4S is one example that aims to enhance ECN marking with more capabilities. The support for L4S in transport layer protocol implementations, e.g., the Linux kernel, is an ongoing effort. Each sender-side CCA needs to implement rate update decisions in response to L4S signals separately. If L4S establishes itself and gains many users, future CCA evaluations should include experiments in which L4S is enabled. These experiments would test whether or not a CCA can take advantage of L4S signals.

5) Priority Queueing: To meet the distinct service requirements of diverse applications, priority-aware AQMs have been proposed on the basis of DiffServ [243] and other traffic classification schemes [250]. Experimental evaluations of the performance of CCAs in scenarios that use priority queueing would be a valuable contribution. The experiments should investigate how CCAs affect the interactions of flows with different priority classes.

6) Application-Specific Scenarios and Metrics: Table I indicates that bulk transfer, Web, and streaming applications are commonly used to generate traffic in CCA evaluations. Each such traffic generation model can be further subdivided depending on the exact application that is evaluated. Bulk transfer traffic may stem from HTTP file downloads, FTP downloads, or peer-to-peer file sharing applications. Web traffic may use different HTTP versions and specific request-response cycles that depend on the application. Streaming traffic may stem from audio, video, gaming, or immersive volumetric media (virtual reality or augmented reality applications). Each such more specific traffic model generates a characteristic traffic pattern in terms of data volume and message transmission timings at the application. Future research should identify which applications and corresponding traffic models should be included as evaluation scenarios in comprehensive CCAs evaluations. For example, a *gaming traffic* scenario would generate characteristic gaming traffic and evaluate if the real-time service requirements of the gaming applications can be fulfilled when a CCA under test is evaluated. Correspondingly, future research should design tailor-made Quality of Experience (QoE) metrics for such application-specific scenarios, e.g., in a video streaming scenario, playback stalls could be evaluated as a QoE metric.

7) Evaluating CCAs in Large-Scale Networks and Data Centers: Simulations, emulations, and testbed evaluations of CCAs typically set up small networks that have a single bottleneck. With such small-scale scenarios many characteristics of CCAs can be evaluated, as attested by this article. Real networks, however, often exceed the scale of these experimental networks by far. Experimental evaluations of CCAs in scenarios with large-scale networks are desirable to provide evidence that a CCA can be deployed in real networks. However, large-scale experiments are challenging for a variety of reasons. Simulating large-scale networks with many nodes that generate, consume, or forward packets, may be infeasible due to the scaling of the runtime complexity. The runtime complexity of discrete-event simulations scales linearly with the number of scheduled events, e.g., packet transmissions. Likewise, emulations running on a single host face scaling problems as the processing powers of CPUs are limited. When these CPU limits are reached, growing processing delays become a source of error for the evaluation results. Testbeds are typically small-scale by design as the monetary means of researchers are limited.

To evaluate CCAs in large-scale networks, two approaches can commonly be found in the CCA literature. Internet experiments set up hosts that send data through their ISP to a server. These experiments provide evidence that a CCA performs well *in the wild* Internet. If multiple servers are available, the experiments can be performed through different Internet paths, as done by Pantheon [11]. The disadvantages of these Internet experiments are that the network topology is unknown, also the competing traffic is unknown and may change between experiments; hence, the experiments are not reproducible.

DCs are also large-scale networks with hundreds of nodes. In contrast to the Internet, a single organization typically owns the DC. One example of a large-scale CCA evaluation in a DC is presented in [77] for the BBR CCA. For BBR, the designers conducted deployment tests in Google's B4 WAN network and in YouTube edge deployments as A/B tests against Cubic. By demonstrating that BBR outperforms Cubic in these production networks with realistic traffic, the designers created a strong argument that BBR can be deployed at scale.

Most researchers do not have access to large-scale DC networks and therefore cannot conduct experiments of that scale in a controlled environment. Future work should address this problem by finding alternatives for large-scale experiments that provide similar evidence that a new CCA is safe to deploy.

8) Evaluation of CCAs Based on Learning Algorithms: Recently, many new CCAs have been proposed that use machine learning algorithms. The machine learning algorithms provide estimations that are used by the CCAs to make rate update decisions. Machine learning algorithms are used as black boxes that cannot explain their estimation outcomes. Therefore, it may not be possible to explain why a

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

learning-based CCAs updates its rate in the way it does. The lack of explainability and theoretical modeling, as explained in Section XIII-A, undermines the trustworthiness of such CCAs for real deployments.

The behaviors of learning-based CCAs depend on the data they are trained with. As a consequence, training these CCAs on a wide range of scenarios is important for a real deployment. But what if the CCA faces a network condition it has not been trained on, i.e., is the CCA robust towards unseen network conditions? In that case, it may happen that the CCA behaves in an unexpected way, which reduces performance and stability. The robustness of learning-based CCAs needs to be further investigated in future work before learning-based CCAs become feasible for real deployments. One approach is to combine a learning-based approach with manual rules to mitigate unexpected behaviors [232].

Another issue of learning-based CCAs is that of increased computation and memory consumption. High resource consumption can prevent a CCA from being deployed at hosts with few resources or hosts that maintain many connections in parallel. A CCA that introduces a significant computation delay or that uses a large fraction of system resources cannot be deployed widely. Future work needs to assess the feasibility of learning-based CCAs in terms of the consumption of system resources in detail.

9) Time-Sensitive Networking: Time-Sensitive Networking (TSN) is a set of link layer standards developed by the IEEE 802.1 work group. The goal of TSN is to replace the best-effort service of link layer switches with a deterministic real-time service. To achieve such a real-time service, TSN is primarily based on three components: i) time synchronization, ii) traffic shaping and scheduling, and iii) path selection. Traffic shaping and scheduling are used to plan packet transmissions. Future work should evaluate CCAs in scenarios that use TSN at the link layer.

Some high-priority TSN traffic streams are constant bitrate streams with low to moderate bitrates, e.g., periodic control messages to and from robots, that will likely not require congestion control. However, other high-priority TSN traffic streams, e.g., video feeds of the controlled robots, may generate bursty traffic that may require congestion control. Future research should examine the relevance of CCAs for the TSN traffic flows, and develop appropriate TSN network and traffic models for evaluating CCAs for TSN traffic streams.

Importantly, TSN traffic flows typically share the underlying physical network structure with non-TSN best-effort (bulk-transfer) traffic [346], [347]. The impact of the scheduling of the TSN traffic flows on this best-effort traffic, specifically, the CCA behavior of these best-effort flows, i.e., the coexistence of TSN and best-effort flows, needs to be examined in future CCA evaluations. For instance, TSN may operate without or with a so-called guardband preceding each transmission window of TSN traffic to protect TSN traffic from interfering best-effort traffic. The guardbands prevent the transmission of best-effort traffic and may contribute to congestion for best-effort traffic, and thus affect the CCAs of best-effort traffic. Also, the interfacing of TSN networks with other networks and the resulting potential congestion points [346], [347], [348], and broadly the interactions between TSN and CCAs, need to be considered in CCA evaluations.

Generally, evaluations involving TSN require highly precise evaluation methodologies with timing precisions as fine-grained as tens of nanoseconds [349], [350], [351], [352], [353]. Future research needs to examine what time granularities are required for the meaningful evaluation of scenarios involving TSN mechanisms. Especially, emerging CC mechanisms for low latency, e.g., [295], should be evaluated for these timeliness aspects.

10) Deterministic Networking: TSN is a link layer protocol for a point-to-point real-time service over a link. In contrast, Deterministic Networking (DetNet) aims to achieve deterministic real-time communication at the network layer. In addition to the traffic engineering of TSN at the link layer, DetNet implements a transmission resources reservation scheme at the network layer. Resources can be reserved in advance for traffic that is transmitted through a route in order to achieve a deterministic end-to-end network layer. DetNet uses a Software-Defined Networking (SDN) controller (see Section XIII-C11) to reserve resources at routers. Furthermore, multipath routing is used to increase the reliability. Future work should conduct a performance evaluation of CCAs operating in a DetNet network.

11) Software-Defined Networking and Network Function Virtualization: The Internet infrastructure connects dedicated network hardware that is designed to perform specific network functions with high speed, low processing delays, and low power consumption. SDN aims to add flexible network management to efficient implementations of network functions. A key principle of SDN is to separate the control plane and data plane. The control plane is used for network management, e.g., reconfigure the switching policies of a switch. A SDN controller has the task of configuring other network functions. Standardized APIs exist for the reconfiguration, e.g., OpenFlow [354]. The data plane handles the transmission of the actual user data according to the policies set by the controller. While the network functions may still be implemented in hardware, they can be reconfigured through the standardized API.

The flexibility of SDN enables network operators to reconfigure their networks over time. For CCAs, a reconfiguration likely changes the end-to-end characteristics between a sender and a receiver. For example, delay variations, out-of-order packet deliveries, and capacity changes may occur as a consequence. The reconfigurations cannot be observed by the hosts at the transport layer. The lack of observability can be problematic for CCAs that predict end-to-end parameters, e.g., the two-way propagation delay or bottleneck capacity.

Another technology that aims to provide flexibility is Network Function Virtualization (NFV). With NFV, network functions, e.g., switches or firewalls, are implemented in software instead of hardware. The softwarized network functions are called Virtual Network Functions (VNFs) as they leverage virtualization technologies, such as virtual machines or containers. VNFs provide flexibility in terms of deployment and management. A VNF instance can be spawned, executed, and terminated dynamically on a host. Furthermore, it is

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

possible to migrate an instance to another host. Computation and memory resources of a VNF instance can be set by the host and may change over time (vertical scaling). Furthermore, the number of VNF instances of a specific type may be increased or decreased over time (horizontal scaling). If multiple VNF instances of the same type exist, the ingress data can be distributed among the instances for load balancing. Both horizontal and vertical scaling can be conducted in a reactive manner depending on the traffic load that a VNF has to process. A data packet may pass through multiple VNFs on its way from the sender to the receiver. The sequence of VNFs constitutes a service function chain. VNFs instances running on different physical hosts may be part of one service function chain. Similar to SDN, the capabilities of NFV may introduce new end-to-end dynamics that CCAs have to cope with. NFV may increase the variance of processing delays and the likelihood of out-of-order packet deliveries.

In-network computing is a paradigm that differs from the end-to-end principle that keeps networks simple. Instead of simple store-and-forward operations, network functions may implement store-compute-forward operations [355]. SDN and NFV can be used as enabling technologies to implement in-network computing functionalities. Applications may benefit from the computation capabilities provided by the network. For example, sensor networks may perform sensor aggregation within the network to reduce the traffic volume. In contrast to NFV, the functions provided by the network may go beyond the implementation of network functions, such as firewalls.

To summarize, SDN, NFV, and in-network computing technologies introduce new end-to-end dynamics for CCAs. Future work should propose scenarios that incorporate these dynamics in order to evaluate how CCAs are affected. SDN and NFV can be used for network management in a variety of ways because of their flexibility. In real networks, however, network management possibly follows common policies that may be reproducible in evaluations. Pinpointing the exact evaluation scenarios to replicate such network management policies would be valuable for the evaluation of CCAs.

12) Proxy-Based Network Security and Content Distribution Services: Communication networks face a plethora of cybersecurity threats [356], [357], [358], [359], which have prompted the development of a wide range of network security services. Commonly, network security services encompass firewall, intrusion detection and prevention, as well as antibot protection. These services can be bundled in a Unified Threat Management (UTM) service [360], [361], [362], [363], which can be provided with dedicated (physical) UTM devices or can be virtualized in the form of UTM VNFs.

For the scenario of dedicated UTM devices, a potential future research direction could explore the strategy of terminating TCP connections at the UTM devices. The functionality of terminating TCP connections at an intermediate network node is typically implemented by a proxy server; in the UTM context, the UTM devices would become UTM proxy servers. With such a UTM proxy server scenario, there could, for instance, be a TCP connection from a local client host in a UTM-protected client network domain to the client's UTM service (UTM proxy server), a TCP connection from the client's UTM service to the server's UTM service, and a TCP connection from the server's UTM service (UTM proxy server) to the actual server host in the UTM-protected server network domain.

Some cloud security and content providers, e.g., Cloudflare Inc., Google Cloud Armor, and Microsoft Azure DDoS Protection, combine DDoS protection with content distribution network services in a global cloud network. With such a global cloud network, there is typically a TCP connection from the client to the cloud network and a TCP connection from the cloud network to the origin server. Many COTS UTM services already have this UTM proxy server functionality (which is common, e.g., for proxy firewalls). However, the UTM proxy server mode is typically limited to select protocols, e.g., HTTP/HTTPS and FTP, that are good candidates for proxy services, as not all TCP-based application protocols work well by splitting a TCP session. Proxy services on the UTM tend to add more load to the UTM as the UTM now has to split the TCP connection, inspect each connection, and rebuild it. In newer deployments, the UTM proxy inspects encrypted traffic (in particular HTTPS), i.e., the UTM acts as a proxy server, intercepts the traffic, decrypts the traffic by rendering the certificate, inspects the traffic, and re-encrypts the traffic for forwarding. Future research on CCA evaluation experiments should explore scenarios for splitting TCP connections at proxies, which are commonly employed for security and content services.

13) Wireless Communication Systems:

a) 5G and beyond: The fifth generation (5G) of cellular networks, which has been deployed since 2020, enables use cases that require high throughput, reliable communication with low latency, and massive connectivity. Since the use cases have different requirements, network slicing may be used to divide the physical network into multiple virtual networks. Each network slice is configured to provide a tailor-made service that fulfills specific requirements attributed to the slice. Whenever a user creates a session to transmit data, a network slice is assigned to that user. Hence, network slicing strongly influences the end-to-end path properties between a sender and receiver. The interaction of network slicing and CCA performance should be investigated in detail in future research.

5G uses NFV to implement its functionalities with VNFs. NFV provides more flexible software solutions compared to dedicated hardware, as explained in Section XIII-C11. Furthermore, 5G networks may use *smart* network management algorithms that leverage machine learning to dynamically reconfigure the network. Such smart networks are envisioned to organize themselves and adjust network configurations in a proactive and reactive manner. These abrupt reconfigurations may be challenging for CCAs and further research on designing evaluation experiments that reflect these abrupt reconfigurations in an authentic manner is required.

Since 5G is designed to support high data rates (more than 1 Gbits/s), CCAs may have to be able to scale up to such high-speed networks. In contrast to wired networks, the wireless communication channel with its varying transmission properties may impede CCAs from scaling up their transmission rate

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

within a reasonable time. The growth of the transmission rate of Cubic may be too slow to grow many orders of magnitude quickly, e.g., from 10 Mbits/s to 1 Gbits/s. A research question is whether widely used CCAs can achieve full utilization in 5G networks.

As in previous cellular network generations, packet loss, a varying end-to-end capacity, and a varying RTT may occur. 5G adds dynamic network management functions to the dynamic nature of the end-to-end path that includes the wireless radio link of a cellular network. Future research needs to develop experiments for evaluating the performance of CCAs in 5G networks. In particular, the research question whether widely used CCAs can adjust to the dynamics of 5G needs to be examined in detail.

While 5G is rolled out by network operators today, the sixth generation (6G) of cellular networks is being explored in research [65], [364], [365], [366]. Besides further improvements for the physical layer radio link transmissions, it is envisioned that the trend towards smart self-managing networks continues. 6G is being designed to incorporate artificial intelligence into the network, both for user application and network functions. Hence, dynamic reconfigurations—which challenge CCAs to adjust—are likely becoming more prevalent with 6G. Furthermore, 6G is supposed to provide global coverage by integrating different access network types besides the radio links of cellular networks. Satellite communication, unmanned aerial vehicle communication, and maritime communication networks are envisioned to seamlessly integrate into multiple radio access technologies. For CCAs a switch of the radio access technology is a routing change that abruptly changes the end-to-end path properties. To evaluate CCA scenarios with multiple radio access technologies, the radio access technologies, including their wireless channels, have to be modeled. However, it is currently unclear which level of abstraction (detail) each of these models should have for meaningful CCA evaluations. Therefore, future research needs to investigate the appropriate level of abstractions in radio access technology models for the purpose of evaluating CCAs. Also, further research is required to narrow down which scenarios should be covered to evaluate CCAs in 6G.

b) Satellite communication: In the past, GEO satellite networks were used for applications in remote areas that lack broadband Internet access from terrestrial networks. With low data rates and high latencies, the requirements of many user applications could not be satisfied by these satellite networks. Starting from 2016, the companies Telesat [367], SpaceX [368], and Amazon [369] successively announced the deployment of LEO satellites in large constellations. Today, the Starlink network of SpaceX deploys more than 6000 LEO satellites and plans to increase this number up to 12000. In contrast to former GEO satellite networks, the market of LEO satellite networks has shifted from niche applications to competing with terrestrial networks in providing broadband Internet access [370], [371], [372], [373].

LEO satellite networks can provide high data rates and low end-to-end delays. Apart from many distinct technological advances, the system design of the LEO satellite networks makes broadband Internet access feasible. High data rates and low delays are achievable because the satellite fleets are huge and the LEO is relatively close to the surface of the earth. In contrast to GEO satellites, LEO satellites require higher velocities to remain in their orbit. LEO satellites complete their orbit in approximately 100 min, depending on the exact elevation. Hence, the satellites are not static with respect to the Earth's surface. The satellites maintain a limited number of links to other satellites (inter-satellite links) and to ground stations. As each satellite follows its own trajectory in its orbit, the constellation (physical topology) of satellites changes over time. Consequently, handoffs are required to establish or terminate links between satellites and ground stations [374]. Handoffs result in route changes that influence the properties of end-to-end satellite paths.

CCAs may misinterpret the propagation delay variations originating from route changes as congestion [375]. The delay dynamics may affect many recently proposed CCAs [376], which estimate the two-way propagation delay, e.g., BBR [77], Copa [170], or PCC [138]. Furthermore, in the worst case, network outages are possible for coverage areas with a sparse satellite density. Similar to other wireless networks, outages may cause RTOs and may lead to underutilization as an aftereffect. Some CCAs, such as Westwood [252], Cherry [377], Peach [196], and SEARCH [378], were designed for satellite networks. However, the existing CCAs have primarily been designed for static GEO satellite networks with low data rates and high RTTs (*long thin pipes*).

The dynamic LEO satellite networks exhibit different network properties. Thorough performance evaluations of CCAs operating in LEO satellite networks are important future work. To conduct LEO satellite network evaluations, ns-3 simulation models of LEO satellite networks have been proposed in [379]. Initial simulations suggest that packet reordering may occur after a route change, which can trigger a loss recovery phase due to dupACKs. Loss-based CCAs that respond by decreasing the transmission rate are at risk of underutilization. However, delay variations, packet reordering, and outages are phenomena that are also present in other dynamic networks (see Section XI). Further research is required to investigate if the results of CCA evaluations operating in other dynamic networks translate to LEO satellite networks.

D. Multipath TCP and Generalized Networking Scenarios

This survey has focused on unicast packet traffic flows that traverse a single path from sender to receiver (whereby the single path could change, e.g., due to dynamic routing). However, a wide variety of generalized networking scenarios and corresponding generalized CCAs have been studied. For instance, a CCA for multicast QUIC is studied in [380] while CCAs for multipath transmissions [381], [382] have been studied in [339], [374], [383], [384], [385], [386], [387], [388]. Beyond multicast and multipath transmissions, generalized networking scenarios encompass cooperative and distributed networking scenarios [389], [390] (e.g., for content distribution [391], [392]), as well as scenarios with broadcast or synchronous transmissions [393], [394]. Future CCA

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

evaluation research should incorporate the gamut of such generalized networking scenarios.

We note that generalized networking scenarios may utilize generalized mechanisms at the lower protocol layers as well as cross-layer approaches which can bring significant benefits in challenging network environments [392]. For instance, the current 5G standards employ mainly polar codes for the forward error correction of point-to-point connections involving wireless links [395], [396]. However, generalized networking scenarios beyond point-to-point (unicast) connections are typically challenging for polar codes, while flexible network coding, such as Random Linear Network Coding (RLNC) and its variations [397], [398], [399], readily accommodates generalized networking scenarios, including recoding in intermediate network nodes [65], [400]. Network coding typically combines multiple successive packets in a linear random manner to enable the recovery of packets lost due to wireless link errors at the receiver [401], [402], [403], [404], [405]. The packet combining and recovery dynamics of network coding interact with the congestion control and packet reliability mechanisms of TCP [406], [407], [408], [409], [410]. As part of future comprehensive CCA evaluations, the network and traffic models should include scenarios that challenge the network coding mechanisms and thoroughly evaluate the interactions between the network coding and CCA mechanisms; broadly, cross-layer approaches should be rigorously evaluated in future research.

XIV. Conclusion

We have comprehensively surveyed evaluations for CCAs. We have taxonomized CCA evaluations according to the employed combination of network model, traffic model, and evaluation metrics, cf. Fig. 1 and Table V. We have comprehensively covered the evaluation scenarios (combinations of network model and traffic) and evaluation metrics that have been considered in CCA studies to date. We have organized the survey of the existing CCA evaluations into the main complementary categories of steady-state vs. transient-state evaluations, network variation (static network parameters are varied between runs) vs. dynamics networks (parameters vary dynamically over time, e.g., in wireless networks), and evaluations of competition between different CCAs.

The survey of the existing CCA evaluations indicates that individual CCA studies consider typically only a narrow set of network models, traffic models, and evaluation metrics. However, our survey, which includes quantitative illustrations of key CCA performance characteristics, indicates that comprehensive CCA evaluations are required to accurately assess the performance of CCAs. A comprehensive CCA evaluation includes scenarios with networks featuring different propagation delays, loss models, buffer sizes (deep and shallow buffers), AQM algorithms, and degrees of ACK compression, as well as dynamic network conditions. Furthermore, the traffic should be varied in scenarios with long or short-lived flows, flow start times, and different degrees of multiplexing. Besides the classical performance metrics of throughput, delay, loss, flow completion times, and fairness, we surveyed many other metrics for CCA evaluations.

Based on the comprehensive survey of the existing CCA evaluations, we have developed the ccperf framework [15] which composes comprehensive CCA evaluation experiments from applicable network models, traffic models, and evaluation metrics, as summarized in Tables IV and V. The composed experiments can be conducted with underlying simulation, emulation, or testbed modules; whereby the current implementation is based on an ns-3 simulation module.

Future research on CCA evaluation should develop comprehensive evaluations for emerging network transport paradigms, e.g., in-network computing, software-defined networking, and 5G systems. Also, the evaluation of CCAs based on machine learning requires urgent attention. We invite the networking research community to incorporate upcoming advances in CCA evaluation into the ccperf framework.

Appendix A
Background on CCAs in TCP/IP Stacks

A. Location of Congestion in Networks

Any network node that transmits packets can be the location of congestion when it is overwhelmed by traffic. In real networks, however, some nodes are more likely to be congested than others. Residential broadband networks often have bottlenecks at the edge of the ISP network in the *last mile* close to its users [105]. A bottleneck at the edge of the network is shared by only a small number of hosts accessing the Internet. Similarly, WiFi and cellular networks have wireless channels to connect users with the network. These wireless channels are typically bottlenecks. In contrast, bottlenecks at the core of WANs also exist, for example at Internet exchange points for peering [411], [412], [413]. Congestion at these bottlenecks may involve traffic from thousands of hosts and consequently affects many connections at once.

B. Differences in TCP/IP Stack Implementations

The TCP/IP protocol stack has five protocol layers: the physical, link, network, transport, and application protocol layers. There are many TCP/IP stack implementations that can be used to evaluate CCAs. For example, each modern OS implements its own version of a stack in software. User-space implementations exist as well; for example, mTCP implements a TCP/IP stack that leverages the kernel-bypass library DPDK [414], [415]. The ns-3 simulator provides its own TCP/IP stack implementation as well. Apart from subtle implementation differences of the software code, the TCP/IP stacks may use different sets of transport layer algorithms that affect the results of CCA evaluations in complicated ways that are beyond the scope of this survey. In the following, we present a few implementation examples that affect CCA evaluations. Furthermore, we provide some examples that illustrate how the TCP/IP stacks of Linux and ns-3 differ.

1) Socket Options: Transport layer protocols provide socket options that tweak their behavior. TCP has many options, such as the delayed ACK threshold, the SACK options, and the initial `cwnd` value. These options, e.g., the

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

setting of the initial `cwnd` value [416], can strongly influence CCA evaluations. Each TCP/IP stack has its own set of default values for socket options. Furthermore, the availability of socket options may differ. To make experiments reproducible, the socket options have to be disclosed. Typically, CC evaluation studies do not disclose all the used socket options. Instead, the studies state which TCP/IP stack they use and use the default socket option values, e.g., they state the used Linux distribution, including the version number.

2) Algorithms That Extend Standard TCP: Apart from the algorithms of standard TCP, namely slow start and CA, as well as fast retransmit and fast recovery (for LR), see Section II-F, many more algorithms closely related to CC have been proposed to improve TCP. Each TCP/IP stack version may implement a different set of these algorithms. For example, in Linux many more LR algorithms are implemented that extend the original fast retransmit and fast recovery algorithms. The Eifel detection and response algorithms [203], forward-ACKs [202], and forward-RTO-Recovery [209] are only a few examples that are implemented in the Linux TCP/IP stack (more details in Section IX-B). In contrast, the TCP/IP stack of ns-3 does presently not implement these algorithms. Furthermore, Linux uses the PRR [98] fast recovery algorithm instead of the fast recovery algorithm defined by NewReno [72]. Hence, CCA evaluations may give different results depending on the used TCP/IP stack implementation, e.g., Linux implementation vs. ns-3 implementation. The LR algorithms of Linux may recover from packet loss more quickly and hand over the control of the cwnd to the CA algorithm resuming from a different value. Comparisons of results of CCA evaluations that have been conducted with different TCP/IP stacks have to be conducted very carefully despite the fact that the implementations of the slow start and CA algorithms of a CCA may be very similar.

3) Efficient Data Processing of TCP/IP Stacks: To reach high data rates, fast processing speeds of TCP/IP stacks are required. Modern OSs use sophisticated efficiency-enhancing algorithms to process packets quickly and schedule packet transmissions accurately. The study [417] explains how the TCP implementation of Linux manages to maintain low CPU utilizations even at high data rates. Segmentation offloading and efficient packet pacing queues are two exemplary methods that accelerate data processing.

a) Segmentation offloading: Segmentation offloading is a batch processing technique to reduce per-packet processing overheads. Most CCA evaluations use a Maximum Transmission Unit (MTU) of 1500 B, because the Internet lacks support for larger frames. From a processing standpoint, a larger MTU is desirable to decrease the processing loads of nodes. Segmentation offloading is a technique to process segments at the transport layer that are larger than the Maximum Segment Size (MSS). Hence, a batch of MSS-sized segments can be processed at once to reduce processing overheads. The segmentation is then performed by an external algorithm that is run before packets are transmitted on a link. One approach is to offload the segmentation to the NIC (hardware offloading). TCP Segmentation Offload (TSO) offloads the segmentation before transmission to the NIC. Analogously, with Large Receive Offload (LRO), a NIC merges several MTU-sized packets into a single large packet after reception. An alternative is to use segmentation algorithms implemented by the TCP/IP stack (software offloading). Generic Segmentation Offload (GSO) is the algorithm for packet transmissions and Generic Receive Offload (GRO) is the algorithm for packet receptions. In Linux, the batch size of these algorithms may be as large as 64 kB, so that around 45 segments are either transmitted or received in one burst at the transport layer [418]. The batch size may be adjusted dynamically by a TSO autosizing algorithm.

b) Efficient packet pacing: Some CCAs use the pacing rate as the primary control variable, for example BBRv1. Accurate packet pacing is a prerequisite for these CCAs to operate as expected. When segmentation offloading is used, the packet pacing cannot be done at the transport layer. Linux solves this problem with the FQ scheduler that can efficiently pace packets with a nanosecond-granularity timer [417], [419].

c) Queue limits: Packet queues are essential for efficient data transmissions. When a device is ready for a new packet transmission, a packet has to be available to be pulled from a queue. At the same time, storing too many packets in queues increases latencies and creates problems, such as Head-of-Line Blocking (HoLB). These two issues can be addressed simultaneously by keeping the number of stored packets in queues small while ensuring that packets can be pulled from queues by transmission devices. Linux uses Dynamic Queue Limits (DQL) [420] and TCP Small Queues (TSQ) [421] to limit the number of enqueued packets in a NIC (hardware) and the TCP/IP stack (software), respectively.

C. Transport Layer APIs for CCAs

Transport layer implementations typically provide an API to implement new CCAs. A new CCA has to implement some functions that are called by the transport layer. These functions have access to information of the connection that is required to make rate decisions. For example, the functions can typically access the Transport Control Block (TCB) [192]. The API decouples the transport layer implementation from the CCA implementation. That is, the transport layer is agnostic to how exactly the CCA operates. When a sender creates a network socket for communication, the sender can specify a CCA with a socket option, see Section A-B1. Therefore, CCAs can be chosen on a per-connection basis. This decoupled design is advantageous for the evaluation and comparison of CCAs because the same experiment can easily be repeated for different CCAs. The TCP API of Linux is defined by the **tcp_congestion_ops** structure [422]. An excerpt of the structure of Linux v6.9.7 is shown in Listing 1. The **tcp_congestion_ops** structure contains the **name** member that is used by Linux to identify a CCA, e.g., *"cubic"* or *"reno"*. When opening a TCP socket, an application can specify a name to select a CCA. Each function receives the TCP socket structure (**sock *sk**) as the first argument. The socket structure provides access to the TCB and other state information. TCP calls the **ssthresh** function after LR is entered, so that the CCA can calculate the slow start threshold `ssthresh`. The **cong_avoid** and **pkts_acked** functions are called after an ACK is received. A CCA uses the **cong_avoid** function to

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

```
struct tcp_congestion_ops {
...
 u32 (*ssthresh)(struct sock *sk);
 void (*cong_avoid)(
    struct sock *sk, u32 ack, u32 acked);
 void (*pkts_acked)(
    struct sock *sk, const struct ack_sample
        *sample);
 void (*cong_control)(
    struct sock *sk, const struct rate_sample *rs);
 char name[TCP_CA_NAME_MAX];
...
}
```

Listing 1. Linux Pluggable CCA API (excerpt).

update cwnd. The functions **ssthresh** and **cong_avoid** are the only two functions that are required to implement the original AIMD scheme of Reno. The **ssthresh** function of Reno returns $\max(\texttt{cwnd}/2,\ 2)$ to halve cwnd (multiplicative decrease). Furthermore, in **cong_avoid** the cwnd is either increased by one segment if $\texttt{cwnd} < \texttt{ssthresh}$ during slow start or by $1/\texttt{cwnd}$ in CA (additive increase). During LR, TCP relies on a fast recovery algorithm (PRR in Linux) to control the number of packets in flight, i.e., Reno stops updating cwnd as long as segments are lost. The functions **pkts_acked** and **cong_avoid** are optional and not implemented by Reno. Other CCAs, e.g., Vegas and BBR, use the **pkts_acked** function to process RTT samples (**ack_sample *sample**). For the implementation of BBRv1, which uses the pacing rate as its primary control variable, Linux added the function **cong_control** to allow CCAs to update their pacing rate. The rate sample structure (**rate_sample *rs**) contains information about the estimated delivery rate as described in [92]. When a CCA implements **cong_control**, TCP does not invoke the fast recovery algorithm. Hence, BBR continues updating the pacing rate and cwnd during LR. Further implementation details of CCAs for the Linux TCP/IP stack are covered in [423], [424].

The **tcp_congestion_ops** structure is an exemplary implementation of Linux for a CCA API. Other transport layer implementations, e.g., TCP of ns-3 or Chrome's QUIC, use similar APIs for CCAs. The API design for CCAs has been researched in the past in an effort to unify the design across TCP/IP stacks [425], [426].

Appendix B
Traffic Model Details

A. Flow Identifiers

A flow can be identified via the 5-tuple consisting of the source and destination IP addresses, the source and destination port numbers, and the IP protocol field, which indicates the transport layer protocol, e.g., UDP or TCP. While the 5-tuple is traditionally used as flow classifier, flow classification based on a 3-tuple consisting of an IPv6 flow label as well as source and destination IPv6 addresses can be used in practice [427].

B. Objectives of Elephant and Mice Flows

Many CCA evaluation scenarios focus on elephant flows [113] since they live long enough to converge to steady-state and compete for a major share of the bandwidth resources with other elephant flows. Elephant flows typically stem from applications that generate large data files or streams. A typical objective of these flows is to maximize throughput. In contrast, mice flows are so short that they may end before exiting slow start. Web applications using HTTP are the most common examples. Typically, the objective of the mice flows is to finish as quickly as possible to minimize flow completion times at the transport layer and response times at the application layer.

C. On-Off Traffic Generation Model

A common traffic example from queueing theory is Poisson traffic [428]. With Poisson traffic, the on-time is set such that exactly one packet is generated and the off-time is drawn from an exponential distribution. Thereby, data transmissions from the application follow a Poisson point process with convenient mathematical properties. In experiments, on-off traffic introduces varying degrees of traffic load that CCAs have to cope with.

When multiple flows are aggregated at a forwarding node the traffic is multiplexed. Aggregated traffic has been modeled as smooth Poisson traffic for a long time. However, more recent studies suggest that aggregated traffic is self-similar and bursty [429]. In experimental studies, self-similar traffic can be generated by multiplexing on-off traffic sources [430]. The traffic sources have to generate CBR traffic during the *on* periods and the on- and off-times have to be drawn from a heavy-tailed distribution, e.g., from a Pareto distribution.

D. Media Streaming Traffic

Media streaming applications commonly employ adaptive streaming algorithms at the application layer to adapt the bitrate over time. Similar to CCAs, these adaptive streaming algorithms control the transmission bitrate. However, the purpose of the adaptive streaming control is *not* to avoid congestion; instead, the adaptive streaming control strives to avoid stream buffering and to enhance the quality of experience for the viewer. A popular adaptive bitrate streaming algorithm is DASH, which uses TCP at the transport layer. From the CCA (transport layer) perspective, a DASH application generates (and consumes) traffic that changes in bitrate over time. However, the DASH flow is in principle inelastic as significant deviations from the bitrate prescribed by DASH would noticeably impair the media quality of experience.

Appendix C
Measuring Metrics

A. Sampling Techniques

The sampling times of time series metrics in experiments commonly fall into three categories: i) on a per-packet basis, ii) scheduled, and iii) event-based sampling. Metrics that can be recorded on a per-packet basis, e.g., packet delays, can be captured with a high granularity (i.e., with a high sampling frequency). For these metrics, sampling instants are irregular and coincide with packet-related events, such as packet departures, arrivals, or drops. Packet sniffers are typically used to

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

capture metrics on a per-packet basis. The alternative is to schedule the sampling of a metric, such as periodic sampling or Poisson sampling [106]. In the CC literature, periodic sampling is typically used with fixed sampling intervals, e.g., every second. The choice of the sampling period influences the statistical properties of the sample metric. For some metrics, event-based sampling is used, which samples the metric when a specific event occurs, e.g., an RTO takes place.

In contrast to time series that measure a metric multiple times within one experiment, a metric can be sampled in *n* different experiments. For example, the mean throughput of flows may be sampled from different experiments to examine how the mean throughput is influenced by different evaluation scenarios. This typically happens when an experiment is repeated with different parameters.

B. Measurement Layer

Many metrics can be measured at different network protocol layers. The measurement layer affects the values of metrics. For example, the value of the throughput in bits/s typically depends on the size of the protocol data unit of the measurement layer. The throughput measured at the transport layer with segments (MSS) is less than the throughput measured at the link layer with frames (MTU), because the overhead of packet headers differs. The transport and link layer throughput can be converted to each other with a multiplicative adjustment factor (MSS / MTU). The application layer throughput for TCP, however, may significantly differ from the transport layer throughput, because of the reliable in-order delivery. After a loss event, the transport protocol at the receiver starves the application until the losses have been recovered for an in-order delivery. Hence, the throughput measured at the application layer abruptly drops to zero after a loss event until in-order delivery becomes possible, even though the transport layer throughput may remain high.

Regardless, many studies do not explicitly mention the measurement layer and use the unit *packet* as a generic protocol data unit (similar to type-P packets in [106]). Metrics that include bytes as unit, e.g., transmission rates or the number of sent bytes, can interchangeably be expressed in terms of packets by normalization. For example, the transport layer transmission rate in B/s can be divided by an MSS to be expressed in segments/s; a division by an MTU at the link layer yields a unit of packets/s.

C. Tools to Measure Metrics

To measure and save values of metrics, CCA evaluations use various tools to access state information from TCP/IP protocol stacks.

1) Simulation Experiments: Simulations typically have built-in mechanisms to export the state information at runtime. For example, ns-3 uses a tracing system that allows users to register callback functions for specific variables of interest (trace sources), e.g., the cwnd of a TCP socket. Whenever the trace source changes its value, the callback function is executed and is provided with the new value of the trace source. Hence, a user can log the changes to all the trace sources.

2) Emulations, Testbed, DC, and Internet Experiments: Emulations, as well as testbed, DC, and Internet experiments typically use dedicated programs to extract metrics at runtime. Common programs are *iperf* [99] or *netperf* [431], which log data rates and other statistics from the application layer (and can also generate traffic). The *ss* program [103] can display socket statistics at runtime. For TCP, *ss* displays the CCA, sRTT, cwnd, and other useful state information to extract metrics.

Packet sniffers, such as *tcpdump* [432], can be used to capture traffic on specified network interfaces. The packet sniffers are run while an experiment is executed. The captured traffic is saved to packet capture (pcap) files that contain descriptions of packet contents. To extract metrics from the pcap files, further post-processing of the packet descriptions is required.

D. Additional Flow Fairness Metrics

In [433], a β-fairness function is derived from five axioms that jointly define a mathematical notion of fairness. From these axioms the parameterized β-fairness function

$$f_\beta(\boldsymbol{x};\beta) = \operatorname{sgn}(1-\beta)\left[\sum_k \left(\frac{x_k}{\|\boldsymbol{x}\|_1}\right)^{1-\beta}\right]^{\frac{1}{\beta}} \tag{9}$$

is derived for any $\beta \neq 1$ with $\|\boldsymbol{x}\|_1 = \sum_k |x_k|$, whereby $\operatorname{sgn}(\cdot)$ is the sign function, e.g., $\operatorname{sgn}(-5) = -1$, and $|x_k|$ denotes the absolute value of x_k, e.g., $|-5| = 5$. The β-fairness function incorporates other fairness metrics as special cases. Setting $\beta = -1$ gives Jain's index from Eqn. (6) with $f_J(\boldsymbol{x}) = (1/K) f_\beta(\boldsymbol{x};\beta = -1)$, $\beta = 0$ gives the entropy fairness

$$f_\beta(\boldsymbol{x},\beta = 0) = \exp\left(H\left(\frac{\boldsymbol{x}}{\|\boldsymbol{x}\|_1}\right)\right), \tag{10}$$

where $H(\boldsymbol{x})$ is the Shannon entropy function [434]. Max-min fairness [435] is obtained for $\beta \to \infty$:

$$\lim_{\beta\to\infty} f_\beta(\boldsymbol{x};\beta) = -\max_k \frac{\|\boldsymbol{x}\|_1}{x_k}. \tag{11}$$

Other fairness functions are not part of the β-fairness function family. The product-measure fairness [436] is given by $f_P(\boldsymbol{x}) = \prod_{i=1}^k x_i$. The min-max ratio fairness [145] is given by $f_M(\boldsymbol{x}) = \min_k x_k / \max_k x_k$.

E. Utility Functions

The α-fairness utility function $U(\boldsymbol{x};\boldsymbol{w},\alpha)$ with a weight w_k is given by [435]:

$$U_\alpha(x_k;\alpha,w_k) = \begin{cases} w_k \frac{x_k^{1-\alpha}}{1-\alpha}, & \alpha \geq 0, \alpha \neq 1, \\ w_k \log(x_k), & \alpha = 1. \end{cases} \tag{12}$$

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

It is possible to factorize the aggregated α-fairness utility function into a fairness and utilization component with the help of the β-fairness function of Eqn. (9):

$$\sum_{k=1}^{K} U_\alpha(x_k;\alpha,w_k) = \frac{1}{1-\alpha}|f_\beta(\boldsymbol{x};\beta=\alpha)|^\alpha(\|\boldsymbol{x}\|_1)^{1-\alpha} = \underbrace{|f_\beta(\boldsymbol{x};\alpha)|^\alpha}_{\text{fairness}} \cdot \underbrace{U_\alpha(\|\boldsymbol{x}\|_1)}_{\text{utilization}}. \tag{13}$$

Different choices of α correspond to specific tradeoffs between the fairness and utilization components, which are all Pareto-optimal. Setting $\alpha = 1$ optimizes the proportional fairness of Eqn. (4). For $\alpha = 2$, the minimum potential delay fairness of Eqn. (5) is targeted. With $\alpha \to \infty$, the max-min fairness is the objective.

F. Convergence Criteria

This appendix elaborates Section VI-D3 by detailing convergence criteria.

1) Flow-Specific ϵ-Convergence Criterion: The flow-specific ϵ-convergence criterion is used in the studies [135], [136] with the throughput as a time series $\boldsymbol{x}_t$. To proceed, we define the multi-dimensional (vector) time series $\boldsymbol{x}_t$ as a two-dimensional matrix with elements $x_{k,t}$; $k = 1, 2, \ldots, K$; $t = 1, 2, \ldots, T$. The mean (over time) value $\overline{\boldsymbol{x}} = [\overline{x_1}, \overline{x_2}, \ldots, \overline{x_K}]^{\mathrm{T}}$ is interpreted as a long-term average that approximates the equilibrium. The long-term average is compared to a short-term Simple Moving Average (SMA) given by

$$\mathrm{SMA}(x_{k,t}) = \frac{1}{t}\sum_{i=1}^{t} x_{k,i} \tag{14}$$

for the ϵ-convergence criterion. With flow-wise convergence, a time series $\boldsymbol{x}_t$ converges at time t_c once the relative distance between the SMA of each flow and its mean value is continuously smaller or equal to ϵ for all $t > t_c$, i.e.,

$$t_c = \max_k \max_t \left\{ t\colon \left| \frac{\mathrm{SMA}(x_{k,t}) - \overline{x_k}}{\overline{x_k}} \right| > \epsilon \right\}. \tag{15}$$

With this criterion, the last flow to converge determines when the time series $\boldsymbol{x}_t$ as a whole converges. Note that by definition, $\overline{x_k} = \mathrm{SMA}(x_{k,T})$, which implies that $\boldsymbol{x}_t$ is guaranteed to converge with the ϵ-convergence criterion.

2) Aggregated ϵ-Convergence Criterion: The aggregated ϵ-convergence criterion is also used in the studies [135], [136]. Aggregated convergence is less strict and measures when the relative error between the aggregated time series $\sum_k x_k$ and the aggregated mean value $\sum_k \overline{x_k}$ remains smaller than ϵ, i.e.,

$$t_c = \max_t \left\{ t\colon \left| \frac{\sum_{k=1}^{K}\left[\mathrm{SMA}(x_{k,t}) - \overline{x_k}\right]}{\sum_{k=1}^{K}\overline{x_k}} \right| > \epsilon \right\}. \tag{16}$$

In [139], a third concept is introduced, in which only the allocation of the flow that started last is considered for the ϵ-convergence criterion.

3) δ-Fairness-Criterion: The δ-fairness criterion can be used in experiments with two flows. In the scenario, one flow starts first and acquires all resources. Then, a second flow joins and competes for the resources with the initial throughput allocation $\boldsymbol{\lambda}_0 = (C, \lambda_0)$; here, λ_0 is the minimum throughput of one packet per RTT. In [133], the δ-fairness criterion is fulfilled as soon as the throughput reaches the δ-fair allocation region of $\boldsymbol{\lambda}_c = (C(1+\delta)/2,\ C(1-\delta)/2)$ for $\delta = 0.1$. A stricter variant is used in [138], [149], where the throughput is required to remain within the δ-fair region for five seconds. Similar to the δ-fairness criterion, [97] requires the second flow to reach 80 % of the throughput of the first flow to establish convergence.

G. Remarks on Flow Rate Fairness

Flow rate fairness examines how evenly the transmission rates of the senders are distributed. In Section VI-D2, we surveyed fairness metrics that map an allocation vector to the unit interval. Most CC literature exclusively examines flow rate fairness, for which the allocation vector gathers the transmission rates (or related variables, such as the throughput or cwnd) of flows. In [437], the meaningfulness of flow rate fairness has been questioned for a variety of reasons. Flow rate fairness uses flows as the entities for which a fair allocation is targeted. However, since a single application can start many flows in parallel, fairness between flows does not necessarily correspond to fairness between applications. Many applications, such as peer-to-peer and Web applications, spawn many long-lived connections in parallel to improve their service [204]. Detecting such applications to mitigate this issue is technically challenging because obfuscation methods exist that make these applications indistinguishable from servers that cater to many clients in parallel or many users that share source addresses (as they are behind a router using Network Address Translation (NAT)).

Flow rate fairness implies that applications have the objective to gain a fair share of bandwidth resources. Real-time applications, however, may prioritize low latencies, low jitter, and the absence of latency spikes (tail latencies) as their main requirements. Similarly, so-called *scavenger* CCAs, e.g., Low Extra Delay Background Transport (LEDBAT) [438], rLEDBAT [439] (in active development by the IETF), NICE [440], and PCC Proteus [441], are designed to support large background transfers. These scavenger CCAs utilize unused spare bandwidth that is free but yield to competing flows to minimize interference. Due to characteristic traffic patterns of applications, unfairness between flows may actually increase the application-level performance. For example, in machine learning clusters, unfairness between flows may reduce the overall training time [442]. Hence, flow rate fairness metrics need to be interpreted and compared with caution since the underlying premise may not be fulfilled for all applications and CCAs.

The meaningfulness of flow rate fairness has been defended in [177]. The main argument is that flow rate fairness has been deployed in the Internet for a long time. Despite its shortcomings, flow rate fairness has successfully impeded congestion collapse while allowing flows to acquire some

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

share of the bandwidth, i.e., avoid starvation. Alternatives often require changes to network deployments, which is very challenging for the Internet even if introduced incrementally because of the heterogeneity of the Internet [443].

ACKNOWLEDGMENT

The authors are grateful to Dr. Asif Siddiqui of Prince Sultan University, Saudi Arabia, for discussions on network security.

REFERENCES

[1] M. Allman, V. Paxson, and E. Blanton, "TCP congestion control," IETF, RFC 5681, Sep. 2009. [Online]. Available: http://www.rfc-editor.org/rfc/rfc5681.txt

[2] K. R. Fall and W. R. Stevens, *TCP/IP Illustrated, Volume 1: The Protocols*. San Francisco, CA, USA: Addison-Wesley, 2011.

[3] A. Langley et al., "The QUIC transport protocol: Design and Internet-scale deployment," in *Proc. ACM SIGCOMM*, 2017, pp. 183–196.

[4] V. Jacobson, "Congestion avoidance and control," in *Proc. ACM Symp. Commun. Archit. Protocols (SIGCOMM)*, 1988, pp. 314–329.

[5] J. H. Saltzer, D. P. Reed, and D. D. Clark, "End-to-end arguments in system design," *ACM Trans. Comput. Syst.*, vol. 2, no. 4, pp. 277–288, 1984.

[6] M. S. Blumenthal and D. D. Clark, "Rethinking the design of the Internet: The end-to-end arguments vs. The brave new world," *ACM Trans. Internet Technol.*, vol. 1, no. 1, pp. 70–109, 2001.

[7] T. Moors, "A critical review of 'end-to-end arguments in system design,"' in *Proc. IEEE ICC*, vol. 2, 2002, pp. 1214–1219.

[8] L. Andrew et al., "Towards a common TCP evaluation suite," in *Proc. PFLDnet*, 2008, pp. 1–5.

[9] D. Hayes, D. Ros, L. L. Andrew, and S. Floyd, "Common TCP evaluation suite," Internet Eng. Task Force, Fremont, CA, USA, Jul. 2014. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-irtf-iccrg-tcpeval-01

[10] S. Floyd, "Metrics for the evaluation of congestion control mechanisms," IETF, RFC 5166, Mar. 2008. [Online]. Available: http://www.rfc-editor.org/rfc/rfc5166.txt

[11] F. Y. Yan et al., "Pantheon: The training ground for Internet congestion-control research," in *Proc. USENIX Annu. Techn. Conf.*, Jul. 2018, pp. 731–743.

[12] D. K. Mishra, P. Vankar, and M. P. Tahiliani, "TCP evaluation suite for NS-3," in *Proc. Workshop ns-3*, 2016, pp. 25–32.

[13] K. Nagori, M. Balachandran, A. Deepak, M. P. Tahiliani, and B. R. Chandavarkar, "Common TCP evaluation suite for ns-3: Design, implementation and open issues," in *Proc. Workshop ns-3*, 2017, pp. 9–16.

[14] D. Zarchy, R. Mittal, M. Schapira, and S. Shenker, "Axiomatizing congestion control," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 3, no. 2, pp. 33:1–33:33, Jun. 2019.

[15] C. L. Vielhaus. "Ccperf: Evaluating and benchmarking congestion control algorithms." [Online]. Available: https://gitlab.com/ccperf/ccperf

[16] D. Hong and T. Suda, "Congestion control and prevention in ATM networks," *IEEE Netw.*, vol. 5, no. 4, pp. 10–16, Jul. 1991.

[17] R. Jain, "Congestion control and traffic management in ATM networks: Recent advances and a survey," *Comput. Netw. ISDN Syst.*, vol. 28, no. 13, pp. 1723–1738, 1996.

[18] J. Widmer, R. Denda, and M. Mauve, "A survey on TCP-friendly congestion control," *IEEE Netw.*, vol. 15, no. 3, pp. 28–37, May/Jun. 2001.

[19] S. Ryu, C. Rump, and C. Qiao, "Advances in Internet congestion control," *IEEE Commun. Surveys Tuts.*, vol. 5, no. 1, pp. 28–39, 3rd Quart., 2003.

[20] A. Afanasyev, N. Tilley, P. Reiher, and L. Kleinrock, "Host-to-host congestion control for TCP," *IEEE Commun. Surveys Tuts.*, vol. 12, no. 3, pp. 304–342, 3rd Quart., 2010.

[21] S.-U. Lar and X. Liao, "An initiative for a classified bibliography on TCP/IP congestion control," *J. Netw. Comput. Appl.*, vol. 36, no. 1, pp. 126–133, 2013.

[22] A. Bohloulzadeh and M. Rajaei, "A survey on congestion control protocols in wireless sensor networks," *Int. J. Wireless Inf. Netw.*, vol. 27, no. 3, pp. 365–384, 2020.

[23] A. Ghaffari, "Congestion control mechanisms in wireless sensor networks: A survey," *J. Netw. Comput. Appl.*, vol. 52, pp. 101–115, Jun. 2015.

[24] M. A. Kafi, D. Djenouri, J. Ben-Othman, and N. Badache, "Congestion control protocols in wireless sensor networks: A survey," *IEEE Commun. Surveys Tuts.*, vol. 16, no. 3, pp. 1369–1390, 3rd Quart., 2014.

[25] C. Lim, "A survey on congestion control for RPL-based wireless sensor networks," *Sensors*, vol. 19, no. 11, p. 2567, 2019.

[26] V. Michopoulos, L. Guan, G. Oikonomou, and I. Phillips, "A comparative study of congestion control algorithms in IPv6 wireless sensor networks," in *Proc. IEEE Int. Conf. Distrib. Comput. Sensor Syst. Workshops (DCOSS)*, 2011, pp. 1–6.

[27] C. Sergiou, P. Antoniou, and V. Vassiliou, "A comprehensive survey of congestion control protocols in wireless sensor networks," *IEEE Commun. Surveys Tuts.*, vol. 16, no. 4, pp. 1839–1859, 4th Quart., 2014.

[28] S. A. Shah, B. Nazir, and I. A. Khan, "Congestion control algorithms in wireless sensor networks: Trends and opportunities," *J. King Saud Univ. Comput. Inf. Sci.*, vol. 29, no. 3, pp. 236–245, 2017.

[29] S. L. Yadav and R. Ujjwal, "Mitigating congestion in wireless sensor networks through clustering and queue assistance: A survey," *J. Intell. Manuf.*, vol. 32, no. 8, pp. 2083–2098, 2021.

[30] D. Kanellopoulos, "Congestion control for MANETs: An overview," *ICT Exp.*, vol. 5, no. 2, pp. 77–83, 2019.

[31] D. N. Kanellopoulos, "Congestion control for NDN-based MANETs: Recent advances, enabling technologies, and open challenges," *J. Org. End User Comput.*, vol. 33, no. 5, pp. 111–134, 2021.

[32] C. Lochert, B. Scheuermann, and M. Mauve, "A survey on congestion control for mobile ad hoc networks," *Wireless Commun. Mobile Comput.*, vol. 7, no. 5, pp. 655–676, 2007.

[33] A. Balador et al., "Survey on decentralized congestion control methods for vehicular communication," *Veh. Commun.*, vol. 33, Jan. 2021, Art. no. 100394.

[34] A. Hemmati, M. Zarei, and A. M. Rahmani, "A systematic review of congestion control in Internet of Vehicles and vehicular ad hoc networks: Techniques, challenges, and open issues," *Int. J. Commun. Syst.*, vol. 37, no. 1, 2024, Art. no. e5625.

[35] B. Shabir, M. A. Khan, A. U. Rahman, A. W. Malik, and A. Wahid, "Congestion avoidance in vehicular networks: A contemporary survey," *IEEE Access*, vol. 7, pp. 173196–173215, 2019.

[36] R. Sharma, C. Lamba, and V. Rathore, "Congestion control mechanisms to avoid congestion in VANET: A comparative review," in *Artificial Intelligence and Speech Technology*. Boca Raton, FL, USA: CRC Press, 2021, pp. 453–462.

[37] V. Mekathoti and B. Nithya, "A survey on congestion control algorithms of wireless body area network," in *Proc. Adv. Comput. Netw. Commun. (LNEE)*, vol. 735, 2021, pp. 373–387.

[38] A. P. Silva, S. Burleigh, C. M. Hirata, and K. Obraczka, "A survey on congestion control for delay and disruption tolerant networks," *Ad Hoc Netw.*, vol. 25, pp. 480–494, Feb. 2015.

[39] A. P. Silva, S. Burleigh, C. M. Hirata, and K. Obraczka, "Congestion control in disruption-tolerant networks: A comparative study for interplanetary and terrestrial networking applications," *Ad Hoc Netw.*, vol. 44, pp. 1–18, Jul. 2016.

[40] J. Lorincz, Z. Klarin, and J. Ožegović, "A comprehensive overview of TCP congestion control in 5G networks: Research challenges and future perspectives," *Sensors*, vol. 21, no. 13, 2021, Art. no. 4510.

[41] F. Rad, M. Reshadi, and A. Khademzadeh, "A survey and taxonomy of congestion control mechanisms in wireless network on chip," *J. Syst. Archit.*, vol. 108, Sep. 2020, Art. no. 101807.

[42] V. Kushwaha and R. Gupta, "Congestion control for high-speed wired network: A systematic literature review," *J. Netw. Comput. Appl.*, vol. 45, pp. 62–78, Oct. 2014.

[43] G. Cui, H. Wang, Z. Fan, and P. Li, "NDN congestion control based on fuzzy comprehensive evaluation algorithm," *J. Phys. Conf.*, vol. 1883, no. 1, 2021, Art. no. 12008.

[44] B. Zhang, "Review of congestion control methods in named data network," *Int. J. Comput. Sci. Inf. Techn.*, vol. 3, no. 1, pp. 55–62, 2024.

[45] S. Huang, D. Dong, and W. Bai, "Congestion control in high-speed lossless data center networks: A survey," *Future Gener. Comput. Syst.*, vol. 89, pp. 360–374, Dec. 2018.

[46] T. A. N. Nguyen, S. Gangadhar, and J. P. Sterbenz, "Performance evaluation of TCP congestion control algorithms in data center networks," in *Proc. ACM 11th Int. Conf. Future Internet Techn.*, 2016, pp. 21–28.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

[47] C. Xu, J. Zhao, and G.-M. Muntean, "Congestion control design for multipath transport protocols: A survey," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 4, pp. 2948–2969, 4th Quart., 2016.

[48] A. Matrawy and I. Lambadaris, "A survey of congestion control schemes for multicast video applications," *IEEE Commun. Surveys Tuts.*, vol. 5, no. 2, pp. 22–31, 4th Quart., 2003.

[49] B. Soelistijanto and M. P. Howarth, "Transfer reliability and congestion control strategies in opportunistic networks: A survey," *IEEE Commun. Surveys Tuts.*, vol. 16, no. 1, pp. 538–555, 1st Quart., 2013.

[50] P. Anitha, H. Vimala, and J. Shreyas, "Comprehensive review on congestion detection, alleviation, and control for IoT networks," *J. Netw. Comput. Appl.*, vol. 221, Jan. 2023, Art. no. 103749.

[51] V. K. Jain, A. P. Mazumdar, P. Faruki, and M. C. Govil, "Congestion control in Internet of Things: Classification, challenges, and future directions," *Sustain. Comput. Informat. Syst.*, vol. 35, Sep. 2022, Art. no. 100678.

[52] A. R. Andrade-Zambrano, J. P. A. León, M. E. Morocho-Cayamcela, L. L. Cárdenas, and L. J. de la Cruz Llopis, "A reinforcement learning congestion control algorithm for smart grid networks," *IEEE Access*, vol. 12, pp. 75072–75092, 2024.

[53] C. Döpmann, F. Fiedler, S. Lucia, and F. Tschorsch, "Optimization-based predictive congestion control for the Tor network: Opportunities and challenges," *ACM Trans. Internet Technol.*, vol. 22, no. 4, pp. 97:1–97:30, Nov. 2022.

[54] C. Callegari, S. Giordano, M. Pagano, and T. Pepe, "A survey of congestion control mechanisms in Linux TCP," in *Proc. Int. Conf. Distrib. Comput. Commun. Netw.*, 2013, pp. 28–42.

[55] G. A. Abed, M. Ismail, and K. Jumari, "A survey on performance of congestion control mechanisms for standard TCP versions," *Aust. J. Basic Appl. Sci.*, vol. 5, no. 12, pp. 1345–1352, 2011.

[56] R. Al-Saadi, G. Armitage, J. But, and P. Branch, "A survey of delay-based and hybrid TCP congestion control algorithms," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 4, pp. 3609–3638, 4th Quart., 2019.

[57] Q.-V. Pham and W.-J. Hwang, "Network utility maximization-based congestion control over wireless networks: A survey and potential directives," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 2, pp. 1173–1200, 2nd Quart., 2017.

[58] G. Hasegawa and M. Murata, "Survey on fairness issues in TCP congestion control mechanisms," *IEICE Trans. Commun.*, vol. 84, no. 6, pp. 1461–1472, 2001.

[59] B. Turkovic, F. A. Kuipers, and S. Uhlig, "Fifty shades of congestion control: A performance and interactions evaluation," 2019, *arXiv:1903.03852*.

[60] H. Huang, X. Zhu, J. Bi, W. Cao, and X. Zhang, "Machine learning for broad-sensed Internet congestion control and avoidance: A comprehensive survey," *IEEE Access*, vol. 9, pp. 31525–31545, 2021.

[61] H. Jiang et al., "When machine learning meets congestion control: A survey and comparison," *Comput. Netw.*, vol. 192, Jun. 2021, Art. no. 108033.

[62] L. Li, H. Lin, J. Wan, Z. Ma, and H. Wang, "MF-TCPV: A machine learning and fuzzy comprehensive evaluation-based framework for traffic congestion prediction and visualization," *IEEE Access*, vol. 8, pp. 227113–227125, 2020.

[63] M. Polese, F. Chiariotti, E. Bonetto, F. Rigotto, A. Zanella, and M. Zorzi, "A survey on recent advances in transport layer protocols," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 4, pp. 3584–3608, 4th Quart., 2019.

[64] Z. Tafa and V. Milutinovic, "Machine learning in congestion control: A survey on selected algorithms and a new roadmap to their implementation," 2021, *arXiv:2112.15522*.

[65] F. Tang, B. Mao, Y. Kawamoto, and N. Kato, "Survey on machine learning for intelligent end-to-end communication toward 6G: From network access, routing to traffic control and streaming adaption," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 3, pp. 1578–1598, 3rd Quart., 2021.

[66] W. Wei, H. Gu, and B. Li, "Congestion control: A renaissance with machine learning," *IEEE Netw.*, vol. 35, no. 4, pp. 262–269, Jul./Aug. 2021.

[67] T. Yassin and O. Ali, "Using machine learning to control congestion in SDN: A review," in *Proc. Int. Conf. Emerg. Trends Appl. Artif. Intell.*, 2023, pp. 395–403.

[68] T. Zhang and S. Mao, "Machine learning for end-to-end congestion control," *IEEE Commun. Mag.*, vol. 58, no. 6, pp. 52–57, Jun. 2020.

[69] Y.-J. Lin, R. E. Newman, and H. Latchman, "A new TCP and UDP network benchmark suite," in *Proc. ACM Spring Simulat. Multiconf.*, 2007, pp. 211–217.

[70] R. Lübben and M. Fidler, "A benchmark for the performance of time-varying closed-loop flow control with application to TCP," 2016, *arXiv:1609.00653*.

[71] J. Nagle, "Congestion control in IP/TCP Internetworks," IETF, RFC 896, Jan. 1984.

[72] T. Henderson, S. Floyd, A. Gurtov, and Y. Nishida, "The NewReno modification to TCP's fast recovery algorithm," IETF, RFC 6582, Apr. 2012. [Online]. Available: http://www.rfc-editor.org/rfc/rfc6582.txt

[73] J. Pahdye and S. Floyd, "On inferring TCP behavior," in *Proc. ACM SIGCOMM*, San Diego, CA, USA, 2001, pp. 287–298.

[74] A. Medina, M. Allman, and S. Floyd, "Measuring the evolution of transport protocols in the Internet," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 35, no. 2, pp. 37–52, Apr. 2005.

[75] P. Yang, J. Shao, W. Luo, L. Xu, J. Deogun, and Y. Lu, "TCP congestion avoidance algorithm identification," *IEEE/ACM Trans. Netw.*, vol. 22, no. 4, pp. 1311–1324, Aug. 2014.

[76] S. Ha, I. Rhee, and L. Xu, "CUBIC: A new TCP-friendly high-speed TCP variant," *SIGOPS Oper. Syst. Rev.*, vol. 42, no. 5, pp. 64–74, Jul. 2008.

[77] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-based congestion control," *ACM Queue*, vol. 14, no. 5, pp. 20–53, Sep./Oct. 2016.

[78] Y.-J. Song, G.-H. Kim, I. Mahmud, W.-K. Seo, and Y.-Z. Cho, "Understanding of BBRv2: Evaluation and comparison with BBRv1 congestion control algorithm," *IEEE Access*, vol. 9, pp. 37131–37145, 2021.

[79] A. Mishra, X. Sun, A. Jain, S. Pande, R. Joshi, and B. Leong, "The great Internet TCP congestion control census," in *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 3, Dec. 2019, pp. 1–24.

[80] F. Kelly, A. Maulloo, and D. Tan, "Rate control for communication networks: Shadow prices, proportional fairness and stability," *J. Oper. Res. Soc.*, vol. 49, pp. 237–252, Mar. 1998.

[81] R. Srikant, *The Mathematics of Internet Congestion Control*. Boston, MA, USA: Birkhäuser, Jan. 2004.

[82] D. Clark, "The design philosophy of the DARPA Internet protocols," *SIGCOMM Comput. Commun. Rev.*, vol. 18, no. 4, pp. 106–114, Aug. 1988.

[83] M. Zhang, M. Dusi, W. John, and C. Chen, "Analysis of UDP traffic usage on Internet backbone links," in *Proc. 9th Ann. Int. Symp. Appl. Internet*, 2009, pp. 280–281.

[84] D. Katabi, M. Handley, and C. Rohrs, "Congestion control for high bandwidth-delay product networks," *SIGCOMM Comput. Commun. Rev.*, vol. 32, no. 4, pp. 89–102, Aug. 2002.

[85] S. Floyd and V. Jacobson, "Random early detection gateways for congestion avoidance," *IEEE/ACM Trans. Netw.*, vol. 1, no. 4, pp. 397–413, Aug. 1993.

[86] R. Adams, "Active queue management: A survey," *IEEE Commun. Surveys Tuts.*, vol. 15, no. 3, pp. 1425–1476, 3rd Quart., 2013.

[87] K. Ramakrishnan, S. Floyd, and D. Black, "The addition of explicit congestion notification (ECN) to IP," IETF, RFC 3168, Sep. 2001. [Online]. Available: http://www.rfc-editor.org/rfc/rfc3168.txt

[88] K. D. Schepper and B. Briscoe, "The explicit congestion notification (ECN) protocol for low latency, low loss, and scalable throughput (L4S)," IETF, RFC 9331, Jan. 2023. [Online]. Available: https://www.rfc-editor.org/info/rfc9331

[89] K. D. Schepper, O. Tilmans, B. Briscoe, and V. Goel, "Prague congestion control," Internet Eng. Task Force, Fremont, CA, USA, Jul. 2024. [Online]. Available: https://datatracker.ietf.org/doc/draft-briscoe-iccrg-prague-congestion-control/04/

[90] A. Kuzmanovic, "The power of explicit congestion notification," in *Proc. ACM SIGCOMM*, Philadelphia, PA, USA, 2005, pp. 61–72.

[91] P. Karn and C. Partridge, "Improving round-trip time estimates in reliable transport protocols," in *Proc. ACM Workshop Front. Comput. Commun. Technol.*, 1987, pp. 2–7.

[92] Y. Cheng, N. Cardwell, S. H. Yeganeh, and V. Jacobson, "Delivery rate estimation," Internet Eng. Task Force, Fremont, CA, USA, Mar. 2022. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-cheng-iccrg-delivery-rate-estimation-02

[93] G. F. Franklin, J. D. Powell, A. Emami-Naeini, and J. D. Powell, *Feedback Control of Dynamic Systems*, vol. 4. Upper Saddle River, NJ, USA: Prentice Hall, 2002.

[94] R. C. Dorf and R. H. Bishop, *Modern Control Systems*. Hoboken, NJ, USA: Prentice Hall, 2008.

[95] A. Aggarwal, S. Savage, and T. Anderson, "Understanding the performance of TCP pacing," in *Proc. IEEE INFOCOM*, Jan. 2000, pp. 1157–1165.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

[96] M. Welzl, W. Eddy, V. Goel, and M. Tüxen, "Pacing in transport protocols," Internet Eng. Task Force, Fremont, CA, USA, Oct. 2024. [Online]. Available: https://datatracker.ietf.org/doc/draft-welzl-iccrg-pacing/01/

[97] S. Ha and I. Rhee, "Taming the elephants: New TCP slow start," *Comput. Netw.*, vol. 55, no. 9, pp. 2092–2110, Jun. 2011.

[98] N. Dukkipati, M. Mathis, Y. Cheng, and M. Ghobadi, "Proportional rate reduction for TCP," in *Proc. ACM SIGCOMM Internet Meas. Conf.*, 2011, pp. 155–170.

[99] ESnet. "iperf." Accessed: Jul. 18, 2024. [Online]. Available: https://software.es.net/iperf/

[100] T. R. Henderson, M. Lacage, G. F. Riley, C. Dowell, and J. Kopena, "Network simulations with the ns-3 simulator," *SIGCOMM Demonstration*, vol. 14, no. 14, p. 527, 2008.

[101] A. Varga, "Omnet++," in *Modeling and Tools for Network Simulation*, K. Wehrle, M. Güneş, and J. Gross, Eds. Heidelberg, Germany: Springer, 2010, pp. 35–59.

[102] J. Edge. "Namespaces in operation, part 7: Network namespaces." Accessed: Jul. 18, 2024. [Online]. Available: https://lwn.net/Articles/580893/

[103] Linux Foundation. "IProute2." Accessed: Jul. 18, 2024. [Online]. Available: https://wiki.linuxfoundation.org/networking/iproute2

[104] S. Hemminger et al., "Network emulation with NetEm," in *Proc. Linux Conf AU*, vol. 5, 2005, p. 13.

[105] M. Dischinger, A. Haeberlen, K. P. Gummadi, and S. Saroiu, "Characterizing residential broadband networks," in *Proc. ACM Internet Meas. Conf.*, 2007, pp. 43–56.

[106] D. G. T. Almes, J. Mahdavi, M. Mathis, and D. V. Paxson, "Framework for IP performance metrics," IETF, RFC 2330, May 1998. [Online]. Available: https://www.rfc-editor.org/info/rfc2330

[107] J. Ishac and P. Chimento, "Defining network capacity," IETF, RFC 5136, Feb. 2008. [Online]. Available: https://www.rfc-editor.org/info/rfc5136

[108] J. Kurose and K. Ross, *Computer Networks: A Top Down Approach Featuring the Internet*, 8th Ed. Boston, MA, USA: Pearson, 2021.

[109] J. Gettys, "BufferBloat: Dark buffers in the Internet," *IEEE Internet Comput.*, vol. 15, no. 3, pp. 96–96, May 2011.

[110] S. Floyd and K. Fall, "Promoting the use of end-to-end congestion control in the Internet," *IEEE/ACM Trans. Netw.*, vol. 7, no. 4, pp. 458–472, Aug. 1999.

[111] L. Kleinrock, "Internet congestion control using the power metric: Keep the pipe just full, but no fuller," *Ad Hoc Netw.*, vol. 80, pp. 142–157, Nov. 2018.

[112] A. Giessler, J. D. Haenle, A. König, and E. Pade, "Free buffer allocation—An investigation by simulation," *Comput. Netw.*, vol. 2, no. 3, pp. 191–208, 1978.

[113] L. Guo and I. Matta, "The war between mice and elephants," in *Proc. IEEE Int. Conf. Netw. Protocols (ICNP)*, Nov. 2001, pp. 180–188.

[114] S. Shenker, "Fundamental design issues for the future Internet," *IEEE J. Sel. Areas Commun.*, vol. 13, no. 7, pp. 1176–1188, Sep. 1995.

[115] A. Baiocchi, A. P. Castellani, and F. Vacirca, "YeAH-TCP: Yet another highspeed TCP," in *Proc. 5th Int. Workshop Protocols Fast Long-Distance Netw. (PFLDnet)*, 2007, pp. 37–42.

[116] S. Mascolo, C. Casetti, M. Gerla, M. Y. Sanadidi, and R. Wang, "TCP Westwood: Bandwidth estimation for enhanced transport over wireless links," in *Proc. ACM Mobicom*, 2001, pp. 287–297.

[117] J. Postel. "Transmission control protocol." Sep. 1981. [Online]. Available: http://www.rfc-editor.org/rfc/rfc793.txt

[118] L. S. Brakmo, S. W. O'Malley, and L. L. Peterson, "TCP Vegas: New techniques for congestion detection and avoidance," *SIGCOMM Comput. Commun. Rev.*, vol. 24, no. 4, pp. 24–35, Oct. 1994.

[119] G. Kahe and A. H. Jahangir, "A self-tuning controller for queuing delay regulation in TCP/AQM networks," *Telecommun. Syst.*, vol. 71, no. 2, pp. 215–229, 2019.

[120] C. M. Demichelis and P. Chimento, "IP packet delay variation metric for IP performance metrics (IPPM)," IETF, RFC 3393, Nov. 2002. [Online]. Available: https://www.rfc-editor.org/info/rfc3393

[121] B. Claise and A. Morton, "Packet delay variation applicability statement," IETF, RFC 5481, Mar. 2009. [Online]. Available: https://www.rfc-editor.org/info/rfc5481

[122] K. Nichols, V. Jacobson, and K. Poduri, "An expedited forwarding PHB," IETF, RFC 2598, Jun. 1999. [Online]. Available: https://www.rfc-editor.org/info/rfc2598

[123] S. Poretsky, S. Erramilli, J. Perser, and S. Khurana, "Terminology for benchmarking network-layer traffic control mechanisms," IETF, RFC 4689, Oct. 2006. [Online]. Available: https://www.rfc-editor.org/info/rfc4689

[124] H. Schulzrinne, S. L. Casner, R. Frederick, and V. Jacobson, "RTP: A transport protocol for real-time applications," IETF, RFC 3550, Jul. 2003. [Online]. Available: https://www.rfc-editor.org/info/rfc3550

[125] G. Almes, S. Kalidindi, M. J. Zekauskas, and A. Morton, "A one-way loss metric for IP performance metrics (IPPM)," IETF, RFC 7680, Jan. 2016. [Online]. Available: https://www.rfc-editor.org/info/rfc7680

[126] J. Martin, A. Nilsson, and I. Rhee, "Delay-based congestion avoidance for TCP," *IEEE/ACM Trans. Netw.*, vol. 11, no. 3, pp. 356–369, Jun. 2003.

[127] S. Floyd, M. Handley, J. Padhye, and J. Widmer, "Equation-based congestion control for unicast applications," *SIGCOMM Comput. Commun. Rev.*, vol. 30, no. 4, pp. 43–56, 2000.

[128] S. Floyd, M. Handley, J. Padhye, and J. Widmer, "TCP friendly rate control (TFRC): Protocol specification," IETF, RFC 5348, Sep. 2008. [Online]. Available: http://www.rfc-editor.org/rfc/rfc5348.txt

[129] R. Ravikanth and R. Koodli, "One-way loss pattern sample metrics," IETF, RFC 3357, Aug. 2002. [Online]. Available: https://www.rfc-editor.org/info/rfc3357

[130] L. Massoulie and J. Roberts, "Bandwidth sharing: Objectives and algorithms," *IEEE/ACM Trans. Netw.*, vol. 10, no. 3, pp. 320–328, Jun. 2002.

[131] N. Dukkipati and N. McKeown, "Why flow-completion time is the right metric for congestion control," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 36, no. 1, pp. 59–62, Jan. 2006.

[132] S. Floyd, M. Handley, and J. Padhye, "A comparison of equation-based and AIMD congestion control," in *Proc. Workshop Model. Congest. Control Algorithms*, 2000, pp. 1–12.

[133] D. Bansal, H. Balakrishnan, S. Floyd, and S. Shenker, "Dynamic behavior of slowly-responsive congestion control algorithms," *SIGCOMM Comput. Commun. Rev.*, vol. 31, no. 4, pp. 263–274, Aug. 2001.

[134] Y. R. Yang, N. S. Kim, and S. S. Lam, "Transient behaviors of TCP-friendly congestion control protocols," in *Proc. IEEE INFOCOM*, vol. 3, Apr. 2001, pp. 1716–1725.

[135] D. X. Wei, C. Jin, S. H. Low, and S. Hegde, "FAST TCP: Motivation, architecture, algorithms, performance," *IEEE/ACM Trans. Netw.*, vol. 14, no. 6, pp. 1246–1259, Dec. 2006.

[136] D. X. Wei, P. Cao, and S. H. Low. "Time for a TCP benchmark suite?" 2008. [Online]. Available: https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.60.3550&rep=rep1&type=pdf

[137] J. Lin, L. Cui, Y. Zhang, F. P. Tso, and Q. Guan, "Extensive evaluation on the performance and behavior of TCP congestion control protocols under varied network scenarios," *Comput. Netw.*, vol. 163, Nov. 2019, Art. no. 106872.

[138] M. Dong, Q. Li, D. Zarchy, P. B. Godfrey, and M. Schapira, "PCC: Re-architecting congestion control for consistent high performance," in *Proc. USENIX NSDI*, May 2015, pp. 395–408.

[139] Y.-T. Li, D. Leith, and R. N. Shorten, "Experimental evaluation of TCP protocols for high-speed networks," *IEEE/ACM Trans. Netw.*, vol. 15, no. 5, pp. 1109–1122, Oct. 2007.

[140] S. Arslan, S. Renganathan, and B. Spang, "Green with envy: Unfair congestion control algorithms can be more energy efficient," in *Proc. 22nd ACM Workshop Hot Topics Netw.*, 2023, pp. 220–228.

[141] D. M. Chiu and R. Jain, "Analysis of the increase and decrease algorithms for congestion avoidance in computer networks," *Comput. Netw. ISDN Syst.*, vol. 17, no. 1, pp. 1–14, Jun. 1989.

[142] R. Jain, D. M. Chiu, and H. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," Eastern Res. Lab., Hudson, MA, USA, Rep. DEC-TR-301, Sep. 1984.

[143] R. Ware, M. K. Mukerjee, S. Seshan, and J. Sherry, "Beyond jain's fairness index: Setting the bar for the deployment of congestion control algorithms," in *Proc. 18th ACM Workshop Hot Topics Netw.*, 2019, pp. 17–24.

[144] A. Zapletal and F. Kuipers, "Slowdown as a metric for congestion control fairness," in *Proc. 22nd ACM Workshop Hot Topics Netw.*, 2023, pp. 205–212.

[145] Y. Zhang, S.-R. Kang, and D. Loguinov, "Delayed stability and performance of distributed congestion control," in *Proc. ACM SIGCOMM*, 2004, pp. 307–318.

[146] R. Shorten, D. Leith, J. Foy, and R. Kilduff, "Analysis and design of AIMD congestion control algorithms in communication networks," *Automatica*, vol. 41, no. 4, pp. 725–730, Apr. 2005.

[147] R. Shorten, F. Wirth, and D. Leith, "A positive systems model of TCP-like congestion control: Asymptotic results," *IEEE/ACM Trans. Netw.*, vol. 14, no. 3, pp. 616–629, Jun. 2006.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

[148] T. Lukaseder, L. Bradatsch, B. Erb, R. W. Van Der Heijden, and F. Kargl, "A comparison of TCP congestion control algorithms in 10 G networks," in *Proc. IEEE 41st Conf. Local Comput. Netw. (LCN)*, 2016, pp. 706–714.

[149] M. Dong et al., "PCC Vivace: Online-learning congestion control," in *Proc. USENIX NSDI*, Apr. 2018, pp. 343–356.

[150] S. Floyd and V. Jacobson, "Traffic phase effects in packet-switched gateways," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 21, no. 2, pp. 26–42, Apr. 1991.

[151] C. L. Vielhaus. "ccperf.net." Accessed: Jul. 18, 2024. [Online]. Available: https://ccperf.net

[152] G. Appenzeller, I. Keslassy, and N. McKeown, "Sizing router buffers," in *Proc. ACM SIGCOMM*, 2004, pp. 281–292.

[153] H. Obata, K. Tamehiro, and K. Ishida, "Experimental evaluation of TCP-STAR for satellite Internet over WINDS," in *Proc. 10th IEEE Int. Symp. Auton. Decentralized Syst.*, 2011, pp. 605–610.

[154] ns3 Network Simulator. "TCP models in ns-3." Accessed: Jul. 31, 2023. [Online]. Available: https://www.nsnam.org/docs/models/html/tcp.html #validation

[155] N. Cardwell, I. Swett, and J. Beshay, "BBR congestion control," Internet Eng. Task Force, Fremont, CA, USA, Jul. 2024. [Online]. Available: https://datatracker.ietf.org/doc/draft-cardwell-ccwg-bbr/00/

[156] M. Schapira and K. Winstein, "Congestion-control throwdown," in *Proc. ACM HotNets*, 2017, pp. 122–128.

[157] A. Veres and M. Boda, "The chaotic nature of TCP congestion control," in *Proc. IEEE INFOCOM*, vol. 3, 2000, pp. 1715–1723.

[158] J. Padhye, V. Firoiu, D. Towsley, and J. Kurose, "Modeling TCP throughput: A simple model and its empirical validation," *SIGCOMM Comput. Commun. Rev.*, vol. 28, no. 4, pp. 303–314, Oct. 1998.

[159] M. Mathis, J. Semke, J. Mahdavi, and T. Ott, "The macroscopic behavior of the TCP congestion avoidance algorithm," *SIGCOMM Comput. Commun. Rev.*, vol. 27, no. 3, pp. 67–82, Jul. 1997.

[160] S. H. Low, L. Peterson, and L. Wang, "Understanding TCP Vegas: A duality model," in *Proc. ACM SIGMETRICS*, 2001, pp. 226–235.

[161] T. J. Ott, J. H. B. Kemperman, and M. Mathis. "The stationary behavior of ideal TCP congestion avoidance." 1996. [Online]. Available: https://www.teunisott.com/Papers/TCP_Paradigm/TCPwindow.pdf

[162] T. V. Lakshman and U. Madhow, "The performance of TCP/IP for networks with high bandwidth-delay products and random loss," *IEEE/ACM Trans. Netw.*, vol. 5, no. 3, pp. 336–350, Jun. 1997.

[163] S. Floyd, "HighSpeed TCP for large congestion windows," IETF, RFC 3649, Dec. 2003. [Online]. Available: http://www.rfc-editor.org/rfc/rfc3649.txt

[164] D. Leith and R. Shorten, "H-TCP: TCP for high-speed and long-distance networks," in *Proc. PFLDnet*, 2004, pp. 1–16.

[165] L. Xu, K. Harfoush, and I. Rhee, "Binary increase congestion control (BIC) for fast long-distance networks," in *Proc. IEEE INFOCOM*, vol. 4, 2004, pp. 2514–2524.

[166] T. Kelly, "Scalable TCP: Improving performance in highspeed wide area networks," *SIGCOMM Comput. Commun. Rev.*, vol. 33, no. 2, pp. 83–91, Apr. 2003.

[167] K. Tan, J. Song, Q. Zhang, and M. Sridharan, "A compound TCP approach for high-speed and long distance networks," in *Proc. IEEE INFOCOM*, 2006, pp. 1–12.

[168] C. B. Samios and M. K. Vernon, "Modeling the throughput of TCP Vegas," in *Proc. ACM SIGMETRICS*, San Diego, CA, USA, 2003, pp. 71–81.

[169] S. Liu, T. Basar, and R. Srikant, "TCP-Illinois: A loss- and delay-based congestion control algorithm for high-speed networks," *Perform. Eval.*, vol. 65, no. 6, pp. 417–440, 2008.

[170] V. Arun and H. Balakrishnan, "COPA: Practical delay-based congestion control for the Internet," in *Proc. Appl. Netw. Res. Workshop*, 2018, pp. 19–19.

[171] K. Winstein and H. Balakrishnan, "TCP Ex Machina: Computer-generated congestion control," in *Proc. ACM SIGCOMM*, 2013, pp. 123–134.

[172] R. Mittal et al., "TIMELY: RTT-based congestion control for the datacenter," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 45, no. 4, pp. 537–550, Aug. 2015.

[173] L. De Cicco, G. Carlucci, and S. Mascolo, "Experimental investigation of the Google congestion control for real-time flows," in *Proc. ACM SIGCOMM Workshop Future Human-Centric Multimedia Netw. (FhMN)*, 2013, pp. 21–26.

[174] S. Floyd, "Connections with multiple congested gateways in packet-switched networks—Part 1: One-way traffic," *SIGCOMM Comput. Commun. Rev.*, vol. 21, no. 5, pp. 30–47, Oct. 1991.

[175] V. Misra, W.-B. Gong, and D. Towsley, "Fluid-based analysis of a network of AQM routers supporting TCP flows with an application to RED," in *Proc. ACM SIGCOMM*, Stockholm, Sweden, 2000, pp. 151–160.

[176] J. Wang, J. Wen, J. Zhang, and Y. Han, "TCP-FIT: An improved TCP congestion control algorithm and its performance," in *Proc. IEEE INFOCOM*, 2011, pp. 2894–2902.

[177] S. Floyd and M. Allman, "Comments on the usefulness of simple best-effort traffic," IETF, RFC 5290, Jul. 2008. [Online]. Available: https://www.rfc-editor.org/info/rfc5290

[178] U. Hengartner, J. Bolliger, and T. Gross, "TCP Vegas revisited," in *Proc. IEEE INFOCOM*, vol. 3, 2000, pp. 1546–1555.

[179] K. Srijith, L. Jacob, and A. Ananda, "TCP Vegas-A: Solving the fairness and rerouting issues of TCP Vegas," in *Proc. IEEE Int. Perform. Comput. Commun. Conf.*, 2003, pp. 309–316.

[180] D. Zeynali, E. N. Weyulu, S. Fathalli, B. Chandrasekaran, and A. Feldmann, "Promises and potential of BBRv3," in *Proc. Int. Conf. Passive Active Netw. Meas.*, 2024, pp. 249–272.

[181] S. Wang et al., "PUDICA: Toward near-zero queuing delay in congestion control for cloud gaming," in *Proc. 21st USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, Apr. 2024, pp. 113–129.

[182] K. Kaneko, T. Fujikawa, Z. Su, and J. Katto, "TCP-Fusion: A hybrid congestion control algorithm for high-speed networks," in *Proc. PFLDnet*, 2007, pp. 31–36.

[183] F. Han et al., "ETC: An elastic transmission control using end-to-end available bandwidth perception," in *Proc. USENIX Annu. Technol. Conf. (USENIX ATC)*, Jul. 2024, pp. 265–284.

[184] S. Ma, J. Jiang, W. Wang, and B. Li, "Fairness of congestion-based congestion control: Experimental evaluation and analysis," 2017, *arxiv:1706.09115*.

[185] C. Villamizar and C. Song, "High performance TCP in ANSNET," *SIGCOMM Comput. Commun. Rev.*, vol. 24, no. 5, pp. 45–60, Oct. 1994.

[186] S. Shenker, L. Zhang, and D. Clark, "Some observations on the dynamics of a congestion control algorithm," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 20, pp. 30–39, Oct. 1990.

[187] E. S. Hashem, "Analysis of random drop for gateway congestion control," Defense Techn. Inf. Center, Fort Belvoir, VA, USA, Rep. ADA218737, 1989.

[188] R. Morris, "TCP behavior with many flows," in *Proc. IEEE Int. Conf. Netw. Protocols (ICNP)*, 1997, pp. 205–211.

[189] A. A. Philip, R. Ware, R. Athapathu, J. Sherry, and V. Sekar, "Revisiting TCP congestion control throughput models & fairness properties at scale," in *Proc. 21st ACM Internet Meas. Conf. (IMC)*, 2021, pp. 96–103.

[190] L. Qiu, Y. Zhang, and S. Keshav, "On individual and aggregate TCP performance," in *Proc. 7th IEEE Int. Conf. Netw. Protocols (ICNP)*, 1999, pp. 203–212.

[191] V. Padmanabhan, "TCP fast start: A technique for speeding up Web transfers," in *Proc. IEEE GLOBECOM*, Nov. 1998, pp. 1–8.

[192] J. D. Touch, M. Welzl, and S. Islam, "TCP control block interdependence," IETF, RFC 9040, Jul. 2021. [Online]. Available: https://www.rfc-editor.org/info/rfc9040

[193] T. Flach et al., "Reducing Web latency: The virtue of gentle aggression," in *Proc. ACM SIGCOMM*, 2013, pp. 159–170.

[194] M. Hock, R. Bless, and M. Zitterbart, "Experimental evaluation of BBR congestion control," in *Proc. IEEE Int. Conf. Netw. Protocols (ICNP)*, Oct. 2017, pp. 1–10.

[195] N. Cardwell, S. Savage, and T. Anderson, "Modeling TCP latency," in *Proc. IEEE INFOCOM*, vol. 3, 2000, pp. 1742–1751.

[196] I. F. Akyildiz, G. Morabito, and S. Palazzo, "TCP-Peach: A new congestion control scheme for satellite IP networks," *IEEE/ACM Trans. Netw.*, vol. 9, no. 3, pp. 307–321, Jun. 2001.

[197] M. Welzl, D. Papadimitriou, B. Briscoe, M. Scharf, and M. Welzl, "Open research issues in Internet congestion control," IETF, RFC 6077, Feb. 2011. [Online]. Available: https://rfc-editor.org/rfc/rfc6077.txt

[198] T. Flach et al., "An Internet-wide analysis of traffic policing," in *Proc. ACM SIGCOMM Conf.*, 2016, pp. 468–482.

[199] R. Wang, G. Pau, K. Yamada, M. Sanadidi, and M. Gerla, "TCP startup performance in large bandwidth networks," in *Proc. IEEE INFOCOM*, vol. 2, 2004, pp. 796–805.

[200] M. Mathis, J. Mahdavi, S. Floyd, and A. Romanow, "TCP selective acknowledgment options," IETF, RFC 2018, Oct. 1996.

[201] M. Podolsky, S. Floyd, J. Mahdavi, and M. Mathis, "An extension to the selective acknowledgement (SACK) option for TCP," IETF, RFC 2883, Jul. 2000. [Online]. Available: https://www.rfc-editor.org/info/rfc2883

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

[202] M. Mathis and J. Mahdavi, "Forward acknowledgement: Refining TCP congestion control," in *Proc. ACM SIGCOMM*, Palo Alto, CA, USA, 1996, pp. 281–291.

[203] R. Ludwig and R. H. Katz, "The Eifel algorithm: Making TCP robust against spurious retransmissions," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 30, no. 1, pp. 30–36, Jan. 2000.

[204] H. Balakrishnan, V. Padmanabhan, S. Seshan, M. Stemm, and R. Katz, "TCP behavior of a busy Internet server: Analysis and improvements," in *Proc. IEEE INFOCOM*, vol. 1, 1998, pp. 252–262.

[205] D. Lin and H. Kung, "TCP fast recovery strategies: Analysis and improvements," in *Proc. IEEE INFOCOM*, vol. 1, 1998, pp. 263–271.

[206] S. Floyd, H. Balakrishnan, and M. Allman, "Enhancing TCP's loss recovery using limited transmit," IETF, RFC 3042, Jan. 2001. [Online]. Available: https://www.rfc-editor.org/info/rfc3042

[207] J. Blanton, P. Hurtig, U. Ayesta, K. Avrachenkov, and M. Allman, "Early retransmit for TCP and stream control transmission protocol (SCTP)," IETF, RFC 5827, Apr. 2010. [Online]. Available: https://www.rfc-editor.org/info/rfc5827

[208] Y. Cheng, N. Cardwell, N. Dukkipati, and P. Jha, "The RACK-TLP loss detection algorithm for TCP," IETF, RFC 8985, Feb. 2021. [Online]. Available: https://www.rfc-editor.org/info/rfc8985

[209] M. Kojo, P. Sarolahti, K. Yamamoto, and M. Hata, "Forward RTO-recovery (F-RTO): An algorithm for detecting spurious retransmission timeouts with TCP," IETF, RFC 5682, Sep. 2009. [Online]. Available: https://www.rfc-editor.org/info/rfc5682

[210] E. Blanton and M. Allman, "Using TCP duplicate selective acknowledgement (DSACKs) and stream control transmission protocol (SCTP) duplicate transmission sequence numbers (TSNs) to detect spurious retransmissions," IETF, RFC 3708, Feb. 2004. [Online]. Available: https://www.rfc-editor.org/info/rfc3708

[211] D. Bansal and H. Balakrishnan, "Binomial congestion control algorithms," in *Proc. IEEE INFOCOM*, vol. 2, 2001, pp. 631–640.

[212] L. S. Brakmo and L. L. Peterson, "TCP Vegas: End to end congestion avoidance on a global Internet," *IEEE J. Sel. Areas Commun.*, vol. 13, no. 8, pp. 1465–1480, Oct. 1995.

[213] V. Arun, M. T. Arashloo, A. Saeed, M. Alizadeh, and H. Balakrishnan, "Toward formally verifying congestion control behavior," in *Proc. ACM SIGCOMM*, 2021, pp. 1–16.

[214] J. Mo, R. La, V. Anantharam, and J. Walrand, "Analysis and comparison of TCP Reno and Vegas," in *Proc. IEEE INFOCOM*, vol. 3, 1999, pp. 1556–1563.

[215] N. Jay, N. H. Rotman, P. B. Godfrey, M. Schapira, and A. Tamar, "Internet congestion control via deep reinforcement learning," 2018, *arxiv:1810.03259*.

[216] W. Li, F. Zhou, W. Meleis, and K. Chowdhury, "Learning-based and data-driven TCP design for memory-constrained IoT," in *Proc. Int. Conf. Distrib. Comput. Sensor Syst. (DCOSS)*, 2016, pp. 199–205.

[217] W. Li, F. Zhou, K. R. Chowdhury, and W. Meleis, "QTCP: Adaptive congestion control with reinforcement learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 6, no. 3, pp. 445–458, Jul./Sep. 2019.

[218] E. Khasina. "The CoCo-beholder: Enabling comprehensive evaluation of congestion control algorithms." 2019. [Online]. Available: http://arxiv.org/abs/1912.10531

[219] Y. Zaki, T. Pötsch, J. Chen, L. Subramanian, and C. Görg, "Adaptive congestion control for unpredictable cellular networks," *SIGCOMM Comput. Commun. Rev.*, vol. 45, no. 4, pp. 509–522, Aug. 2015.

[220] X. Li, F. Tang, J. Liu, L. T. Yang, L. Fu, and L. Chen, "AUTO: Adaptive congestion control based on multi-objective reinforcement learning for the satellite-ground integrated network," in *Proc. USENIX Annu. Technol. Conf. (USENIX ATC)*, Jul. 2021, pp. 611–624.

[221] E. D. Souza and D. Agarwal. "A highspeed TCP study: Characteristics and deployment issues." 2003. Accessed: Aug. 6, 2022. [Online]. Available: https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.135.5375

[222] J. A. Arokkiam, X. Wu, K. N. Brown, and C. J. Sreenan, "Experimental evaluation of TCP performance over 10 Gb/s passive optical networks (XG-PON)," in *Proc. IEEE Global Commun. Conf.*, 2014, pp. 2223–2228.

[223] H. Bullot, R. L. Cottrell, and R. Hughes-Jones, "Evaluation of advanced TCP stacks on fast long-distance production networks," *J. Grid Comput.*, vol. 1, no. 4, pp. 345–359, 2003.

[224] M. Allman, "Comments on bufferbloat," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 43, no. 1, pp. 30–37, 2012.

[225] K. Antonakoglou, X. Xu, E. Steinbach, T. Mahmoodi, and M. Dohler, "Toward haptic communications over the 5G tactile Internet," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 3034–3059, 4th Quart., 2018.

[226] F. H. Fitzek, S.-C. Li, S. Speidel, T. Strufe, M. Simsek, and M. Reisslein, *Tactile Internet: With Human-in-the-Loop*. London, U.K.: Academic, 2021.

[227] T. Hoeschele, C. Dietzel, D. Kopp, F. H. Fitzek, and M. Reisslein, "Importance of Internet exchange point (IXP) infrastructure for 5G: Estimating the impact of 5G use cases," *Telecommun. Policy*, vol. 45, no. 3, 2021, Art. no. 102091.

[228] J. Navarro-Ortiz, P. Romero-Diaz, S. Sendra, P. Ameigeiras, J. J. Ramos-Munoz, and J. M. Lopez-Soler, "A survey on 5G usage scenarios and traffic models," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 2, pp. 905–929, 2nd Quart., 2020.

[229] D. Zhang, J. J. P. C. Rodrigues, Y. Zhai, and T. Sato, "Design and implementation of 5G e-health systems: Technologies, use cases, and future challenges," *IEEE Commun. Mag.*, vol. 59, no. 9, pp. 80–85, Sep. 2021.

[230] K. Nichols and V. Jacobson, "Controlling queue delay," *ACM Queue*, vol. 10, no. 5, p. 20, May 2012.

[231] D. Scholz, B. Jaeger, L. Schwaighofer, D. Raumer, F. Geyer, and G. Carle, "Towards a deeper understanding of TCP BBR congestion control," in *Proc. IFIP Netw. Conf. Workshops*, 2018, pp. 1–9.

[232] S. Abbasloo, C.-Y. Yen, and H. J. Chao, "Classic meets modern: A pragmatic learning-based congestion control for the Internet," in *Proc. ACM SIGCOMM*, 2020, pp. 632–647.

[233] F. Kelly, G. Raina, and T. Voice, "Stability and fairness of explicit congestion control with small buffers," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 3, pp. 51–62, Jul. 2008.

[234] G. Raina, D. Towsley, and D. Wischik, "Part II: Control theory for buffer sizing," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 35, no. 3, pp. 79–82, Jul. 2005.

[235] M. Shifrin and I. Keslassy, "Small-buffer networks," *Comput. Netw.*, vol. 53, no. 14, pp. 2552–2565, Sep. 2009.

[236] G. Raina and D. Wischik, "Buffer sizes for large multiplexers: TCP queueing theory and instability analysis," in *Proc. IEE Next Gener. Internet Netw.*, Rome, Italy, 2005, pp. 173–180.

[237] Y. Kong, H. Zang, and X. Ma, "Improving TCP congestion control with machine intelligence," in *Proc. Workshop Netw. Meets AI ML*, 2018, pp. 60–66.

[238] N. Cardwell, Y. Cheng, and K. Yang. "BBRv2." Accessed: Jul. 25, 2024. [Online]. Available: https://datatracker.ietf.org/meeting/112/materials/slides-112-iccrg-bbrv2-update-00

[239] P. Bi, M. Xiao, D. Yu, and G. Zhang, "oBBR: Optimize retransmissions of BBR flows on the Internet," in *Proc. USENIX Annu. Technol. Conf. (USENIX ATC)*, Jul. 2023, pp. 537–551.

[240] L. Yu, J. Sonchack, and V. Liu, "Cebinae: Scalable in-network fairness augmentation," in *Proc. ACM SIGCOMM*, Amsterdam, The Netherlands, 2022, pp. 219–232.

[241] M. Shreedhar and G. Varghese, "Efficient fair queueing using deficit round robin," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 25, no. 4, pp. 231–242, Oct. 1995.

[242] T. Høiland-Jørgensen, P. McKenney, J. Gettys, and E. Dumazet, "The flow queue CoDel packet scheduler and active queue management algorithm," IETF, RFC 8290, Jan. 2018. [Online]. Available: https://www.rfc-editor.org/info/rfc8290

[243] D. L. Black and P. Jones, "Differentiated services (Diffserv) and real-time communication," IETF, RFC 7657, Nov. 2015. [Online]. Available: https://www.rfc-editor.org/info/rfc7657

[244] C. Hollot, V. Misra, D. Towsley, and W.-B. Gong, "On designing improved controllers for AQM routers supporting TCP flows," in *Proc. IEEE INFOCOM*, 2001, pp. 1726–1734.

[245] R. Pan et al., "PIE: A lightweight control scheme to address the bufferbloat problem," in *Proc. IEEE 14th Int. Conf. High Perform. Switch. Routing (HPSR)*, 2013, pp. 148–155.

[246] S. Athuraliya, S. Low, V. Li, and Q. Yin, "REM: Active queue management," *IEEE Netw.*, vol. 15, no. 3, pp. 48–53, May/Jun. 2001.

[247] O. Lhamo et al., "RED-SP-CoDel: Random early detection with static priority scheduling and controlled delay AQM in programmable data planes," *Comput. Commun.*, vol. 214, pp. 149–166, Jan. 2024.

[248] R. Pan, B. Prabhakar, and K. Psounis, "CHOKe—A stateless active queue management scheme for approximating fair bandwidth allocation," in *Proc. IEEE INFOCOM*, vol. 2, 2000, pp. 942–951.

[249] W.-C. Feng, D. Kandlur, D. Saha, and K. Shin, "Stochastic fair blue: A queue management algorithm for enforcing fairness," in *Proc. IEEE INFOCOM*, vol. 3, 2001, pp. 1520–1529.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

[250] T. Hoiland-Jorgensen, D. Täht, and J. Morton, "Piece of CAKE: A comprehensive queue management solution for home gateways," in *Proc. IEEE Int. Symp. Local Metropolitan Area Netw. (LANMAN)*, 2018, pp. 37–42.

[251] V. Arun, M. Alizadeh, and H. Balakrishnan, "Starvation in end-to-end congestion control," in *Proc. ACM SIGCOMM*, Amsterdam, The Netherlands, 2022, pp. 177–192.

[252] L. A. Grieco and S. Mascolo, "TCP westwood and easy RED to improve fairness in high-speed networks," in *Proc. IFIP/IEEE Int. Workshop Protocols High Speed Netw.*, 2002, pp. 130–146.

[253] L. Zhang, S. Shenker, and D. D. Clark, "Observations on the dynamics of a congestion control algorithm: The effects of two-way traffic," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 21, no. 4, pp. 133–147, Aug. 1991.

[254] J. C. Mogul, "Observing TCP dynamics in real networks," in *Proc. ACM SIGCOMM*, 1992, pp. 305–317.

[255] T. V. Lakshman, U. Madhow, and B. Suter, "Window-based error recovery and flow control with a slow acknowledgement channel: A study of TCP/IP performance," in *Proc. IEEE INFOCOM*, vol. 3, 1997, pp. 1199–1209.

[256] M. Sooriyabandara, G. Fairhurst, V. Padmanabhan, and H. Balakrishnan, "TCP performance implications of network path asymmetry," IETF, RFC 3449, Dec. 2002. [Online]. Available: https://rfc-editor.org/rfc/rfc3449.txt

[257] R. Netravali et al., "MAHIMAHI: Accurate record-and-replay for HTTP," in *Proc. USENIX Annu. Technol. Conf.*, 2015, pp. 417–429.

[258] L. Grieco and S. Mascolo, "End-to-end bandwidth estimation for congestion control in packet networks," in *Proc. 2nd Int. Workshop Qual. Service Multiservices IP Netw. (QoS-IP)*, 2003, pp. 645–658.

[259] D. Leith, L. Andrew, T. Quetchenbach, R. Shorten, and K. Lavi, "Experimental evaluation of delay/loss-based TCP congestion control algorithms," in *Proc. 6th Int. Workshop Protocols Fast Long-Distance Netw. (PFLDnet)*, 2008, pp. 1–6.

[260] L. A. Grieco and S. Mascolo, "Performance evaluation and comparison of Westwood+, New Reno, and Vegas TCP congestion control," *SIGCOMM Comput. Commun. Rev.*, vol. 34, no. 2, pp. 25–38, Apr. 2004.

[261] P. Kanuparthy and C. Dovrolis, "ShaperProbe: End-to-end detection of ISP traffic shaping using active methods," in *Proc. ACM Internet Meas. Conf.*, 2011, pp. 473–482.

[262] P. Goyal, A. Narayan, F. Cangialosi, S. Narayana, M. Alizadeh, and H. Balakrishnan, "Elasticity detection: A building block for Internet congestion control," in *Proc. ACM SIGCOMM*, 2022, pp. 158–176.

[263] D. Shan, P. Zhang, W. Jiang, H. Li, and F. Ren, "Towards the fairness of traffic Policer," in *Proc. IEEE INFOCOM*, 2021, pp. 1–10.

[264] G. Kumar et al., "SWIFT: Delay is simple and effective for congestion control in the datacenter," in *Proc. ACM SIGCOMM*, 2020, pp. 514–528.

[265] P. Dong, X. Lu, T. Huang, L. Chen, Y. Yang, and L. Zhang, "Predictive queue-based rate control for low latency in lossless data center networks," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 3428–3439, Jun. 2024.

[266] C. Luo, H. Gu, L. Zhu, and H. Zhang, "FlowStar: Fast convergence per-flow state accurate congestion control for InfiniBand," *IEEE/ACM Trans. Netw.*, vol. 32, no. 3, pp. 2662–2674, Jun. 2024.

[267] Y. Zhang, Q. Meng, C. Hu, and F. Ren, "Revisiting congestion control for lossless Ethernet," in *Proc. 21st USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2024, pp. 131–148.

[268] M. Alizadeh et al., "Data center TCP (DCTCP)," in *Proc. ACM SIGCOMM*, New Delhi, India, 2010, pp. 63–74.

[269] C. Wilson, H. Ballani, T. Karagiannis, and A. Rowtron, "Better never than late: Meeting deadlines in datacenter networks," in *Proc. ACM SIGCOMM*, 2011, pp. 50–61.

[270] M. Al-Fares, A. Loukissas, and A. Vahdat, "A scalable, commodity data center network architecture," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 4, pp. 63–74, Oct. 2008.

[271] A. Singh et al., "Jupiter rising: A decade of Clos topologies and centralized control in Google's datacenter network," in *Proc. ACM SIGCOMM*, 2015, pp. 183–197.

[272] Y. Zhu et al., "Congestion control for large-scale RDMA deployments," *SIGCOMM Comput. Commun. Rev.*, vol. 45, no. 4, pp. 523–536, Aug. 2015.

[273] P. X. Gao, A. Narayan, G. Kumar, R. Agarwal, S. Ratnasamy, and S. Shenker, "pHost: Distributed near-optimal datacenter transport over commodity network fabric," in *Proc. 11th ACM Conf. Emerg. Netw. Exp. Technol.*, 2015, pp. 1–12.

[274] L. Chen, K. Chen, W. Bai, and M. Alizadeh, "Scheduling mix-flows in commodity datacenters with Karuna," in *Proc. ACM SIGCOMM*, 2016, pp. 174–187.

[275] A. Saeed et al., "Annulus: A dual congestion control loop for datacenter and WAN traffic aggregates," in *Proc. ACM SIGCOMM*, 2020, pp. 735–749.

[276] V. Vasudevan et al., "Safe and effective fine-grained TCP retransmissions for datacenter communication," in *Proc. ACM SIGCOMM*, Barcelona, Spain, 2009, pp. 303–314.

[277] A. Phanishayee et al., "Measurement and analysis of TCP throughput collapse in cluster-based storage systems," in *Proc. USENIX Conf. File Storage Technol. (FAST)*, San Jose, CA, USA, 2008, pp. 175–188.

[278] Y. Chen, R. Griffith, J. Liu, R. H. Katz, and A. D. Joseph, "Understanding TCP incast throughput collapse in datacenter networks," in *Proc. 1st ACM Workshop Res. Enterprise Netw. (WREN)*, Barcelona, Spain, 2009, pp. 73–82.

[279] N. Finn. "802.1Qau—Congestion notification." 2011. [Online]. Available: https://1.ieee802.org/dcb/802-1qau/

[280] C. DeSanti. "802.1Qbb—Priority-based flow control." 2011. [Online]. Available: https://1.ieee802.org/dcb/802-1qbb/

[281] M. Alizadeh, B. Atikoglu, A. Kabbani, A. Lakshmikantha, R. Pan, and B. Prabhakar "Data center transport mechanisms: Congestion control theory and IEEE standardization," in *Proc. 46th IEEE Annu. Allerton Conf. Commun. Control Comput.*, 2008, pp. 1270–1277.

[282] A. Dragojević, D. Narayanan, M. Castro, and O. Hodson, "FaRM: Fast remote memory," in *Proc. 11th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, Apr. 2014, pp. 401–414.

[283] S. Agarwal, A. Krishnamurthy, and R. Agarwal, "Host congestion control," in *Proc. ACM SIGCOMM*, 2023, pp. 275–287.

[284] *RDMA over Converged Ethernet (RoCE), Architecture Specification, Volume 1, Release 1.2.2, Annex A16*, InfiniBand Trade Assoc., Beaverton, OR, USA, 2010.

[285] Y. Zhu, M. Ghobadi, V. Misra, and J. Padhye, "ECN or delay: Lessons learnt from analysis of DCQCN and TIMELY," in *Proc. ACM 12th Int. Conf. Emerg. Netw. Exp. Technol.*, 2016, pp. 313–327.

[286] T. Kim, D. M. Ng, J. Gong, Y. Kwon, M. Yu, and K. Park, "Rearchitecting the TCP stack for I/O-Offloaded content delivery," in *Proc. 20th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, Apr. 2023, pp. 275–292.

[287] N. Dukkipati, "Rate control protocol (RCP): Congestion control to make flows complete quickly," Ph.D. dissertation, Dept. Comput. Sci., Stanford Univ., Stanford, CA, USA, 2008.

[288] I. Cho, K. Jang, and D. Han, "Credit-scheduled delay-bounded congestion control for datacenters," in *Proc. ACM SIGCOMM*, 2017, pp. 239–252.

[289] M. Handley et al., "Re-architecting datacenter networks and stacks for low latency and high performance," in *Proc. ACM SIGCOMM*, 2017, pp. 29–42.

[290] B. Montazeri, Y. Li, M. Alizadeh, and J. Ousterhout, "HOMA: A receiver-driven low-latency transport protocol using network priorities," in *Proc. ACM SIGCOMM*, 2018, pp. 221–235.

[291] C.-Y. Hong, M. Caesar, and P. B. Godfrey, "Finishing flows quickly with preemptive scheduling," in *Proc. ACM SIGCOMM*, 2012, pp. 127–138.

[292] M. Alizadeh et al., "pFabric: Minimal near-optimal datacenter transport," *SIGCOMM Comput. Commun. Rev.*, vol. 43, no. 4, pp. 435–446, Aug. 2013.

[293] J. Perry, A. Ousterhout, H. Balakrishnan, D. Shah, and H. Fugal, "FastPass: A centralized 'zero-queue' datacenter network," in *Proc. ACM SIGCOMM*, 2014, pp. 307–318.

[294] M. P. Grosvenor et al., "Queues do not matter when you can JUMP them!" in *Proc. 12th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, May 2015, pp. 1–14.

[295] S. Arslan, Y. Li, G. Kumar, and N. Dukkipati, "BOLT: Sub-RTT congestion control for ultra-low latency," in *Proc. 20th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2023, pp. 219–236.

[296] W. Bai, K. Chen, S. Hu, K. Tan, and Y. Xiong, "Congestion control for high-speed extremely shallow-buffered datacenter networks," in *Proc. 1st ACM Asia–Pac. Workshop Netw. (APNet)*, Hong Kong, 2017, pp. 29–35.

[297] W. Cheng, K. Qian, W. Jiang, T. Zhang, and F. Ren, "Re-architecting congestion management in lossless Ethernet," in *Proc. 17th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, Feb. 2020, pp. 19–36.

[298] V. Addanki, M. Apostolaki, M. Ghobadi, S. Schmid, and L. Vanbever, "ABM: Active buffer management in datacenters," in *Proc. ACM SIGCOMM*, 2022, pp. 36–52.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

[299] T. R. Henderson and R. H. Katz, "Transport protocols for Internet-compatible satellite networks," *IEEE J. Sel. Areas Commun.*, vol. 17, no. 2, pp. 326–344, Feb. 1999.

[300] C. Caini and R. Firrincieli, "TCP Hybla: A TCP enhancement for heterogeneous networks," *Int. J. Satellite Commun. Netw.*, vol. 22, no. 5, pp. 547–566, Sep. 2004.

[301] C. Partridge and T. Shepard, "TCP/IP performance over satellite links," *IEEE Netw.*, vol. 11, no. 5, pp. 44–49, Sep./Oct. 1997.

[302] A. A. Hanbali, E. Altman, and P. Nain, "A survey of TCP over ad hoc networks," *IEEE Commun. Surveys Tuts.*, vol. 7, no. 3, pp. 22–36, 3rd Quart., 2005.

[303] B. Bakshi, P. Krishna, N. Vaidya, and D. Pradhan, "Improving performance of TCP over wireless networks," in *Proc. 17th IEEE Int. Conf. Distrib. Comput. Syst. (ICDCS)*, 1997, pp. 365–373.

[304] K. Chandran, S. Ragbunathan, S. Venkatesan, and R. Prakash, "A feedback based scheme for improving TCP performance in ad-hoc wireless networks," in *Proc. 18th IEEE Int. Conf. Distrib. Comput. Syst. (ICDCS)*, 1998, pp. 472–479.

[305] G. Holland and N. Vaidya, "Analysis of TCP performance over mobile ad hoc networks," in *Proc. ACM/IEEE MobiCom*, Seattle, WA, USA, 1999, pp. 219–230.

[306] J. Liu and S. Singh, "ATCP: TCP for mobile ad hoc networks," *IEEE J. Sel. Areas Commun.*, vol. 19, no. 7, pp. 1300–1315, Jul. 2001.

[307] Z. Fu, P. Zerfos, H. Luo, S. Lu, L. Zhang, and M. Gerla, "The impact of multihop wireless channel on TCP throughput and loss," in *Proc. IEEE INFOCOM*, vol. 3, 2003, pp. 1744–1753.

[308] D. Kim, C.-K. Toh, and Y. Choi, "TCP-BuS: Improving TCP performance in wireless ad hoc networks," *J. Commun. Netw.*, vol. 3, no. 2, pp. 1–12, 2001.

[309] K. Tan, F. Jiang, Q. Zhang, and X. Shen, "Congestion control in multihop wireless networks," *IEEE Trans. Veh. Technol.*, vol. 56, no. 2, pp. 863–873, Mar. 2007.

[310] R. Caceres and L. Iftode, "The effects of mobility on reliable transport protocols," in *Proc. 14th IEEE Int. Conf. Distrib. Comput. Syst. (ICDCS)*, 1994, pp. 12–20.

[311] H. Balakrishnan, S. Seshan, E. Amir, and R. H. Katz, "Improving TCP/IP performance over wireless networks," in *Proc. ACM MobiCom*, Berkeley, CA, USA, 1995, pp. 2–11.

[312] P. Goyal, A. Agarwal, R. Netravali, M. Alizadeh, and H. Balakrishnan, "ABC: A simple explicit congestion controller for wireless networks," in *Proc. 17th USENIX Conf. Netw. Syst. Design Implement. (NSDI)*, Santa Clara, CA, USA, 2020, pp. 353–372.

[313] Y. Horie, D. T. T. Hien, K. Nguyen, and H. Sekiya, "A comparison of congestion control algorithms in emulated Wi-Fi networks," in *Proc. Int. Conf. Inf. Commun. Technol. Converg. (ICTC)*, 2020, pp. 305–310.

[314] J. Zhang et al., "Bridging the gap between QoE and QoS in congestion control: A large-scale mobile Web service perspective," in *Proc. USENIX Annu. Techn. Conf. (USENIX ATC)*, Jul. 2023, pp. 553–569.

[315] T. Goff, J. Moronski, D. Phatak, and V. Gupta, "Freeze-TCP: A true end-to-end TCP enhancement mechanism for mobile environments," in *Proc. IEEE INFOCOM*, vol. 3, 2000, pp. 1537–1545.

[316] A. Bakre and B. Badrinath, "I-TCP: Indirect TCP for mobile hosts," in *Proc. 15th IEEE Int. Conf. Distrib. Comput. Syst.*, 1995, pp. 136–143.

[317] A. DeSimone, M. C. Chuah, and O.-C. Yue, "Throughput performance of transport-layer protocols over wireless LANs," in *Proc. IEEE GLOBECOM*, 1993, pp. 542–549.

[318] Y. Tian, K. Xu, and N. Ansari, "TCP in wireless environments: Problems and solutions," *IEEE Commun. Mag.*, vol. 43, no. 3, pp. S27–S32, Mar. 2005.

[319] C. P. Fu and S. C. Liew, "TCP Veno: TCP enhancement for transmission over wireless access networks," *IEEE J. Sel. Areas Commun.*, vol. 21, no. 2, pp. 216–228, Feb. 2003.

[320] T. Li et al., "TACK: Improving wireless transport performance by taming acknowledgments," in *Proc. ACM SIGCOMM*, 2020, pp. 15–30.

[321] J. Huang et al., "An in-depth study of LTE: Effect of network protocol and application behavior on performance," in *Proc. ACM SIGCOMM*, Hong Kong, 2013, pp. 363–374.

[322] A. Ahmed, A. Al-Dweik, Y. Iraqi, H. Mukhtar, M. Naeem, and E. Hossain, "Hybrid automatic repeat request (HARQ) in wireless communications systems and standards: A contemporary survey," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 4, pp. 2711–2752, 4th Quart., 2021.

[323] F. Gabriel, S. Wunderlich, S. Pandi, F. H. Fitzek, and M. Reisslein, "Caterpillar RLNC with feedback (CRLNC-FB): Reducing delay in selective repeat ARQ through cooding," *IEEE Access*, vol. 6, pp. 44787–44802, 2018.

[324] H. Jiang, Y. Wang, K. Lee, and I. Rhee, "Tackling bufferbloat in 3G/4G networks," in *Proc. ACM Internet Meas. Conf. (IMC)*, Boston, MA, USA, 2012, pp. 329–342.

[325] H. Jiang, Z. Liu, Y. Wang, K. Lee, and I. Rhee, "Understanding bufferbloat in cellular networks," in *Proc. ACM SIGCOMM Workshop Cellular Netw. Oper. Challenges Future Design*, 2012, pp. 1–6.

[326] I. Johansson, "Self-clocked rate adaptation for conversational video in LTE," in *Proc. ACM SIGCOMM Workshop Capacity Sharing Workshop (CSWS)*, Chicago, IL, USA, 2014, pp. 51–56.

[327] S. Abbasloo, T. Li, Y. Xu, and H. J. Chao. "Cellular controlled delay TCP (C2TCP)." 2018. [Online]. Available: http://arxiv.org/abs/1807.02689

[328] K. Winstein, A. Sivaraman, and H. Balakrishnan, "Stochastic forecasts achieve high throughput and low delay over cellular networks," in *Proc. USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2013, pp. 459–471.

[329] C.-Y. Yen, S. Abbasloo, and H. J. Chao, "Computers can learn from the heuristic designs and master Internet congestion control," in *Proc. ACM SIGCOMM*, 2023, pp. 255–274.

[330] S. Mangiante et al., "Congestion control for future mobile networks," in *Proc. 13th ACM Workshop Challenged Netw.*, 2018, pp. 55–61.

[331] S.-C. Tsao, Y.-C. Lai, and Y.-D. Lin, "Taxonomy and evaluation of TCP-friendly congestion-control schemes on fairness, aggressiveness, and responsiveness," *IEEE Netw.*, vol. 21, no. 6, pp. 6–15, Nov./Dec. 2007.

[332] R. Ware, M. K. Mukerjee, S. Seshan, and J. Sherry, "Modeling BBR's interactions with loss-based congestion control," in *Proc. ACM Internet Meas. Conf. (IMC)*, Amsterdam, The Netherlands, 2019, pp. 137–143.

[333] A. Agarwal, V. Arun, D. Ray, R. Martins, and S. Seshan, "Towards provably performant congestion control," in *Proc. 21st USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2024, pp. 951–978.

[334] P. G. Dhawane, P. Navghare, B. Manjre, P. Dhule, P. Fale, and M. Kahile, "Wireless TCP congestion control based on loss discrimination approach using machine learning," in *Proc. Int. Conf. Commun. Cyber-Phys. Eng.*, 2024, pp. 343–349.

[335] L. Jia, C. Zhou, T. Huang, C. Li, and L. Sun, "Meet challenges of RTT jitter, a hybrid Internet congestion control algorithm," in *Proc. ACM Web Conf.*, 2024, pp. 2768–2776.

[336] J. P. A. León, L. J. de la Cruz Llopis, and F. J. Rico-Novella, "A machine learning based distributed congestion control protocol for multi-hop wireless networks," *Comput. Netw.*, vol. 231, Jul. 2023, Art. no. 109813.

[337] H. Li and L. Duan, "Distributed learning for dynamic congestion games," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, 2024, pp. 3654–3659.

[338] X. Liao, H. Tian, C. Zeng, X. Wan, and K. Chen, "Astraea: Towards fair and efficient learning-based congestion control," in *Proc. 19th ACM Eur. Conf. Comput. Syst. (EuroSys)*, 2024, pp. 99–114.

[339] S. R. Pokhrel, J. Kua, D. Satish, S. Ozer, J. Howe, and A. Walid, "DDPG-MPCC: An experience driven multipath performance oriented congestion control," *Future Internet*, vol. 16, no. 2, p. 37, 2024.

[340] S. Rajasekaran, S. Narang, A. A. Zabreyko, and M. Ghobadi, "MLTCP: A distributed technique to approximate centralized flow scheduling for machine learning," in *Proc. 23rd ACM Workshop Hot Topics Netw. (HotNets)*, 2024, pp. 167–176.

[341] Z. Xia et al., "*Glider*: Rethinking congestion control with deep reinforcement learning," *World Wide Web*, vol. 26, no. 1, pp. 115–137, 2023.

[342] W. Yang, Y. Liu, C. Tian, J. Jiang, and L. Guo, "*Gemini*: Divide-and-conquer for practical learning-based Internet congestion control," in *Proc. IEEE INFOCOM*, 2023, pp. 1–10.

[343] BBR team at Google. "Transperf." Accessed: Nov. 21, 2024. [Online]. Available: https://github.com/google/transperf

[344] Z. Alharbi, A. S. Thyagaturu, M. Reisslein, H. ElBakoury, and R. Zheng, "Performance comparison of R-PHY and R-MACPHY modular cable access network architectures," *IEEE Trans. Broadcast.*, vol. 64, no. 1, pp. 128–145, Mar. 2018.

[345] D. Shan et al., "Enforcing fairness in the traffic policer among heterogeneous congestion control algorithms," *IEEE/ACM Trans. Netw.*, vol. 32, no. 1, pp. 34–49, Feb. 2024.

[346] G. Xie, Y. Zhang, N. Chen, and W. Chang, "A high-flexibility CAN-TSN gateway with a low-congestion TSN-to-CAN scheduler," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 42, no. 12, pp. 5072–5083, Dec. 2023.

[347] W. Yan, D. Wei, B. Fu, R. Li, and G. Xie, "A mixed-criticality traffic scheduler with mitigating congestion for CAN-to-TSN gateway," *ACM Trans. Design Autom. Elect. Syst.*, vol. 29, no. 5, pp. 1–28, 2024.

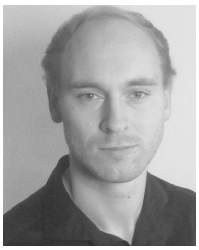
Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

[348] A. M. Kentis, M. S. Berger, and J. Soler, "Effects of port congestion in the gate control list scheduling of time sensitive networks," in *Proc. 8th Int. Conf. Netw. Future (NOF)*, 2017, pp. 138–140.
[349] D. Andronovici, I. Turcanu, J. Bigge, and C. Sommer, "Cross-validating open source in-vehicle TSN simulation models with a COTS hardware testbed," in *Proc. IEEE Veh. Netw. Conf. (VNC)*, 2024, pp. 172–179.
[350] M. Bosk et al., "Methodology and infrastructure for TSN-based reproducible network experiments," *IEEE Access*, vol. 10, pp. 109203–109239, 2022.
[351] H.-H. Liu et al., "Improving TSN simulation accuracy in OMNeT++: A hardware-aligned approach," *IEEE Access*, vol. 12, pp. 79937–79956, 2024.
[352] F. Rezabek, M. Bosk, L. Seidlitz, J. Ott, and G. Carle, "Context matters: Lessons learned from emulated and simulated TSN environments," in *Proc. IEEE Int. Conf. Pervasive Compu. Commun. Workshops (PerCom Workshops)*, 2024, pp. 499–504.
[353] M. Ulbricht et al., "TSN-FlexTest: Flexible TSN measurement testbed," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 2, pp. 1387–1402, Apr. 2024.
[354] N. McKeown et al., "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, Mar. 2008.
[355] A. Sapio, I. Abdelaziz, A. Aldilaijan, M. Canini, and P. Kalnis, "In-network computation is a dumb idea whose time has come," in *Proc. 16th ACM Workshop Hot Topics Netw. (HotNets)*, 2017, pp. 150–156.
[356] Ö. Aslan, S. S. Aktuğ, M. Ozkan-Okay, A. A. Yilmaz, and E. Akin, "A comprehensive review of cyber security vulnerabilities, threats, attacks, and solutions," *Electronics*, vol. 12, no. 6, p. 1333, 2023.
[357] A. Cirne, P. R. Sousa, J. S. Resende, and L. Antunes, "Hardware security for Internet of Things identity assurance," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 2, pp. 1041–1079, 2nd Quart., 2024.
[358] M. Mahyoub, A. AbdulGhaffar, E. Alalade, E. Ndubisi, and A. Matrawy, "Security analysis of critical 5G interfaces," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 4, pp. 2382–2410, 4th Quart., 2024.
[359] O. Obi et al., "Comprehensive review on cybersecurity: Modern threats and advanced defense strategies," *Comput. Sci. IT Res. J.*, vol. 5, no. 2, pp. 293–310, 2024.
[360] J. Gharat, A. Vidhate, and A. Barve, "Performance analysis of unified threat management (UTM)," in *Proc. Invent. Commun. Comput. Technol. (ICICCT)*, 2020, pp. 121–133.
[361] K. Tam, M. H. Salvador, K. McAlpine, R. Basile, B. Matsugu, and J. More, "Chapter 1—Introduction to UTM (unified threat management)," in *UTM Security With Fortinet*. Oxford, U.K.: Syngress, 2013, pp. 3–34.
[362] A. Siddiqui, B. P. Rimal, M. Reisslein, and Y. Wang, "Survey on unified threat management (UTM) systems for home networks," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 4, pp. 2459–2509, 4th Quart., 2024.
[363] A. Siddiqui, B. P. Rimal, M. Reisslein, D. Gc, and Y. Wang, "SUTMS: Designing a unified threat management system for home networks," *IEEE Access*, vol. 12, pp. 80930–80949, 2024.
[364] J. Chen, X. Liang, J. Xue, Y. Sun, H. Zhou, and X. Shen, "Evolution of RAN architectures towards 6G: Motivation, development, and enabling technologies," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 3, pp. 1950–1988, 3rd Quart., 2024.
[365] X. You et al., "Towards 6G wireless communication networks: Vision, enabling technologies, and new paradigm shifts," *Sci. China Inf. Sci.*, vol. 64, pp. 1–74, Feb. 2021.
[366] C.-X. Wang et al., "On the road to 6G: Visions, requirements, key technologies, and testbeds," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 2, pp. 905–974, 2nd Quart., 2023.
[367] Telesat. "Telesat lightspeed." Accessed: Jun. 27, 2024. [Online]. Available: https://www.telesat.com/leo-satellites/
[368] SpaceX. "Starlink." Accessed: Jun. 27, 2024. [Online]. Available: https://www.starlink.com/
[369] Amazon. "Project Kuiper." Accessed: Jun. 27, 2024. [Online]. Available: https://www.aboutamazon.com/what-we-do/devices-services/project-kuiper
[370] O. Kodheli et al., "Satellite communications in the new space era: A survey and future challenges," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 1, pp. 70–109, 1st Quart., 2021.
[371] S. Mahboob and L. Liu, "Revolutionizing future connectivity: A contemporary survey on ai-empowered satellite-based non-terrestrial networks in 6G," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 2, pp. 1279–1321, 2nd Quart., 2024.
[372] S. R. Pokhrel and J. Choi, "Data-driven satellite communication and control for future IoT: Principles and opportunities," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 60, no. 3, pp. 3307–3318, Jun. 2024.
[373] S. Salim, N. Moustafa, and M. Reisslein, "Cybersecurity of satellite communications systems: A comprehensive survey of the space, ground, and links segments," *IEEE Commun. Surveys Tuts.*, early access, Jun. 3, 2024, doi: 10.1109/COMST.2024.3408277.
[374] W. Yang, L. Cai, S. Shu, and J. Pan, "Mobility-aware congestion control for multipath QUIC in integrated terrestrial satellite networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 11620–11634, Dec. 2024.
[375] D. Bhattacherjee et al., "Gearing up for the 21st century space race," in *Proc. 17th ACM Workshop Hot Topics Netw.*, 2018, pp. 113–119.
[376] D. Ramos, O. Esparza, J. Mata-Díaz, and J. Alins, "Evaluation of TCP congestion control algorithms with traffic control policies in a PEP-based geosynchronous satellite scenario," *Comput. Netw.*, vol. 239, Feb. 2024, Art. no. 110131.
[377] S. Utsumi, S. M. S. Zabir, and N. Shiratori, "TCP-Cherry: A new approach for TCP congestion control over satellite IP networks," *Comput. Commun.*, vol. 31, no. 10, pp. 2541–2561, 2008.
[378] M. A. Kachooei, J. Chung, F. Li, B. Peters, J. Chung, and M. Claypool, "Improving TCP slow start performance in wireless networks with search," in *Proc. 25th IEEE Int. Symp. World Wireless Mobile Multimedia Netw. (WoWMoM)*, 2024, pp. 1–10.
[379] S. Kassing, D. Bhattacherjee, A. B. Águas, J. E. Saethre, and A. Singla, "Exploring the 'Internet from space' with Hypatia," in *Proc. ACM Internet Meas. Conf. (IMC)*, 2020, pp. 214–229.
[380] Y. Chen, H. Shi, Q. Weng, and Z. Shi, "Congestion control design of multicast QUIC based on reinforcement learning," in *Proc. IEEE Int. Conf. Ubiquitous Commun.*, 2023, pp. 232–236.
[381] X. Jiang et al., "Spatio-temporal routing, redundant coding and multipath scheduling for deterministic satellite network transmission," *IEEE Trans. Commun.*, vol. 71, no. 5, pp. 2860–2875, May 2023.
[382] Y. Ni et al., "CellFusion: Multipath vehicle-to-cloud video streaming with network coding in the wild," in *Proc. ACM SIGCOMM*, 2023, pp. 668–683.
[383] Z. Deng, Y. Liu, J. Liu, A. Argyriou, and D. Liu, "BBR-based and fairness-guaranteed congestion control and packet scheduling for MPQUIC over heterogeneous networks," *Comput. Commun.*, vol. 224, pp. 213–224, Aug. 2024.
[384] Y. Li and B.-H. Oh, "MPTCP-based link congestion detection and traffic control scheme in software-defined networks," *ICT Exp.*, vol. 10, no. 4, pp. 735–746, 2024.
[385] I. Mahmud, T. Lubna, Y.-J. Song, and Y.-Z. Cho, "Coupled multipath BBR (C-MPBBR): A efficient congestion control algorithm for multipath TCP," *IEEE Access*, vol. 8, pp. 165497–165511, 2020.
[386] W. Wei, K. Xue, J. Han, D. S. Wei, and P. Hong, "Shared bottleneck-based congestion control and packet scheduling for multipath TCP," *IEEE/ACM Trans. Netw.*, vol. 28, no. 2, pp. 653–666, Apr. 2020.
[387] H. Yu et al., "When classic meets intelligence: A hybrid multipath congestion control framework," *IEEE/ACM Trans. Netw.*, vol. 32, no. 4, pp. 3575–3590, Aug. 2024.
[388] R. Zhuang, J. Han, K. Xue, J. Li, Q. Sun, and J. Lu, "ProactMP: A proactive multipath transport protocol for low-latency datacenters," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 4, pp. 3919–3932, Aug. 2024.
[389] Y. R. Julio, I. G. García, and J. M. Dí az, "Fulcrum rateless multicast distributed coding design," *IEEE Access*, vol. 11, pp. 73839–73849, 2023.
[390] S. Pan and X. M. Zhang, "Cooperative gigabit content distribution with network coding for mmWave vehicular networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 2, pp. 1863–1877, Feb. 2023.
[391] A. Masood et al., "A review on AI-enabled congestion control schemes for content centric networks," in *Proc. 14th IEEE Int. Conf. Inf. Commun. Technol. Converg. (ICTC)*, 2023, pp. 659–662.
[392] D. Yuan et al., "Context-aware cross-layer congestion control for large-scale live streaming," *IEEE/ACM Trans. Netw.*, vol. 32, no. 5, pp. 3743–3759, Oct. 2024.
[393] E. Garro et al., "5G mixed mode: NR multicast-broadcast services," *IEEE Trans. Broadcast.*, vol. 66, no. 2, pp. 390–403, Jun. 2020.
[394] M. Zimmerling, L. Mottola, and S. Santini, "Synchronous transmissions in low-power wireless: A survey of communication protocols and network services," *ACM Comput. Surveys*, vol. 53, no. 6, pp. 121:1–121:39, 2020.
[395] V. Bioglio, C. Condo, and I. Land, "Design of polar codes in 5G new radio," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 1, pp. 29–40, 1st Quart., 2021.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.

[396] Z. B. K. Egilmez, L. Xiang, R. G. Maunder, and L. Hanzo, "The development, operation and performance of the 5G polar codes," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 1, pp. 96–122, 1st Quart., 2020.
[397] J. Choi, "Sliding network coding for URLLC," *IEEE Trans. Wireless Commun.*, vol. 21, no. 6, pp. 4424–4433, Jun. 2022.
[398] E. Dias et al., "Sliding window network coding enables next generation URLLC millimeter-wave networks," *IEEE Netw. Lett.*, vol. 5, no. 3, pp. 159–163, Sep. 2023.
[399] P. Enenche, D. H. Kim, and D. You, "Network coding as enabler for achieving URLLC Under TCP and UDP environments: A survey," *IEEE Access*, vol. 11, pp. 76647–76674, 2023.
[400] M. Centenaro, C. E. Costa, F. Granelli, C. Sacchi, and L. Vangelista, "A survey on technologies, standards and open challenges in satellite IoT," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 3, pp. 1693–1720, 3rd Quart., 2021.
[401] R. Bassoli, H. Marques, J. Rodriguez, K. W. Shum, and R. Tafazolli, "Network coding theory: A survey," *IEEE Commun. Surveys Tuts.*, vol. 15, no. 4, pp. 1950–1978, 4th Quart., 2013.
[402] M. Z. Farooqi, S. M. Tabassum, M. H. Rehmani, and Y. Saleem, "A survey on network coding: From traditional wireless networks to emerging cognitive radio networks," *J. Netw. Comput. Appl.*, vol. 46, pp. 166–181, Nov. 2014.
[403] D. E. Lucani et al., "Fulcrum: Flexible network coding for heterogeneous devices," *IEEE Access*, vol. 6, pp. 77890–77910, 2018.
[404] A. Naeem, M. H. Rehmani, Y. Saleem, I. Rashid, and N. Crespi, "Network coding in cognitive radio networks: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 3, pp. 1945–1973, 3rd Quart., 2017.
[405] V. Nguyen, E. Tasdemir, G. T. Nguyen, D. E. Lucani, F. H. Fitzek, and M. Reisslein, "DSEP Fulcrum: Dynamic sparsity and expansion packets for Fulcrum network coding," *IEEE Access*, vol. 8, pp. 78293–78314, 2020.
[406] C.-C. Chen, C. Chen, S. Y. Oh, J.-S. Park, M. Gerla, and M. Sanadidi, "ComboCoding: Combined intra-/inter-flow network coding for TCP over disruptive MANETs," *J. Adv. Res.*, vol. 2, no. 3, pp. 241–252, 2011.
[407] J. Cloud, F. D. P. Calmon, W. Zeng, G. Pau, L. M. Zeger, and M. Médard, "Multi-path TCP with network coding for mobile devices in heterogeneous networks," in *Proc. IEEE 78th Veh. Technol. Conf. (VTC Fall)*, 2013, pp. 1–5.
[408] M. Li, A. Lukyanenko, and Y. Cui, "Network coding based multipath TCP," in *Proc. IEEE INFOCOM Workshops*, 2012, pp. 25–30.
[409] M. Ouyang et al., "Network coding-based multipath transmission for LEO satellite networks with domain cluster," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21659–21673, Jun. 2024.
[410] J. K. Sundararajan, D. Shah, M. Médard, S. Jakubczak, M. Mitzenmacher, and J. Barros, "Network coding meets TCP: Theory and implementation," *Proc. IEEE*, vol. 99, no. 3, pp. 490–512, Mar. 2011.
[411] A. Akella, S. Seshan, and A. Shaikh, "An empirical evaluation of wide-area Internet bottlenecks," in *Proc. ACM Int. Conf. Meas. Model. Comput. Syst.*, 2003, pp. 316–317.
[412] D. Genin and J. Splett. "Where in the Internet is congestion?" 2013. [Online]. Available: https://arxiv.org/abs/1307.3696
[413] A. Dhamdhere et al., "Inferring persistent interdomain congestion," in *Proc. ACM SIGCOMM*, 2018, pp. 1–15.
[414] "DPDK." Accessed: Aug. 23, 2024. [Online]. Available: https://www.dpdk.org
[415] J. EunYoung et al., "mTCP: A highly scalable user-level TCP stack for multicore systems," in *Proc. 11th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, Apr. 2014, pp. 489–502.
[416] N. Dukkipati et al., "An argument for increasing TCP's initial congestion window," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 40, no. 3, pp. 26–33, Jul. 2010.
[417] Y. Cheng and N. Cardwell, "Making Linux TCP fast," in *Proc. NetDev Conf.*, Oct. 2016, pp. 1–5.
[418] E. Sharafzadeh, S. Abdous, and S. Ghorbani, "Understanding the impact of host networking elements on traffic bursts," in *Proc. 20th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, Apr. 2023, pp. 237–254.
[419] J. Corbet. "TSO sizing and the FQ scheduler." Accessed: Jul. 18, 2024. [Online]. Available: https://lwn.net/Articles/564978/
[420] J. Corbet. "Network transmit queue limits." Accessed: Jul. 18, 2024. [Online]. Available: https://lwn.net/Articles/454390/
[421] J. Corbet. "TCP small queues." Accessed: Jul. 18, 2024. [Online]. Available: https://lwn.net/Articles/507065/
[422] B. Elixir. "Linux v6.9.7 tcp_congestion_ops." Accessed: Jun. 27, 2024. [Online]. Available: https://elixir.bootlin.com/linux/v6.9.7/source/include/net/tcp.h
[423] P. Sarolahti and A. Kuznetsov, "Congestion control in Linux TCP," in *Proc. USENIX Annu. Technol. Conf.*, 2002, pp. 49–62.
[424] S. Arianfar, "TCP's congestion control implementation in Linux kernel," in *Proc. Netw. Protocols Oper. SysT.*, 2013, pp. 16–21.
[425] H. Balakrishnan, H. S. Rahul, and S. Seshan, "An integrated congestion management architecture for Internet hosts," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 29, no. 4, pp. 175–187, Aug. 1999.
[426] A. Narayan et al., "Restructuring endpoint congestion control," in *Proc. ACM SIGCOMM*, 2018, pp. 30–43.
[427] S. Amante, J. Rajahalme, B. E. Carpenter, and S. Jiang, "IPv6 flow label specification," IETF, RFC 6437, Nov. 2011. [Online]. Available: https://www.rfc-editor.org/info/rfc6437
[428] V. Frost and B. Melamed, "Traffic modeling for telecommunications networks," *IEEE Commun. Mag.*, vol. 32, no. 3, pp. 70–81, Mar. 1994.
[429] V. Paxson and S. Floyd, "Wide area traffic: The failure of Poisson modeling," *IEEE/ACM Trans. Netw.*, vol. 3, no. 3, pp. 226–244, Jun. 1995.
[430] W. E. Leland, M. S. Taqqu, W. Willinger, and D. V. Wilson, "On the self-similar nature of Ethernet traffic (extended version)," *IEEE/ACM Trans. Netw.*, vol. 2, no. 1, pp. 1–15, Feb. 1994.
[431] Hewlett-Packard Company. "Netperf." Accessed: Jul. 18, 2024. [Online]. Available: https://hewlettpackard.github.io/netperf/
[432] Tcpdump Group. "tcpdump." Accessed: Jul. 31, 2024. [Online]. Available: https://www.tcpdump.org
[433] T. Lan, D. Kao, M. Chiang, and A. Sabharwal, "An axiomatic theory of fairness in network resource allocation," in *Proc. IEEE INFOCOM*, Mar. 2010, pp. 1–9.
[434] M. Uchida and J. Kurose, "An information-theoretic characterization of weighted alpha-proportional fairness," in *Proc. IEEE INFOCOM*, Apr. 2009, pp. 1053–1061.
[435] J. Mo and J. Walrand, "Fair end-to-end window-based congestion control," *IEEE/ACM Trans. Netw.*, vol. 8, no. 5, pp. 556–567, Oct. 2000.
[436] K. Bharath-Kumar and J. Jaffe, "A new approach to performance-oriented flow control," *IEEE Trans. Commun.*, vol. C-29, no. 4, pp. 427–435, Apr. 1981.
[437] B. Briscoe, "Flow rate fairness: Dismantling a religion," *SIGCOMM Comput. Commun. Rev.*, vol. 37, no. 2, pp. 63–74, Apr. 2007.
[438] S. Shalunov, G. Hazel, J. Iyengar, and M. Kühlewind, "Low extra delay background transport (LEDBAT)," IETF, RFC 6817, Dec. 2012. [Online]. Available: https://rfc-editor.org/rfc/rfc6817.txt
[439] M. Bagnulo, A. Garcia-Martinez, G. Montenegro, and P. Balasubramanian, "rLEDBAT: Receiver-driven low extra delay background transport for TCP," Internet Eng. Task Force, Fremont, CA, USA, Jun. 2024. [Online]. Available: https://datatracker.ietf.org/doc/draft-irtf-iccrg-rledbat/08/
[440] A. Venkataramani, R. Kokku, and M. Dahlin, "TCP Nice: A mechanism for background transfers," *SIGOPS Oper. Syst. Rev.*, vol. 36, no. SI, pp. 329–343, Dec. 2003.
[441] T. Meng, N. R. Schiff, P. B. Godfrey, and M. Schapira, "PCC proteus: Scavenger transport and beyond," in *Proc. ACM SIGCOMM*, 2020, pp. 615–631.
[442] S. Rajasekaran, M. Ghobadi, G. Kumar, and A. Akella, "Congestion control in machine learning clusters," in *Proc. 21st ACM Workshop Hot Topics Netw.*, 2022, pp. 235–242.
[443] B. E. Carpenter, "Architectural principles of the Internet," IETF, RFC 1958, Jun. 1996. [Online]. Available: https://www.rfc-editor.org/info/rfc1958

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.



Christian L. Vielhaus received the Diploma (Dipl.-Ing.) degree in electrical engineering from Technical University Dresden, Germany, in 2019, where he is currently pursuing the Ph.D. degree in electrical engineering with the Deutsche Telecom Chair of Communication Networks. His research interests include congestion control, machine learning, and cellular networks.

Caspar von Lengerke (Graduate Student Member, IEEE) received the bachelor's and master's degrees in electrical engineering from RWTH Aachen University, Germany, in 2017 and 2019, respectively. He joined the Deutsche Telekom Chair of Communication Networks, TU Dresden in 2021. His research interests include goal-oriented communication, message identification, and their role in next-generation communication networks.

Vincent Latzko received the Diploma degree in electrical engineering from Technische Universität Darmstadt, Germany. He joined the Deutsche Telekom Chair of Communication Networks, TU Dresden in 2017. His research interests include efficient compression, representation, and scaling of cloud resources under quantified uncertainty.

Justus Rischke received the Diploma (Dipl.-Ing.) degree in electrical engineering and the Ph.D. degree from the Deutsche Telekom Chair of Communication Networks, from Technical University Dresden Dresden, Germany, in 2017 and 2023, respectively. His research focuses on 5G measurements and reliable low-latency communications.

Martin Reisslein (Fellow, IEEE) received the Ph.D. degree in systems engineering from the University of Pennsylvania, Philadelphia, PA, USA, in 1998. He is currently a Professor of Electrical Engineering and the Program Chair for Computer Engineering with the School of Electrical, Computer, and Energy Engineering, Arizona State University, Tempe, AZ, USA. He is currently an Associate Editor for IEEE ACCESS and IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT. He currently serves an Area Editor for Optical Communications for the IEEE COMMUNICATIONS SURVEYS AND TUTORIALS and as the Co-Editor-in-Chief of *Optical Switching and Networking*.

Frank H. P. Fitzek (Fellow, IEEE) received the Diploma (Dipl.-Ing.) degree in electrical engineering from the University of Technology Rheinisch-Westfälische Technische Hochschule, Aachen, Germany, in 1997, the Ph.D. (Dr.-Ing.) degree in electrical engineering from the Technical University Berlin, Germany, in 2002, and the Honorary degree (Doctor Honoris Causa) from the Budapest University of Technology and Economics in 2015. He is currently a Professor and also the Head of the Deutsche Telekom Chair of Communication Networks, Technische Universität Dresden, Germany, where he coordinates the 5G Lab Germany.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 13,2026 at 13:18:36 UTC from IEEE Xplore. Restrictions apply.